C# & Software Engineering Field Guide

Fundamentals, design patterns, system design, and the practical stuff in between

Written for a working .NET engineer who wants to refresh the basics: simple explanations first, then runnable-style examples, then practice questions.

C# / .NET / ASP.NET Core / EF Core / React / SQL / DevOps

Contents

1. The .NET Landscape 10

C#, .NET, the CLR: who is who? . . . 10
What happens when you run `dotnet run` . . . 10
Anatomy of a solution . . . 11
A minimal .csproj . . . 11
The dotnet CLI cheat sheet . . . 11
Top-level statements and the entry point . . . 12
Namespaces and using directives . . . 12
Assemblies and access modifiers . . . 12
Debug vs Release, and what 'framework-dependent' means . . . 13
LTS vs STS . . . 13

2. C# Language Fundamentals 14

Variables and built-in types . . . 14
Value types vs reference types . . . 14
Stack vs heap, simply . . . 15
Boxing and unboxing . . . 15
Strings . . . 15
Control flow . . . 16
Methods . . . 16
Arrays and the main collections . . . 17
Interfaces for collections . . . 18
Null and nullable reference types . . . 18
Enums . . . 18
Equality: == vs Equals vs ReferenceEquals . . . 19
Operators worth remembering . . . 19
Practice questions . . . 19

3. Object-Oriented Programming in C# 20

Classes and objects . . . 20
Pillar 1: Encapsulation . . . 20
Property flavors . . . 21
Constructors, initializers, primary constructors . . . 21
Pillar 2: Inheritance . . . 21

Pillar 3: Polymorphism . . . 22
Pillar 4: Abstraction and interfaces . . . 22
Composition over inheritance . . . 23
Structs . . . 23
Records . . . 24
Static, sealed, partial, readonly . . . 24
Generics . . . 24
Delegates, events, and lambdas . . . 25
Extension methods . . . 25
Operator overloading and indexers (occasionally useful) . . . 25
Practice questions . . . 26

4. Modern C# Features You Should Know 27

Pattern matching . . . 27
Switch expressions with tuples . . . 27
Collection expressions and spread (C# 12) . . . 28
Tuples and deconstruction . . . 28
Local functions and static lambdas . . . 28
Target-typed new, init-only, required members . . . 28
Nullable flow analysis helpers . . . 28
Ranges and indices . . . 29
Using declarations and `using` statements . . . 29
Iterators: `yield return` . . . 29
Attributes and reflection basics . . . 29
Generic math and `INumber` (just so you recognize it) . . . 30
Exceptions: the essentials . . . 30
The Result pattern: errors as values . . . 30
Dates and times: the traps . . . 31
File I/O and JSON . . . 31
Regular expressions (the 30-second version) . . . 31

5. LINQ and Working with Data 32

The mental model . . . 32
Core operators . . . 32
GroupBy in practice . . . 33
Joins . . . 33
Query syntax vs method syntax . . . 34

Deferred execution: the classic gotcha . . . 34
IEnumerable vs IQueryable . . . 34
Performance habits . . . 34
Immutability and `IReadOnly*` . . . 35
Sorting and comparing . . . 35
Hashing and why GetHashCode matters . . . 35
Practice . . . 35

6. Async, Threads, and Concurrency 37

Why async exists . . . 37
Task and Task . . . 37
What the compiler does . . . 37
Rules of thumb . . . 38
CancellationToken . . . 38
Running things concurrently . . . 38
Exceptions in async code . . . 39
ConfigureAwait(false) and SynchronizationContext . . . 39
Threads, thread pool, and parallelism . . . 39
Race conditions and locking . . . 39
Deadlock in one picture . . . 40
Producer/consumer with Channels . . . 40
Background work in ASP.NET Core . . . 40
async streams: IAsyncEnumerable . . . 41
Practice . . . 41

7. Memory, GC, and Performance 42

How the garbage collector works . . . 42
Managed vs unmanaged resources . . . 42
IDisposable and `using` . . . 42
Memory leaks in a garbage-collected language . . . 43
Span and avoiding allocations . . . 44
Measure before optimizing . . . 44
The usual suspects (what actually makes apps slow) . . . 45
Caching strategies . . . 45
Big-O in plain English . . . 46

8. ASP.NET Core Essentials 47

The request pipeline and middleware . . . 47

Dependency Injection (DI) . . . 48
Controllers . . . 49
Minimal APIs . . . 49
Model binding and validation . . . 49
DTOs: never expose your entities . . . 50
HTTP status codes cheat sheet . . . 50
Problem Details and global exception handling . . . 51
Configuration and the Options pattern . . . 51
Logging (Serilog + structured logs) . . . 52
Authentication vs authorization . . . 52
CORS, rate limiting, health checks . . . 53
API design best practices . . . 53
Practice . . . 53

9. EF Core and SQL Foundations 55

Relational database basics . . . 55
SQL you must know . . . 55
Joins . . . 55
Aggregation . . . 56
Subqueries, CTEs, window functions . . . 56
Indexes: what to know . . . 57
Transactions and ACID . . . 57
Normalization in 60 seconds . . . 58
EF Core: the model . . . 58
EF Core: querying . . . 59
EF Core: writing and change tracking . . . 59
Concurrency control . . . 60
Migrations . . . 60
EF Core gotchas checklist . . . 60
The repository pattern: do you need it with EF Core? . . . 61
SQL vs NoSQL: a quick decision guide . . . 61

10. SOLID, Clean Code, and Architecture 62

S: Single Responsibility Principle . . . 62
O: Open/Closed Principle . . . 62
L: Liskov Substitution Principle . . . 63
I: Interface Segregation Principle . . . 63

D: Dependency Inversion Principle . . . 63
Other principles worth knowing . . . 64
Clean code habits . . . 64
Code smells (when to refactor) . . . 65
Value objects: fixing primitive obsession . . . 65
Layered architecture . . . 65
Clean / Onion / Hexagonal architecture . . . 66
Vertical slices (feature folders) . . . 66
CQRS in plain English . . . 66
Domain-Driven Design (DDD) vocabulary . . . 67
Monolith, modular monolith, microservices . . . 68
Practice . . . 68

11. Design Patterns in C# 69

How to use this chapter . . . 69
CREATIONAL PATTERNS . . . 69
STRUCTURAL PATTERNS . . . 71
BEHAVIORAL PATTERNS . . . 73
APPLICATION-LEVEL PATTERNS . . . 76
Pattern cheat sheet . . . 77
Patterns .NET already gives you . . . 77
Practice . . . 78

12. Testing and Quality 79

The test pyramid . . . 79
xUnit basics . . . 79
Test doubles: fakes, stubs, mocks . . . 80
Testing EF Core code . . . 81
Integration tests for APIs . . . 81
What to test (and not to) . . . 81
TDD in three steps . . . 82
Code coverage: a tool, not a goal . . . 82
Postman and API testing . . . 82
Other quality tools . . . 82
Code review checklist . . . 83
Practice . . . 83

13. System Design 84

The vocabulary . . . 84
Scaling: vertical vs horizontal . . . 84
A typical web architecture . . . 85
Load balancing . . . 85
Caching . . . 85
Databases at scale . . . 86
Messaging and queues . . . 87
Rate limiting . . . 87
API gateway, BFF, service discovery . . . 88
Observability: logs, metrics, traces . . . 88
Reliability patterns . . . 88
A repeatable system design process . . . 88
Worked example 1: URL shortener . . . 89
Worked example 2: notification service . . . 90
Worked example 3: citation / ticket management app (your own project) . . . 90
Worked example 4: course scheduling (constraint problem) . . . 91
Other classic prompts to practice . . . 91
Monolith to cloud: Azure and AWS service map . . . 91
Practice . . . 92

14. Security Fundamentals 93

Core ideas . . . 93
OWASP Top 10: the list to know . . . 93
SQL injection . . . 94
Cross-site scripting (XSS) . . . 94
CSRF (cross-site request forgery) . . . 94
Authentication in depth . . . 95
Authorization patterns . . . 96
Secrets management . . . 97
Transport and headers . . . 97
Encryption vocabulary . . . 97
Securing your CI/CD and containers . . . 98
Threat modeling in 5 minutes (STRIDE) . . . 98
Security checklist for a new API . . . 98

15. DevOps: Git, Docker, CI/CD, Cloud 99

Git essentials . . . 99

Docker fundamentals . . . 100
CI/CD concepts . . . 101
GitHub Actions: a complete .NET pipeline . . . 101
Deployment strategies . . . 102
Infrastructure as Code (IaC) . . . 102
Kubernetes in one page . . . 103
Cloud basics . . . 104
Monitoring and incident response . . . 104
Twelve-factor app (cloud-friendly design in brief) . . . 104
Practice . . . 105

16. TypeScript and React Essentials 106

JavaScript facts that surprise C# developers . . . 106
TypeScript essentials . . . 106
Modern JavaScript you will read daily . . . 107
React: the mental model . . . 108
Hooks you must know . . . 108
Rendering patterns . . . 109
Data fetching and state management . . . 110
Typing an API client against your ASP.NET Core backend . . . 110
Performance, accessibility, and testing . . . 110
Tooling quick map . . . 111
Express + PostgreSQL in 20 lines (to refresh your citation app stack) . . . 111
Practice . . . 112

17. Data Structures and Algorithms Refresher 113

Choosing the right data structure . . . 113
Complexity cheat sheet . . . 113
Pattern 1: Hash map for lookups . . . 114
Pattern 2: Two pointers . . . 114
Pattern 3: Sliding window . . . 114
Pattern 4: Binary search . . . 115
Pattern 5: Stack . . . 115
Pattern 6: BFS and DFS on graphs . . . 115
Pattern 7: Recursion and memoization (dynamic programming) . . . 116
Pattern 8: Priority queue (top-K, scheduling) . . . 117
Pattern 9: Sorting + intervals . . . 117

Pattern 10: Linked list and trees . . . 117
Pattern 11: Caching with LRU . . . 118
Problem-solving approach for interviews . . . 118
Practice . . . 119

18. Interview Q&A;, Cheat Sheets, and Study Plan 120

C# and .NET: rapid-fire answers . . . 120
Design and architecture: rapid-fire answers . . . 121
Behavioral questions: use STAR . . . 122
Questions to ask the interviewer . . . 123
One-page C# cheat sheet . . . 123
One-page SQL cheat sheet . . . 123
HTTP and REST cheat sheet . . . 124
Git and Docker cheat sheet . . . 124
A 12-week study plan . . . 124
Recommended resources . . . 125
A final word . . . 125

1. The .NET Landscape

KEY IDEA
Before any syntax, it helps to have a mental map: what C# is, what .NET is, and what happens between typing code and seeing it run. Most 'I forgot the basics' moments come from a fuzzy map, not from missing facts.

C#, .NET, the CLR: who is who?

People mix these up constantly. They are three different things that work together.

Term	What it is	Analogy
C#	The programming language you write.	English (the language)
Compiler (Roslyn)	Turns C# source into IL (Intermediate Language) stored in a .dll or .exe.	A translator to a universal shorthand
CLR	The runtime. Loads IL, JIT-compiles it to machine code, manages memory (GC), enforces type safety.	The stage crew that runs the show
BCL	Base Class Library: List<T>, Task, HttpClient, File, DateTime, etc.	The toolbox that ships with the language
.NET SDK	Command-line tools (`dotnet build`, `dotnet run`, `dotnet test`) plus compilers.	The workshop
NuGet	Package manager for .NET libraries.	An app store for libraries
ASP.NET Core	Web framework built on .NET for APIs, MVC, Razor Pages, Blazor.	A specialized kit for web apps
EF Core	Object-relational mapper (ORM) that maps classes to database tables.	A translator between objects and SQL

What happens when you run `dotnet run`

1. **Restore**: NuGet packages listed in your `.csproj` are downloaded.
2. **Compile**: Roslyn compiles your `.cs` files into IL inside `bin/Debug/net9.0/YourApp.dll`.
3. **Start**: the host process (`dotnet` or the generated executable) boots the CLR.
4. **JIT**: as methods are called for the first time, the Just-In-Time compiler converts IL to native machine code for your CPU. Hot methods can be re-optimized (tiered compilation).
5. **Run**: your `Main` (or top-level statements) executes. The garbage collector cleans up unused objects in the background.

KEY IDEA
IL is portable. The same compiled DLL runs on Windows, Linux and macOS because the JIT produces CPU-specific code at runtime. That is why your Docker/Linux deployments of a Windows-developed app just work.

Anatomy of a solution

A **solution** (`.sln`) is a container for one or more **projects** (`.csproj`). Each project builds to one assembly (a .dll or .exe). A typical layered ASP.NET Core solution looks like this:

```
MyApp.sln
 |- src/
 |   |- MyApp.Domain/          # entities, value objects, business rules (no dependencies)
 |   |- MyApp.Application/     # use cases, interfaces, DTOs (depends on Domain)
 |   |- MyApp.Infrastructure/  # EF Core, email, file storage (depends on Application)
 |   `- MyApp.Api/             # controllers / minimal APIs, DI wiring (depends on all)
 `- tests/
     |- MyApp.Domain.Tests/
     `- MyApp.Api.Tests/
```

Your Eltse Hall project follows exactly this Domain/Application/Infrastructure/API layering. The rule that matters: **dependencies point inward**. Domain knows nothing about EF Core or HTTP.

A minimal .csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>           <!-- turns on null-safety warnings -->
    <ImplicitUsings>enable</ImplicitUsings> <!-- auto-imports System, System.Linq, ... -->
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Serilog.AspNetCore" Version="8.0.0" />
    <ProjectReference Include="..\MyApp.Application\MyApp.Application.csproj" />
  </ItemGroup>
</Project>
```

The dotnet CLI cheat sheet

Command	What it does
`dotnet new webapi -n MyApi`	Create a new Web API project
`dotnet new sln` / `dotnet sln add src/X/X.csproj`	Create a solution / add a project to it
`dotnet add package Serilog`	Add a NuGet package
`dotnet add reference ../Other/Other.csproj`	Reference another project
`dotnet build` / `dotnet run` / `dotnet watch`	Compile / run / run with hot reload
`dotnet test`	Run all tests (xUnit, NUnit, MSTest)
`dotnet publish -c Release -o out`	Produce deployable output
`dotnet ef migrations add Init`	Create an EF Core migration
`dotnet ef database update`	Apply migrations to the database
`dotnet user-secrets set "Key" "value"`	Store a secret outside source control (dev only)

Top-level statements and the entry point

Modern C# lets you skip the ceremony of a `Program` class and `Main` method. These two programs are equivalent:

```
// Classic
namespace Demo;
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello");
    }
}

// Top-level statements (what templates generate today)
Console.WriteLine("Hello");
```

Namespaces and using directives

A namespace groups related types and prevents name collisions. A `using` directive lets you refer to types without the full name. File-scoped namespaces (one line ending in `;`) save an indentation level and are the modern default.

```
using System.Text;              // import types from System.Text

namespace MyApp.Domain.Orders;  // file-scoped: everything below lives here

public class Order { }
```

Assemblies and access modifiers

Access modifiers control who can see a type or member. The default if you write nothing is the *most restrictive* that makes sense (`internal` for top-level types, `private` for members).

Modifier	Visible to
`public`	Everyone, including other assemblies
`internal`	Only code in the same assembly (project)
`private`	Only inside the same class/struct
`protected`	The class and its subclasses
`protected internal`	Same assembly OR subclasses
`private protected`	Same assembly AND subclasses
`file`	Only the same source file (C# 11+)

> **TIP**
> Make everything `internal` or `private` by default and promote to `public` only when another project needs it. Small public surface areas are easier to maintain and test.

Debug vs Release, and what 'framework-dependent' means

- **Debug** builds keep extra info for the debugger and skip some optimizations. **Release** builds are optimized. Always benchmark and deploy Release.
- **Framework-dependent** deployment needs the .NET runtime installed on the target machine (small output).
- **Self-contained** deployment bundles the runtime (larger, but runs anywhere).
- In Docker you normally use the official `mcr.microsoft.com/dotnet/aspnet` runtime image for the final stage and the `sdk` image to build.

LTS vs STS

Microsoft ships a new .NET version every November. Even-numbered releases (8, 10) are **LTS** (Long Term Support, 3 years). Odd-numbered ones (9, 11) are **STS** (Standard Term Support, shorter). For production systems at companies such as insurers, LTS is the safe choice; check the current support policy on Microsoft's site because dates shift.

2. C# Language Fundamentals

KEY IDEA

This chapter is the core vocabulary of C#: types, variables, control flow, strings, collections, and null handling. If you can read every snippet here without hesitation, you have the foundation.

Variables and built-in types

```
int count = 42;              // 32-bit integer
long big = 9_000_000_000L;   // 64-bit; underscores are just readability
double price = 19.99;        // 64-bit floating point (fast, imprecise)
decimal money = 19.99m;      // 128-bit base-10 (use for currency!)
bool isActive = true;
char letter = 'A';           // single quotes, one UTF-16 code unit
string name = "Cezar";       // double quotes, immutable text
var inferred = 10;           // compiler infers int; still strongly typed
const double Pi = 3.14159;   // compile-time constant
```

TIP

Use `decimal` for money. `double` cannot represent 0.1 exactly, so `0.1 + 0.2 == 0.3` is false for doubles. `decimal` stores base-10 digits and avoids that.

Type	Size	Typical use
`byte`	1 byte (0-255)	Raw data, file bytes
`int`	4 bytes	Default whole number
`long`	8 bytes	IDs, timestamps, large counts
`float` / `double`	4 / 8 bytes	Science, graphics, GPS coordinates
`decimal`	16 bytes	Money and exact base-10 math
`bool`	1 byte	True / false
`DateTime` / `DateTimeOffset`	8 / 10 bytes	Dates; prefer `DateTimeOffset` when time zones matter
`Guid`	16 bytes	Globally unique identifiers

Value types vs reference types

This is the single most important concept in C# memory behavior.

- **Value types** (`int`, `bool`, `double`, `struct`, `enum`, `DateTime`) hold their data directly. Assigning one to another **copies the data**.
- **Reference types** (`class`, `string`, arrays, `List<T>`, delegates, `record`) hold a **pointer** to an object on the heap. Assigning copies the *pointer*, so both variables see the same object.

```
// Value type: independent copies
int a = 5;
int b = a;
```

```
b = 10;
Console.WriteLine(a);        // 5

// Reference type: shared object
var list1 = new List<int> { 1, 2 };
var list2 = list1;           // same list, two names
list2.Add(3);
Console.WriteLine(list1.Count);  // 3
```

KEY IDEA

'Value types live on the stack, reference types on the heap' is a useful simplification but not the whole truth. A value-type field inside a class lives inside that heap object. What really matters is the **copy semantics** above.

Stack vs heap, simply

- **Stack**: fast, automatic, scoped to the method call. Local value types and the *pointers* to objects live here. Freed instantly when the method returns.
- **Heap**: where objects are allocated with `new`. Freed later by the garbage collector when nothing references them.

Boxing and unboxing

Boxing wraps a value type in a heap object (so it can be treated as `object`). It costs an allocation and is a hidden performance trap, especially in loops.

```
int x = 5;
object boxed = x;       // boxing: allocates on the heap
int y = (int)boxed;     // unboxing: copies it back out

// Generics avoid boxing: List<int> stores ints directly,
// the old ArrayList boxed every element.
```

Strings

Strings are **immutable**: every 'modification' creates a new string. They are reference types, but `==` compares their *contents*.

```
string first = "Cezar", last = "Pedroso";

// Interpolation (preferred)
string full = $"{first} {last}";
string padded = $"{count,5} | {price:C2} | {DateTime.Now:yyyy-MM-dd}";

// Verbatim and raw strings
string path = @"C:\temp\file.txt";
string json = """
    { "name": "Cezar", "age": 25 }
    """;                        // raw string literal, no escaping needed

// Common methods
"  hi  ".Trim();                      // "hi"
"a,b,c".Split(',');                   // ["a","b","c"]
"Hello".Contains("ell");              // true
"Hello".StartsWith("He");             // true
"Hello".Substring(1, 3);              // "ell"
"Hello".Replace("l", "L");            // "HeLLo"
```

```
string.Join(", ", new[] {"a", "b"});   // "a, b"
string.IsNullOrWhiteSpace("   ");      // true

// Many appends in a loop? Use StringBuilder
var sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.Append(i).Append(',');
string result = sb.ToString();
```

WATCH OUT

String comparison is culture-sensitive by default in some APIs. For identifiers, keys, and file extensions use `StringComparison.OrdinalIgnoreCase`, for example `a.Equals(b, StringComparison.OrdinalIgnoreCase)`.

Control flow

```
// if / else
if (score >= 90) grade = "A";
else if (score >= 80) grade = "B";
else grade = "C";

// ternary
string label = isActive ? "On" : "Off";

// switch statement
switch (day)
{
    case DayOfWeek.Saturday:
    case DayOfWeek.Sunday:
        Console.WriteLine("Weekend");
        break;
    default:
        Console.WriteLine("Weekday");
        break;
}

// loops
for (int i = 0; i < 5; i++) { }
foreach (var item in items) { }
while (condition) { }
do { } while (condition);

// control keywords
break;      // exit the loop
continue;   // skip to next iteration
return;     // exit the method
```

Methods

```
// Basic method
static int Add(int a, int b) => a + b;     // expression-bodied

// Optional and named parameters
static void Greet(string name, string greeting = "Hello")
    => Console.WriteLine($"{greeting}, {name}");

Greet("Ana");                          // Hello, Ana
Greet(greeting: "Hi", name: "Ana");   // named args, any order

// out parameter: return extra values
if (int.TryParse("123", out int number))
```

```
    Console.WriteLine(number * 2);

// ref: pass a variable itself (changes are visible to the caller)
static void Increment(ref int n) => n++;

// params: variable number of arguments
static int Sum(params int[] nums) => nums.Sum();
Sum(1, 2, 3, 4);

// Tuples: return several values without a class
static (int Min, int Max) Range(int[] xs) => (xs.Min(), xs.Max());
var (lo, hi) = Range(new[] { 3, 9, 1 });
```

Arrays and the main collections

```
int[] arr = new int[3];              // fixed size, zero-initialized
int[] arr2 = { 1, 2, 3 };
int[,] grid = new int[3, 3];         // 2D array

var list = new List<string> { "a", "b" };   // dynamic array
list.Add("c"); list.Remove("a"); list.Insert(0, "z");
list.Contains("b"); list.IndexOf("b"); list.Sort();

var dict = new Dictionary<string, int>();   // hash map
dict["apples"] = 3;
dict.TryAdd("pears", 5);
if (dict.TryGetValue("apples", out var n)) Console.WriteLine(n);
foreach (var (key, value) in dict) Console.WriteLine($"{key}={value}");

var set = new HashSet<int> { 1, 2, 2, 3 };  // unique values, O(1) lookup
var queue = new Queue<int>();               // FIFO: Enqueue / Dequeue
var stack = new Stack<int>();               // LIFO: Push / Pop
var linked = new LinkedList<int>();         // rarely needed
var sorted = new SortedDictionary<string,int>(); // ordered by key
```

Collection	Lookup	Add	Use when
`T[]`	O(1) by index	fixed size	Size known up front
`List<T>`	O(1) by index, O(n) search	O(1) amortized at end	Default ordered list
`Dictionary<K,V>`	O(1) average	O(1) average	Fast lookup by key
`HashSet<T>`	O(1) average	O(1) average	Uniqueness, membership checks
`Queue<T>` / `Stack<T>`	n/a	O(1)	FIFO / LIFO processing
`SortedDictionary<K, V>`	O(log n)	O(log n)	Need keys in order
`ImmutableList<T>`	O(log n)	O(log n)	Thread-safe snapshots

TIP

Rule of thumb: if you search a `List<T>` with `Contains` inside a loop, you probably wanted a `HashSet<T>` or `Dictionary`. That change often turns an O(n squared) algorithm into O(n).

Interfaces for collections

Accept the **most general** type in parameters and return the most useful one. This keeps your methods flexible.

```
IEnumerable<T>       // can be looped over (read-only, forward only)
ICollection<T>       // + Count, Add, Remove
IList<T>             // + index access
IReadOnlyList<T>     // index access, no mutation
IReadOnlyDictionary<K,V>

// Good signatures
public decimal Total(IEnumerable<OrderLine> lines) => lines.Sum(l => l.Price * l.Qty);
public IReadOnlyList<Order> GetOrders() => _orders;
```

Null and nullable reference types

Tony Hoare called null his 'billion-dollar mistake'. C# 8+ lets the compiler help: with `<Nullable>enable</Nullable>`, `string` means *never null* and `string?` means *may be null*. You get warnings when you break that contract.

```
string name = null;            // warning: assigning null to non-nullable
string? maybe = null;          // OK, explicitly nullable

// Null-handling operators
int len = maybe?.Length ?? 0;      // ?. short-circuits, ?? supplies a default
maybe ??= "default";               // assign only if null
string safe = maybe!;              // ! = "trust me, not null" (use sparingly)

// Nullable value types
int? maybeInt = null;
if (maybeInt.HasValue) Console.WriteLine(maybeInt.Value);
int actual = maybeInt.GetValueOrDefault(-1);

// Guard clauses
ArgumentNullException.ThrowIfNull(order);
ArgumentException.ThrowIfNullOrWhiteSpace(name);
```

Enums

```
public enum OrderStatus { Pending = 1, Paid, Shipped, Cancelled }

var s = OrderStatus.Paid;
Console.WriteLine((int)s);                      // 2
Enum.TryParse<OrderStatus>("Shipped", out var parsed);

// Flags enums combine with bitwise OR
[Flags]
public enum Permissions { None = 0, Read = 1, Write = 2, Delete = 4 }
var p = Permissions.Read | Permissions.Write;
bool canWrite = p.HasFlag(Permissions.Write);   // true
```

Equality: == vs Equals vs ReferenceEquals

Type	== compares	Why
`int`, `struct`	Values	Value types compare their data
`string`	Contents	`string` overloads `==`
Plain `class`	Reference (same object?)	Default behavior unless overridden
`record`	All property values	Compiler generates value equality

Operators worth remembering

```
x++ ; ++x ;  x += 5;              // increment, compound assignment
a && b ; a || b ; !a              // logical (short-circuit)
a & b ; a | b ; a ^ b ; ~a        // bitwise
a << 2 ; a >> 1                   // bit shifts
obj is string s                   // type test + cast
obj as Customer                   // cast or null
(int)someDouble                   // explicit cast (may truncate)
typeof(string) ; nameof(Order)    // type object ; string name at compile time
default(int)                      // 0
x switch { 1 => "one", _ => "many" }   // switch expression
```

Practice questions

1. What is the output of: `string a = "hi"; string b = a; b += "!"; Console.WriteLine(a);`? (Answer: `hi`. Strings are immutable, so `b` points to a new string.)
2. Why use `decimal` instead of `double` for prices?
3. What is the difference between `int?` and `string?`?
4. When would you pick `HashSet<T>` over `List<T>`?
5. What does `?.` do and how does it differ from `??`?

3. Object-Oriented Programming in C#

KEY IDEA

OOP is a way to organize code around objects that bundle data with behavior. The four pillars (encapsulation, abstraction, inheritance, polymorphism) are interview staples, but more importantly they are how you keep large codebases understandable.

Classes and objects

A **class** is a blueprint. An **object** (instance) is a thing built from it. Classes have **fields** (data), **properties** (controlled access to data), **methods** (behavior), and **constructors** (how to create one).

```
public class BankAccount
{
    private decimal _balance;                // field: private state

    public string Owner { get; }             // read-only property
    public decimal Balance => _balance;      // computed, read-only
    public DateTime OpenedAt { get; init; }  // settable only during construction

    public BankAccount(string owner, decimal initial = 0)
    {
        Owner = owner;
        _balance = initial;
        OpenedAt = DateTime.UtcNow;
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        _balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (amount > _balance) throw new InvalidOperationException("Insufficient funds");
        _balance -= amount;
    }
}

var acct = new BankAccount("Ana", 100m);
acct.Deposit(50m);
Console.WriteLine(acct.Balance);   // 150
// acct.Balance = 1_000_000;       // compile error: no setter. Encapsulation!
```

Pillar 1: Encapsulation

Hide internal state and expose only safe operations. In the example above nobody can set `Balance` to nonsense. They must go through `Deposit` and `Withdraw`, which enforce rules. **If a rule must always hold, put the rule next to the data it protects.**

KEY IDEA

Anemic models (classes with only public getters/setters and no behavior) push business rules into services, where they get duplicated and forgotten. Prefer 'rich' domain objects when the rules matter.

Property flavors

```
public string A { get; set; }              // read + write
public string B { get; private set; }      // public read, private write
public string C { get; init; }             // set only in constructor/object initializer
public required string D { get; init; }    // C# 11: caller MUST provide it
public string E => $"{A}-{B}";             // expression-bodied computed property

// Full property with logic
private string _email = "";
public string Email
{
    get => _email;
    set => _email = value?.Trim().ToLowerInvariant() ?? throw new ArgumentNullException();
}
```

Constructors, initializers, primary constructors

```
// Object initializer
var p = new Person { Name = "Ana", Age = 30 };

// Primary constructor (C# 12): parameters usable across the class
public class Greeter(string greeting)
{
    public string Greet(string name) => $"{greeting}, {name}!";
}

// Constructor chaining
public class Rect
{
    public int W { get; }
    public int H { get; }
    public Rect(int w, int h) { W = w; H = h; }
    public Rect(int size) : this(size, size) { }   // calls the other constructor
}

// Static constructor runs once per type; static members belong to the type
public class Config
{
    public static readonly string Env;
    static Config() { Env = Environment.GetEnvironmentVariable("ENV") ?? "dev"; }
}
```

Pillar 2: Inheritance

A subclass reuses and extends a base class. Use it for genuine 'is-a' relationships, and keep hierarchies shallow.

```
public abstract class Shape
{
    public string Name { get; }
    protected Shape(string name) => Name = name;

    public abstract double Area();                          // subclasses MUST implement
```

```
    public virtual string Describe() => $"{Name} with area {Area():F2}"; // may override
}

public class Circle(double r) : Shape("Circle")
{
    public override double Area() => Math.PI * r * r;
}

public class Square(double side) : Shape("Square")
{
    public override double Area() => side * side;
    public override string Describe() => base.Describe() + " (all sides equal)";
}

public sealed class Final : Shape   // sealed: nobody can inherit from this
{
    public Final() : base("Final") { }
    public override double Area() => 0;
}
```

Pillar 3: Polymorphism

'Many forms': code written against a base type works with any subtype, and the *correct override runs at runtime* (dynamic dispatch).

```
List<Shape> shapes = [new Circle(2), new Square(3)];
foreach (Shape s in shapes)
    Console.WriteLine(s.Describe());   // calls Circle's or Square's version

// `virtual` + `override` = runtime polymorphism
// `new` keyword on a member HIDES instead of overriding (almost always a bug)
```

Pillar 4: Abstraction and interfaces

An **interface** is a contract: 'whoever implements me promises to provide these members.' It says *what*, not *how*. Interfaces are the backbone of testable, swappable code and dependency injection.

```
public interface IEmailSender
{
    Task SendAsync(string to, string subject, string body);
}

public class SmtpEmailSender : IEmailSender
{
    public Task SendAsync(string to, string subject, string body) { /* real SMTP */ return
        Task.CompletedTask; }
}

public class FakeEmailSender : IEmailSender   // for tests
{
    public List<string> Sent = new();
    public Task SendAsync(string to, string subject, string body)
    { Sent.Add(to); return Task.CompletedTask; }
}

public class WelcomeService(IEmailSender email)   // depends on the contract only
{
    public Task WelcomeAsync(string address) =>
        email.SendAsync(address, "Welcome!", "Glad you are here.");
}
```

Abstract class vs interface

	Abstract class	Interface
Multiple inheritance	No (one base class)	Yes (implement many)
State (fields)	Yes	No instance fields
Default behavior	Yes (virtual methods)	Yes, via default interface methods (rare)
Constructor	Yes	No
Use when	Sharing code among closely related types	Defining a capability or contract

TIP
Default to interfaces for dependencies and abstract classes only when subclasses really share implementation. If you are unsure, start with an interface.

Composition over inheritance

Inheritance couples a child to its parent forever. Composition ('has-a') assembles behavior from small pieces and is easier to change.

```
// Inheritance version: brittle
class Duck : Bird { }              // what about a duck that cannot fly?

// Composition version: flexible
class Duck(IFlyBehavior fly, ISoundBehavior sound)
{
    public void Fly() => fly.Fly();
    public void Quack() => sound.Make();
}
```

Structs

A `struct` is a value type. It is copied on assignment and is best for small, immutable, data-like things (a point, a money amount, a coordinate).

```
public readonly struct Point(int x, int y)
{
    public int X { get; } = x;
    public int Y { get; } = y;
    public double DistanceTo(Point o) => Math.Sqrt(Math.Pow(X - o.X, 2) + Math.Pow(Y - o.Y, 2));
}
```

	class	struct
Kind	Reference type	Value type
Assignment	Copies the reference	Copies the data
Default value	null	all fields zeroed
Inheritance	Yes	No (interfaces only)
Choose when	Identity matters, larger objects, shared mutation	Small (<16 bytes), immutable, short-lived

Records

A `record` is a class (or struct) designed for **data with value-based equality**. The compiler writes `Equals`, `GetHashCode`, `ToString`, and non-destructive copying (`with`) for you. Perfect for DTOs, commands, and value objects.

```
public record Person(string Name, int Age);

var a = new Person("Ana", 30);
var b = new Person("Ana", 30);
Console.WriteLine(a == b);              // True (compares values)
var older = a with { Age = 31 };        // copy with one change; a is untouched
Console.WriteLine(a);                   // Person { Name = Ana, Age = 30 }

var (name, age) = a;                    // deconstruction for free

public record struct Money(decimal Amount, string Currency);   // value-type record
```

Static, sealed, partial, readonly

- `static` members belong to the type, not an instance. Static classes cannot be instantiated (good for pure helper functions).
- `sealed` prevents inheritance. Many frameworks recommend sealing classes you did not design for inheritance (also slightly faster).
- `partial` splits one type across files (code generators use this).
- `readonly` fields can only be assigned in the constructor or initializer.

Generics

Generics let you write code once for many types while keeping compile-time type safety. `List<int>` and `List<string>` are the same code with different type arguments.

```
public class Result<T>
{
    public bool Success { get; }
    public T? Value { get; }
    public string? Error { get; }
    private Result(bool ok, T? v, string? e) { Success = ok; Value = v; Error = e; }

    public static Result<T> Ok(T value) => new(true, value, null);
    public static Result<T> Fail(string error) => new(false, default, error);
}

var r = Result<int>.Ok(5);

// Constraints restrict what T can be
public T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) >= 0 ? a : b;

public class Repository<T> where T : class, IEntity, new() { }
// where T : class | struct | new() | BaseClass | IInterface | notnull
```

Covariance and contravariance (in one paragraph)

`IEnumerable<out T>` is **covariant**: an `IEnumerable<Dog>` can be used where `IEnumerable<Animal>` is expected, because you only *read* items out. `Action<in T>` is **contravariant**: an `Action<Animal>` can be used where an `Action<Dog>` is expected, because it only *consumes* items. `List<T>` is invariant because it both reads and writes.

Delegates, events, and lambdas

A **delegate** is a type-safe function pointer. `Action` (no return), `Func<..., TResult>` (returns something), and `Predicate<T>` (returns bool) cover 99% of needs. A **lambda** is an inline anonymous function. An **event** is a delegate with restricted access: outsiders can subscribe, but only the owner can raise it.

```
Func<int, int, int> add = (a, b) => a + b;
Action<string> log = msg => Console.WriteLine(msg);
Predicate<int> isEven = n => n % 2 == 0;

Console.WriteLine(add(2, 3));       // 5

// Events (the observer pattern built into the language)
public class Button
{
    public event EventHandler<string>? Clicked;
    public void Click() => Clicked?.Invoke(this, "clicked!");
}

var btn = new Button();
btn.Clicked += (sender, msg) => Console.WriteLine(msg);
btn.Click();
```

Extension methods

Add methods to existing types without modifying them. LINQ is built entirely from these.

```
public static class StringExtensions
{
    public static bool IsEmail(this string s) => s.Contains('@') && s.Contains('.');
    public static string Truncate(this string s, int max) =>
        s.Length <= max ? s : s[..max] + "...";
}

"hi@x.com".IsEmail();        // looks like an instance method
"A long sentence".Truncate(5);
```

Operator overloading and indexers (occasionally useful)

```
public readonly record struct Money(decimal Amount, string Currency)
{
    public static Money operator +(Money a, Money b) =>
        a.Currency == b.Currency ? new(a.Amount + b.Amount, a.Currency)
                                 : throw new InvalidOperationException("Currency mismatch");
}

public class Matrix
{
    private readonly double[,] _d = new double[3, 3];
    public double this[int r, int c] { get => _d[r, c]; set => _d[r, c] = value; }
}
```

Practice questions

1. Explain the four pillars of OOP in one sentence each, with a C# example.
2. What is the difference between `abstract` and `virtual`? What does `override` do?
3. Why might you choose a `record` over a `class`?
4. A method takes `IEnumerable<Animal>`. Can you pass a `List<Dog>`? Why?
5. When is a `struct` a better choice than a `class`? When is it worse?
6. What is the difference between composition and inheritance? Which is usually preferred?

4. Modern C# Features You Should Know

KEY IDEA
C# evolves every year. These features make code shorter and safer, and interviewers and code reviewers expect you to recognize them. Everything here works in current .NET versions.

Pattern matching

Pattern matching lets you test the shape of data and extract pieces in one step. It replaces long chains of `if` and casts.

```
object o = 42;

// Type pattern
if (o is int n) Console.WriteLine(n + 1);

// Property pattern
if (order is { Status: OrderStatus.Paid, Total: > 100 }) ApplyFreeShipping();

// Relational and logical patterns
string size = temp switch
{
    < 0           => "freezing",
    >= 0 and < 15 => "cold",
    >= 15 and < 25 => "mild",
    _             => "hot"
};

// Switch expression over types
decimal Area(Shape s) => s switch
{
    Circle c    => (decimal)(Math.PI * c.R * c.R),
    Rectangle r => r.W * r.H,
    null        => throw new ArgumentNullException(nameof(s)),
    _           => throw new NotSupportedException()
};

// List patterns (C# 11)
int[] xs = [1, 2, 3];
if (xs is [1, .., 3]) Console.WriteLine("starts with 1, ends with 3");

// Not / null patterns
if (name is not null) { }
if (obj is not string) { }
```

Switch expressions with tuples

```
string Decide(bool isMember, bool isHoliday) => (isMember, isHoliday) switch
{
    (true, true)   => "50% off",
    (true, false)  => "20% off",
    (false, true)  => "10% off",
    (false, false) => "full price"
};
```

Collection expressions and spread (C# 12)

```
int[] a = [1, 2, 3];
List<int> b = [4, 5];
int[] all = [..a, ..b, 6];        // spread: [1,2,3,4,5,6]
List<string> empty = [];
```

Tuples and deconstruction

```
(string Name, int Age) person = ("Ana", 30);
Console.WriteLine(person.Name);

var (n, a) = person;                      // deconstruct
(n, a) = (a.ToString(), 31);              // swap or reassign in one line

foreach (var (key, value) in dictionary)  // deconstruct while looping
    Console.WriteLine($"{key}: {value}");
```

Local functions and static lambdas

```
int Fib(int n)
{
    return Helper(n);

    int Helper(int k) => k < 2 ? k : Helper(k - 1) + Helper(k - 2);  // local function
}

Func<int, int> twice = static x => x * 2;   // 'static' = cannot capture outer variables
```

Target-typed new, init-only, required members

```
Person p = new("Ana", 30);              // type inferred from the left side
List<Order> orders = new();

public class Settings
{
    public required string ConnectionString { get; init; }   // must be set
    public int Timeout { get; init; } = 30;
}
var s = new Settings { ConnectionString = "..." };
// s.Timeout = 5;   // error: init-only after construction
```

Nullable flow analysis helpers

```
public bool TryGetUser(int id, [NotNullWhen(true)] out User? user) { ... }

if (TryGetUser(5, out var u))
    Console.WriteLine(u.Name);   // compiler knows u is not null here
```

Ranges and indices

```
var arr = new[] { 10, 20, 30, 40, 50 };
arr[^1];        // 50   (^1 = last)
arr[1..3];      // [20, 30]  (start inclusive, end exclusive)
arr[..2];       // [10, 20]
arr[3..];       // [40, 50]
"hello"[1..4];  // "ell"
```

Using declarations and `using` statements

```
// Disposed at the end of the enclosing scope. No extra braces needed.
using var conn = new SqlConnection(cs);
using var reader = new StreamReader("file.txt");
string text = await reader.ReadToEndAsync();
```

Iterators: `yield return`

`yield return` builds a lazy sequence. Nothing runs until someone enumerates it, and it produces one item at a time without building a whole list in memory.

```
IEnumerable<int> Evens(int max)
{
    for (int i = 0; i <= max; i += 2)
        yield return i;        // execution pauses here until the next item is requested
}

foreach (var e in Evens(10)) Console.Write(e + " ");   // 0 2 4 6 8 10

// Great for streaming large data
IEnumerable<string> ReadLines(string path)
{
    using var r = new StreamReader(path);
    while (r.ReadLine() is { } line) yield return line;
}
```

Attributes and reflection basics

Attributes attach metadata to code. Frameworks read them via **reflection** (inspecting types at runtime). You see them everywhere in ASP.NET Core.

```
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    [HttpGet("{id:int}")]
    [ProducesResponseType(typeof(OrderDto), 200)]
    [Authorize(Roles = "Admin")]
    public IActionResult Get(int id) => Ok();
}

// Custom attribute
[AttributeUsage(AttributeTargets.Property)]
public class MaxWordsAttribute(int max) : Attribute { public int Max => max; }

// Reading it
var props = typeof(Post).GetProperties()
    .Where(p => p.GetCustomAttribute<MaxWordsAttribute>() != null);
```

Generic math and `INumber<T>` (just so you recognize it)

```
static T Sum<T>(IEnumerable<T> xs) where T : INumber<T>
{
    T total = T.Zero;
    foreach (var x in xs) total += x;
    return total;
}
```

Exceptions: the essentials

```
try
{
    var text = File.ReadAllText(path);
}
catch (FileNotFoundException ex)             // most specific first
{
    logger.LogWarning(ex, "Missing {Path}", path);
}
catch (Exception ex) when (ex is IOException or UnauthorizedAccessException)  // filter
{
    logger.LogError(ex, "Cannot read file");
    throw;                                   // rethrow preserving the stack trace
}
finally
{
    // always runs: cleanup
}

// Custom exception
public class InsufficientFundsException(decimal needed)
    : Exception($"Need {needed:C} more");

// 'throw;' keeps the stack trace. 'throw ex;' RESETS it. Never do the second.
```

Do	Don't
Throw exceptions for *exceptional* situations	Use exceptions for normal control flow (parsing user input: use `TryParse`)
Catch only what you can handle	Swallow with empty `catch { }`
Use `throw;` to rethrow	Use `throw ex;`
Include context in messages	Throw bare `Exception("error")`
Validate arguments with guard clauses	Let nulls travel deep into the code

The Result pattern: errors as values

For *expected* failures (validation, not found), returning a result object is often clearer and cheaper than throwing.

```
public record Error(string Code, string Message);

public readonly struct Result<T>
{
    public T? Value { get; }
    public Error? Error { get; }
    public bool IsSuccess => Error is null;
    private Result(T? v, Error? e) { Value = v; Error = e; }
```

```
    public static Result<T> Success(T v) => new(v, null);
    public static Result<T> Failure(Error e) => new(default, e);
}

Result<Order> Place(Cart cart) =>
    cart.Items.Count == 0
        ? Result<Order>.Failure(new("cart.empty", "Cart is empty"))
        : Result<Order>.Success(new Order(cart));

// In an endpoint
var result = Place(cart);
return result.IsSuccess ? Results.Ok(result.Value) : Results.BadRequest(result.Error);
```

Dates and times: the traps

- Store and compare timestamps in **UTC** (`DateTime.UtcNow` or `DateTimeOffset.UtcNow`). Convert to local time only for display.
- `DateTimeOffset` carries the UTC offset and avoids most 'which time zone?' bugs. `DateOnly` and `TimeOnly` exist for pure dates/times.
- Inject a `TimeProvider` (built into .NET 8+) instead of calling `DateTime.UtcNow` directly, so tests can control the clock.

```
public class TokenService(TimeProvider clock)
{
    public bool IsExpired(DateTimeOffset expiresAt) => clock.GetUtcNow() >= expiresAt;
}
// In production: services.AddSingleton(TimeProvider.System);
// In tests: new FakeTimeProvider(startTime) from Microsoft.Extensions.TimeProvider.Testing
```

File I/O and JSON

```
await File.WriteAllTextAsync("a.txt", "hello");
string text = await File.ReadAllTextAsync("a.txt");
foreach (var line in File.ReadLines("big.txt")) { }   // lazy, memory friendly
Path.Combine("folder", "file.txt");                   // never concatenate paths by hand

using System.Text.Json;
var json = JsonSerializer.Serialize(new Person("Ana", 30));
var back = JsonSerializer.Deserialize<Person>(json);

var options = new JsonSerializerOptions
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    WriteIndented = true
};
```

Regular expressions (the 30-second version)

```
using System.Text.RegularExpressions;

bool ok = Regex.IsMatch("ana@x.com", @"^[^@\s]+@[^@\s]+\.[^@\s]+$");
var m = Regex.Match("Order #1234", @"#(\d+)");
if (m.Success) Console.WriteLine(m.Groups[1].Value);   // 1234

// Source-generated regex: compiled at build time, fast
[GeneratedRegex(@"^\d{3}-\d{4}$")]
private static partial Regex PhoneRegex();
```

5. LINQ and Working with Data

> **KEY IDEA**
> LINQ (Language Integrated Query) lets you filter, transform, group, and aggregate any collection with a consistent set of methods. It is used constantly in .NET, including inside EF Core queries, so it is worth knowing cold.

The mental model

Think of LINQ as an assembly line: data flows in, passes through stations (filter, sort, project), and a final station collects the output. Most stations are **lazy**: nothing happens until you enumerate the result.

```
var people = new List<Person>
{
    new("Ana", 30, "Iowa"), new("Bruno", 25, "Texas"),
    new("Carla", 35, "Iowa"), new("Diego", 28, "Texas"),
};

var result = people
    .Where(p => p.Age >= 28)           // filter
    .OrderBy(p => p.Name)              // sort
    .Select(p => new { p.Name, p.State })   // project (shape the output)
    .ToList();                         // execute now and materialize
```

Core operators

Operator	Purpose	Example
`Where`	Filter	`xs.Where(x => x > 3)`
`Select`	Transform each item	`xs.Select(x => x * 2)`
`SelectMany`	Flatten nested collections	`orders.SelectMany(o => o.Lines)`
`OrderBy` / `ThenBy`	Sort ascending (+ tie-breaker)	`.OrderBy(p => p.State).ThenBy(p => p.Name)`
`OrderByDescending`	Sort descending	
`GroupBy`	Bucket by a key	`.GroupBy(p => p.State)`
`Join`	Inner join two sequences	`a.Join(b, x => x.Id, y => y.AId, ...)`
`Distinct`	Remove duplicates	`xs.Distinct()`
`Skip` / `Take`	Paging	`.Skip(20).Take(10)`
`First` / `FirstOrDefault`	First item (throws / null if none)	`xs.FirstOrDefault(x => x.Id == 5)`
`Single` / `SingleOrDefault`	Exactly one item (throws if 0 or 2+)	
`Any` / `All`	Existence checks	`xs.Any(x => x.IsActive)`
`Count` / `Sum` / `Min` / `Max` / `Average`	Aggregates	`orders.Sum(o => o.Total)`

Operator	Purpose	Example
`Aggregate`	Custom fold	`xs.Aggregate((a, b) => a * b)`
`Zip`	Pair two sequences	`names.Zip(ages)`
`Chunk`	Split into batches	`xs.Chunk(100)`
`ToList` / `ToArray` / `ToDictionary` / `ToHashSet`	Materialize	
`Union` / `Intersect` / `Except`	Set operations	
`MinBy` / `MaxBy`	Item with min/max key	`people.MaxBy(p => p.Age)`

GroupBy in practice

```
var byState = people.GroupBy(p => p.State);

foreach (var g in byState)
{
    Console.WriteLine($"{g.Key}: {g.Count()} people, avg age {g.Average(p => p.Age)}");
}

// Project the groups into a report
var report = people
    .GroupBy(p => p.State)
    .Select(g => new
    {
        State = g.Key,
        Count = g.Count(),
        Oldest = g.MaxBy(p => p.Age)!.Name,
    })
    .OrderByDescending(r => r.Count);

// Dictionary from a list (keys must be unique!)
var byName = people.ToDictionary(p => p.Name);
// Lookup allows duplicate keys
var lookup = people.ToLookup(p => p.State);
var iowans = lookup["Iowa"];
```

Joins

```
var orders = new[] { new Order(1, 10), new Order(2, 20), new Order(3, 10) };
var customers = new[] { new Customer(10, "Ana"), new Customer(20, "Bruno") };

var joined = orders.Join(customers,
    o => o.CustomerId,
    c => c.Id,
    (o, c) => new { o.Id, Customer = c.Name });

// Left join: GroupJoin + DefaultIfEmpty (or LeftJoin in .NET 10)
var left = customers
    .GroupJoin(orders, c => c.Id, o => o.CustomerId, (c, os) => new { c, os })
    .SelectMany(x => x.os.DefaultIfEmpty(), (x, o) => new { x.c.Name, OrderId = o?.Id });
```

Query syntax vs method syntax

```
// Query syntax (SQL-like)
var q = from p in people
        where p.Age > 26
        orderby p.Name
        select p.Name;

// Same thing in method syntax (what most teams write)
var m = people.Where(p => p.Age > 26).OrderBy(p => p.Name).Select(p => p.Name);
```

Deferred execution: the classic gotcha

```
var numbers = new List<int> { 1, 2, 3 };
var query = numbers.Where(n => n > 1);    // NOT executed yet

numbers.Add(4);                           // changes the source

foreach (var n in query) Console.Write(n + " ");   // 2 3 4  (runs NOW, sees the 4)

// Enumerating twice runs the whole pipeline twice
var expensive = people.Select(p => ExpensiveCall(p));
var a = expensive.Count();    // runs ExpensiveCall for every person
var b = expensive.ToList();   // runs it AGAIN
// Fix: call .ToList() once and reuse the list.
```

WATCH OUT

Multiple enumeration of an `IEnumerable<T>` is a frequent bug and a common code-review comment. If you need the data twice, materialize it once with `ToList()`.

IEnumerable vs IQueryable

`IEnumerable<T>` runs in memory using delegates. `IQueryable<T>` holds an **expression tree** that a provider (like EF Core) translates to SQL. The difference decides *where* your filter runs.

```
// Good: Where is translated to SQL, only matching rows come back
var active = await db.Users.Where(u => u.IsActive).ToListAsync();

// Bad: AsEnumerable() / ToList() first pulls EVERY row, then filters in memory
var slow = db.Users.ToList().Where(u => u.IsActive);

// Also bad: calling your own C# method inside the query can't be translated
db.Users.Where(u => MyHelper(u.Name));    // runtime error or client-side evaluation
```

Performance habits

- `Count() > 0` loops through everything on some sequences. Use `Any()`.
- For `List<T>` and arrays, use `.Count`/`.Length` properties rather than `Count()`.
- Avoid LINQ in extremely hot loops; a plain `for` loop allocates nothing.
- Order matters: `Where` before `Select`/`OrderBy` shrinks the data early.
- `First()` stops at the first match, but `OrderBy(...).First()` sorts everything. Use `MinBy`/`MaxBy`.
- Prefer `ToDictionary`/`ToHashSet` for repeated lookups instead of `Where` or `FirstOrDefault` inside a loop (the N+1 trap in memory).

Immutability and `IReadOnly*`

```
public class Cart
{
    private readonly List<Item> _items = new();
    public IReadOnlyList<Item> Items => _items;      // callers can't Add/Remove

    public void Add(Item item) { /* validate */ _items.Add(item); }
}

using System.Collections.Immutable;
var im = ImmutableList.Create(1, 2, 3);
var im2 = im.Add(4);        // im unchanged, im2 is a new list
```

Sorting and comparing

```
people.Sort((a, b) => a.Age.CompareTo(b.Age));      // in place
var sorted = people.OrderBy(p => p.Age).ToList();   // new list

public class Person : IComparable<Person>
{
    public int CompareTo(Person? other) => Age.CompareTo(other!.Age);
}

// Custom equality for sets and dictionaries
public class CaseInsensitive : IEqualityComparer<string>
{
    public bool Equals(string? a, string? b) => string.Equals(a, b,
        StringComparison.OrdinalIgnoreCase);
    public int GetHashCode(string s) => s.ToUpperInvariant().GetHashCode();
}
var set = new HashSet<string>(new CaseInsensitive());
// Or simply: new HashSet<string>(StringComparer.OrdinalIgnoreCase)
```

Hashing and why GetHashCode matters

`Dictionary` and `HashSet` first compute `GetHashCode()` to pick a bucket, then use `Equals` to confirm. The contract: **if two objects are equal, they must return the same hash code**. Override both together (records do this for you) and never use mutable fields in the hash of an object that lives inside a set.

```
public class Sku(string code)
{
    public string Code { get; } = code;
    public override bool Equals(object? o) => o is Sku s && Code == s.Code;
    public override int GetHashCode() => Code.GetHashCode();
}
// or HashCode.Combine(Field1, Field2) for multiple fields
```

Practice

1. Given a list of orders, return the top 3 customers by total spend. (GroupBy, Sum, OrderByDescending, Take)
2. Find duplicate emails in a list of users. (GroupBy + Where(g => g.Count() > 1))
3. Explain deferred execution and show a bug it can cause.

4. Why is `db.Users.ToList().Where(...)` bad? What should it be?
5. Turn `List<Order>` into `Dictionary<int, Order>` and explain what happens with duplicate keys.

6. Async, Threads, and Concurrency

> **KEY IDEA**
> async/await is where many developers say 'I use it but I could not explain it.' This chapter fixes that. The goal: understand what a Task really is, why async helps web servers, and how to avoid the classic deadlocks and race conditions.

Why async exists

Imagine a restaurant with 10 waiters. A **synchronous** waiter takes an order, walks to the kitchen, and *stands there* until the food is ready. 10 customers and you are out of waiters. An **asynchronous** waiter hands the order to the kitchen, goes to serve other tables, and returns when the food is ready.

In a web API the 'waiters' are **thread pool threads**. Database calls, HTTP calls, and file reads are *I/O-bound*: the CPU is idle while waiting for the network or disk. Async frees the thread during that wait so it can serve other requests. That is how a small server handles thousands of concurrent requests.

> **KEY IDEA**
> Async is about **scalability** (using fewer threads for the same load), not about making a single request faster. A single async call takes the same wall-clock time as the sync version.

Task and Task<T>

A `Task` is a **promise of a future result**: an object representing work that may not be finished. `Task<int>` will eventually produce an `int`. `await` means: 'if this isn't done yet, give the thread back and resume here when it is.'

```
public async Task<string> GetPageAsync(string url)
{
    using var http = new HttpClient();
    string html = await http.GetStringAsync(url);   // thread is released while waiting
    return html.ToUpper();                          // continues here when the response arrives
}

// Calling it
string page = await GetPageAsync("https://example.com");
```

What the compiler does

An `async` method is rewritten into a **state machine**. Each `await` is a possible pause point: the method saves its local variables, returns to the caller, and later continues from the same spot. You write sequential-looking code; the compiler generates the callbacks.

Rules of thumb

Rule	Why
Name async methods with the `Async` suffix	Convention; signals the caller should `await`
Return `Task` / `Task<T>`, not `void`	`async void` exceptions crash the process and cannot be awaited. Only for event handlers
Async all the way up	Mixing sync and async leads to deadlocks and thread starvation
Never call `.Result` or `.Wait()` on a Task	Blocks a thread; can deadlock in UI/legacy contexts
Pass a `CancellationToken` through	Lets callers cancel work, e.g. when the HTTP request is aborted
Use `ValueTask<T>` only for hot paths that often complete synchronously	Avoids allocating a Task, but is trickier to use
Do not use `Task.Run` in ASP.NET Core to 'make it async'	It just moves work to another pool thread; no scalability benefit for I/O

CancellationToken

```
[HttpGet]
public async Task<IActionResult> Search(string q, CancellationToken ct)   // bound automatically
{
    var results = await _db.Products
        .Where(p => p.Name.Contains(q))
        .ToListAsync(ct);          // aborts the query if the client disconnects
    return Ok(results);
}

// Creating your own with a timeout
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5));
try { await DoWorkAsync(cts.Token); }
catch (OperationCanceledException) { /* timed out or cancelled */ }

// Inside long loops
while (!ct.IsCancellationRequested) { ... }   // or ct.ThrowIfCancellationRequested();
```

Running things concurrently

`await` one after another runs them **sequentially**. To run independent operations at the same time, start them all and then await together.

```
// Sequential: total time = A + B + C
var user = await GetUserAsync(id);
var orders = await GetOrdersAsync(id);
var prefs = await GetPrefsAsync(id);

// Concurrent: total time = max(A, B, C)
var userTask = GetUserAsync(id);
var ordersTask = GetOrdersAsync(id);
var prefsTask = GetPrefsAsync(id);
await Task.WhenAll(userTask, ordersTask, prefsTask);
var dto = new Dashboard(await userTask, await ordersTask, await prefsTask);

// First one to finish
var first = await Task.WhenAny(t1, t2);

// Limit parallelism over a big list (.NET 6+)
```

```
await Parallel.ForEachAsync(urls, new ParallelOptions { MaxDegreeOfParallelism = 8 },
    async (url, ct) => await DownloadAsync(url, ct));
```

WATCH OUT

Do not run several operations on the **same EF Core** `DbContext` concurrently. `DbContext` is not thread-safe. Create separate scopes/contexts or await sequentially.

Exceptions in async code

```
try
{
    await DoAsync();
}
catch (Exception ex)            // exceptions flow through await just like normal code
{
    ...
}

// With WhenAll, await rethrows only the FIRST exception.
// Inspect all of them via the tasks:
var all = Task.WhenAll(t1, t2);
try { await all; }
catch { foreach (var e in all.Exception!.InnerExceptions) Log(e); }
```

ConfigureAwait(false) and SynchronizationContext

In UI apps and old ASP.NET, `await` resumes on the original 'context' (the UI thread, for example). Blocking that thread with `.Result` while the continuation needs it is the classic **deadlock**. ASP.NET Core has **no** synchronization context, so deadlocks are rare there, but library authors still use `ConfigureAwait(false)` so their code works anywhere. In your own ASP.NET Core application code you generally do not need it.

Threads, thread pool, and parallelism

- A **thread** is an OS-level path of execution. Creating them is expensive, so .NET keeps a **thread pool** that reuses them.
- **Concurrency** = dealing with many things at once (async). **Parallelism** = doing many things at the same instant (multiple cores). Async is for waiting; parallelism is for CPU work.
- CPU-bound work (image processing, heavy math): use `Parallel.For`, PLINQ (`.AsParallel()`), or `Task.Run` in non-web apps.
- I/O-bound work (network, disk, DB): use async/await.

Race conditions and locking

A **race condition** happens when two threads touch shared data and the result depends on timing.

```
// BUG: counter++ is read, add, write: three steps. Two threads can overwrite each other.
int counter = 0;
Parallel.For(0, 100_000, _ => counter++);
Console.WriteLine(counter);          // less than 100000, different every run

// Fix 1: Interlocked (atomic)
```

```
Parallel.For(0, 100_000, _ => Interlocked.Increment(ref counter));

// Fix 2: lock (one thread at a time inside)
private readonly object _gate = new();
lock (_gate) { counter++; }

// Fix 3: concurrent collections
var bag = new ConcurrentDictionary<string, int>();
bag.AddOrUpdate("hits", 1, (_, old) => old + 1);
```

TIP

You cannot `await` inside a `lock`. For async-friendly mutual exclusion use `SemaphoreSlim` with `await sem.WaitAsync()` and release in a `finally` block.

```
private readonly SemaphoreSlim _sem = new(1, 1);   // 1 = mutex, N = limit concurrency to N

public async Task UpdateAsync()
{
    await _sem.WaitAsync();
    try { await DoCriticalWorkAsync(); }
    finally { _sem.Release(); }
}
```

Deadlock in one picture

```
Thread A holds Lock 1, wants Lock 2
Thread B holds Lock 2, wants Lock 1
-> both wait forever.

Prevent it: always acquire locks in the same order, hold them briefly,
or avoid shared mutable state entirely (immutability, message passing).
```

Producer/consumer with Channels

`System.Threading.Channels` is a modern, async-friendly queue. Perfect for background processing pipelines.

```
var channel = Channel.CreateBounded<Job>(capacity: 100);   // bounded = back-pressure

// Producer
await channel.Writer.WriteAsync(new Job("send-email"));
channel.Writer.Complete();

// Consumer
await foreach (var job in channel.Reader.ReadAllAsync(ct))
    await ProcessAsync(job);
```

Background work in ASP.NET Core

```
public class CleanupService(ILogger<CleanupService> log, IServiceScopeFactory scopes) :
    BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stop)
    {
        using var timer = new PeriodicTimer(TimeSpan.FromMinutes(5));
        while (await timer.WaitForNextTickAsync(stop))
        {
```

```
            using var scope = scopes.CreateScope();            // scoped services need a scope
            var db = scope.ServiceProvider.GetRequiredService<AppDb>();
            await db.Sessions.Where(s => s.Expired).ExecuteDeleteAsync(stop);
            log.LogInformation("Cleanup done");
        }
    }
}
// Program.cs: builder.Services.AddHostedService<CleanupService>();
```

async streams: IAsyncEnumerable<T>

```
public async IAsyncEnumerable<Order> StreamOrdersAsync([EnumeratorCancellation]
    CancellationToken ct = default)
{
    await foreach (var o in _db.Orders.AsAsyncEnumerable().WithCancellation(ct))
        yield return o;
}

await foreach (var o in StreamOrdersAsync()) Console.WriteLine(o.Id);
```

Practice

1. Why does async improve scalability of a web API but not the latency of a single request?
2. What is wrong with `public async void SaveAsync()`?
3. Rewrite three sequential awaits so they run concurrently.
4. Explain a race condition and give two ways to fix it.
5. Why can't you use `lock` around an `await`? What do you use instead?
6. What does `CancellationToken` do in a controller action?

7. Memory, GC, and Performance

KEY IDEA
You do not need to be a performance engineer, but you should know how .NET manages memory, why `IDisposable` exists, and which habits keep your apps fast. Most real-world slowness comes from a handful of repeat offenders, listed at the end of this chapter.

How the garbage collector works

When you write `new Order()`, memory is allocated on the **managed heap**. You never free it manually. The **garbage collector (GC)** periodically finds objects no longer reachable from any 'root' (static fields, stack variables, etc.) and reclaims them.

Generations

Most objects die young (a temporary string, a request DTO). So the GC divides the heap into **generations** and collects the young ones most often.

Generation	Contains	Collected
Gen 0	Newly allocated objects	Very frequently; fast
Gen 1	Survivors of one Gen 0 collection	Sometimes; a buffer between 0 and 2
Gen 2	Long-lived objects (caches, singletons)	Rarely; slower, can pause the app
LOH (Large Object Heap)	Objects of 85,000+ bytes (big arrays/strings)	Only with Gen 2; fragmentation risk

KEY IDEA
GC cost is mostly about **how much you allocate**, not how much you hold. Allocating fewer short-lived objects in hot paths directly reduces GC pressure.

Managed vs unmanaged resources

The GC only knows about memory. Things like file handles, database connections, sockets, and OS handles are **unmanaged resources**. They must be released deterministically, because waiting for the GC could keep a connection open for minutes.

IDisposable and `using`

```
public class FileLogger : IDisposable
{
    private readonly StreamWriter _writer = new("log.txt");
    private bool _disposed;

    public void Log(string msg)
    {
        ObjectDisposedException.ThrowIf(_disposed, this);
        _writer.WriteLine(msg);
    }
```

```
    public void Dispose()
    {
        if (_disposed) return;
        _writer.Dispose();       // release what we own
        _disposed = true;
    }
}

using (var log = new FileLogger()) { log.Log("hi"); }  // Dispose() called automatically
using var log2 = new FileLogger();                       // disposed at end of scope

// Async version for things like DbContext, streams
await using var ctx = new AppDb();

public class Thing : IAsyncDisposable
{
    public async ValueTask DisposeAsync() { await _stream.DisposeAsync(); }
}
```

TIP

Rule: if you *create* something that implements `IDisposable`, you dispose it. If the DI container creates it, the container disposes it. Never dispose something you were merely *given*.

HttpClient: the famous exception

Do not `new HttpClient()` per request (it can exhaust sockets). Inject `IHttpClientFactory` or a typed client; it pools handlers and respects DNS changes.

```
builder.Services.AddHttpClient<GitHubClient>(c =>
{
    c.BaseAddress = new Uri("https://api.github.com/");
    c.DefaultRequestHeaders.UserAgent.ParseAdd("MyApp");
});

public class GitHubClient(HttpClient http)
{
    public Task<Repo?> GetRepoAsync(string owner, string name) =>
        http.GetFromJsonAsync<Repo>($"repos/{owner}/{name}");
}
```

Memory leaks in a garbage-collected language

Yes, they happen. A leak in .NET means 'objects are still *reachable* when they should not be.' Common causes:

- **Event handlers** never unsubscribed: the publisher holds a reference to the subscriber forever.
- **Static collections/caches** that only grow (no eviction or size limit).
- **Singletons holding scoped/transient objects** (for instance, capturing a `DbContext` in a singleton).
- Undisposed `IDisposable`s with finalizers piling up.
- Timers and `Task`s that capture large objects and never finish.

```
// Leak: static dictionary grows forever
static readonly Dictionary<string, byte[]> Cache = new();

// Better: IMemoryCache with size limit and expiration
builder.Services.AddMemoryCache(o => o.SizeLimit = 1024);
```

```
cache.Set(key, value, new MemoryCacheEntryOptions
{
    Size = 1,
    AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10)
});
```

Span<T> and avoiding allocations

`Span<T>` is a window over existing memory (array, string, stack memory) with no copying. Slicing a `string` with `Substring` allocates a new string. Slicing a `ReadOnlySpan<char>` does not.

```
string line = "2026-10-16,Alice,42";

// Allocates 3 strings
string[] parts = line.Split(',');

// No allocations
ReadOnlySpan<char> s = line.AsSpan();
int comma = s.IndexOf(',');
ReadOnlySpan<char> date = s[..comma];

// Stack allocation for small temporary buffers
Span<byte> buffer = stackalloc byte[256];

// Array pooling: rent instead of allocate
var pool = ArrayPool<byte>.Shared;
byte[] rented = pool.Rent(4096);
try { /* use */ } finally { pool.Return(rented); }
```

KEY IDEA

`Span<T>` is a `ref struct`: it can only live on the stack. You cannot store it in a class field or use it across an `await`. Use `Memory<T>` when you need async.

Measure before optimizing

Guessing about performance is usually wrong. Use tools:

Tool	Use it for
BenchmarkDotNet	Micro-benchmarks: compare two implementations with statistics and allocation counts
`dotnet-counters` / `dotnet-trace`	Live CPU, GC, and request metrics; trace sampling
Visual Studio / Rider profiler	CPU and memory hot spots in a running app
Application Insights / OpenTelemetry	Production latency, dependency timings, failures
EF Core logging / SQL Profiler	See the actual SQL and spot N+1 queries
`dotnet-dump` / `dotnet-gcdump`	Investigate memory leaks

```
[MemoryDiagnoser]
public class JoinBenchmarks
{
    private readonly string[] _items = Enumerable.Range(0, 1000).Select(i =>
        i.ToString()).ToArray();

    [Benchmark(Baseline = true)]
```

```
    public string Concat() { var s = ""; foreach (var i in _items) s += i; return s; }

    [Benchmark]
    public string Builder() { var sb = new StringBuilder(); foreach (var i in _items)
        sb.Append(i); return sb.ToString(); }
}
// dotnet run -c Release
```

The usual suspects (what actually makes apps slow)

1. **N+1 queries**: one query for a list, then one more per item. Fix with `Include`, projection, or a single batched query.
2. **Missing database indexes** on columns used in `WHERE`, `JOIN`, `ORDER BY`.
3. **Loading too much data**: `SELECT *`, no paging, pulling whole entities when you need two columns.
4. **Sync-over-async** blocking threads (`.Result`, `.Wait()`), causing thread-pool starvation.
5. **No caching** for data that rarely changes.
6. **Chatty network calls**: many small HTTP calls where one batched call would do.
7. **Excessive allocations** in hot loops (string concatenation, LINQ in tight loops, boxing).
8. **Serialization overhead**: huge JSON payloads. Return only what the client needs; paginate; compress.
9. **Logging at verbose levels** in production or expensive string building when logging is off.

```
// N+1 bug
var orders = await db.Orders.ToListAsync();
foreach (var o in orders)
    Console.WriteLine(o.Customer.Name);        // lazy loading: 1 query PER order!

// Fix: eager load
var orders2 = await db.Orders.Include(o => o.Customer).ToListAsync();

// Better: project only what you need
var rows = await db.Orders
    .Select(o => new { o.Id, o.Total, CustomerName = o.Customer.Name })
    .ToListAsync();
```

Caching strategies

Cache	Where	Good for	Caveat
`IMemoryCache`	Inside one process	Small, hot, per-instance data	Each server instance has its own copy; lost on restart
`IDistributedCache` (Redis)	Shared external store	Multi-instance apps, sessions	Network hop; serialization; must handle misses
Output / response caching	HTTP layer	Public GET responses	Don't cache per-user data accidentally
CDN	Edge network	Static files, images	Cache invalidation on deploy
`HybridCache` (.NET 9+)	Memory + distributed	Stampede protection built in	Newer API; verify your version

```
// Cache-aside pattern: check cache, else load and store
public async Task<Product?> GetProductAsync(int id)
```

```
{
    string key = $"product:{id}";
    if (cache.TryGetValue(key, out Product? hit)) return hit;

    var product = await db.Products.FindAsync(id);
    cache.Set(key, product, TimeSpan.FromMinutes(5));
    return product;
}
// Invalidate (delete the key) whenever the product changes.
```

KEY IDEA

'There are only two hard things in computer science: cache invalidation and naming things.' Always decide how a cached value gets refreshed or evicted *before* you add the cache.

Big-O in plain English

Notation	Name	Feels like	Example
O(1)	Constant	Instant, regardless of size	Dictionary lookup, array index
O(log n)	Logarithmic	Doubling data adds one step	Binary search, balanced tree, DB index seek
O(n)	Linear	Double the data, double the time	Loop through a list, `List.Contains`
O(n log n)	Linearithmic	Slightly worse than linear	Good sorting (`List.Sort`, merge sort)
O(n squared)	Quadratic	Double the data, 4x the time	Nested loops, naive duplicate checks
O(2^n)	Exponential	Unusable beyond small n	Naive recursive Fibonacci, brute-force subsets

Interview tip: when asked to optimize, first state the current complexity, then say which data structure removes the bottleneck (usually a `HashSet` or `Dictionary` turning O(n squared) into O(n)).

8. ASP.NET Core Essentials

> **KEY IDEA**
> ASP.NET Core turns HTTP requests into method calls and method results into HTTP responses. If you understand the request pipeline, dependency injection, and configuration, everything else (auth, validation, logging) is a plug-in to those three ideas.

The request pipeline and middleware

Every request flows through a chain of **middleware** components. Each one can do work before and after the next one, or short-circuit and respond immediately. Order matters.

```
Request ->  [Exception handler] -> [HTTPS redirect] -> [Static files] -> [Routing]
        -> [CORS] -> [Authentication] -> [Authorization] -> [Endpoint (your code)]
Response <- (flows back through the same chain in reverse)
```

```
var builder = WebApplication.CreateBuilder(args);

// 1) Register services (the DI container)
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// 2) Build the pipeline (ORDER MATTERS)
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }
app.UseExceptionHandler();
app.UseHttpsRedirection();
app.UseCors();
app.UseAuthentication();     // who are you?
app.UseAuthorization();      // what may you do?
app.MapControllers();

app.Run();
```

Writing custom middleware

```
public class TimingMiddleware(RequestDelegate next, ILogger<TimingMiddleware> log)
{
    public async Task InvokeAsync(HttpContext ctx)
    {
        var sw = Stopwatch.StartNew();
        await next(ctx);                      // call the rest of the pipeline
        sw.Stop();
        log.LogInformation("{Method} {Path} -> {Status} in {Ms}ms",
            ctx.Request.Method, ctx.Request.Path, ctx.Response.StatusCode,
                sw.ElapsedMilliseconds);
    }
}
app.UseMiddleware<TimingMiddleware>();

// Quick inline version
app.Use(async (ctx, next) => { ctx.Response.Headers["X-App"] = "demo"; await next(); });
```

Dependency Injection (DI)

Instead of a class creating its own dependencies with `new`, it **declares** what it needs in its constructor and the container supplies them. This makes classes loosely coupled and trivial to test.

```
// Without DI: tightly coupled
public class OrderService
{
    // cannot swap, cannot test
    private readonly SqlOrderRepo _repo = new SqlOrderRepo("conn...");
}

// With DI
public class OrderService(IOrderRepository repo, IEmailSender email, ILogger<OrderService> log)
{
    public async Task PlaceAsync(Order o)
    {
        await repo.AddAsync(o);
        await email.SendAsync(o.CustomerEmail, "Thanks", "Order placed");
        log.LogInformation("Order {Id} placed", o.Id);
    }
}

// Registration
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();
builder.Services.AddScoped<OrderService>();
builder.Services.AddSingleton<IEmailSender, SmtpEmailSender>();
```

Service lifetimes

Lifetime	Instance created	Use for	Danger
Transient	Every time it is requested	Lightweight, stateless services	Many allocations if heavy
Scoped	Once per HTTP request (scope)	`DbContext`, unit-of-work, per-request state	Don't inject into singletons
Singleton	Once for the app lifetime	Caches, config, HTTP client factories, clocks	Must be thread-safe; can't depend on scoped

> **WATCH OUT**
> **Captive dependency**: a singleton that takes a scoped service (like `DbContext`) keeps that one instance alive forever and shares it across requests. ASP.NET Core can throw at startup in Development when it detects this (scope validation). Fix by injecting `IServiceScopeFactory` and creating a scope per operation.

Registration helpers

```
builder.Services.AddScoped<IFoo, Foo>();
builder.Services.AddScoped<IFoo>(sp => new Foo(sp.GetRequiredService<IBar>()));  // factory
// keyed (.NET 8+)
builder.Services.AddKeyedSingleton<IPaymentGateway, StripeGateway>("stripe");
builder.Services.AddTransient<IValidator<Order>, OrderValidator>();
builder.Services.TryAddScoped<IFoo, Foo>();     // only if not already registered

// Resolving a keyed service
public class Checkout([FromKeyedServices("stripe")] IPaymentGateway gw) { }
```

Controllers

```
[ApiController]
[Route("api/[controller]")]
public class OrdersController(IOrderService orders) : ControllerBase
{
    [HttpGet("{id:int}")]
    public async Task<ActionResult<OrderDto>> Get(int id, CancellationToken ct)
    {
        var order = await orders.FindAsync(id, ct);
        return order is null ? NotFound() : Ok(order);
    }

    [HttpPost]
    public async Task<ActionResult<OrderDto>> Create(CreateOrderRequest req, CancellationToken
        ct)
    {
        var created = await orders.CreateAsync(req, ct);
        // 201 + Location header
        return CreatedAtAction(nameof(Get), new { id = created.Id }, created);
    }

    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Delete(int id) { await orders.DeleteAsync(id); return
        NoContent(); }
}
```

Minimal APIs

```
var orders = app.MapGroup("/api/orders").RequireAuthorization();

orders.MapGet("/{id:int}", async (int id, IOrderService svc, CancellationToken ct) =>
    await svc.FindAsync(id, ct) is { } o ? Results.Ok(o) : Results.NotFound());

orders.MapPost("/", async (CreateOrderRequest req, IOrderService svc) =>
{
    var created = await svc.CreateAsync(req);
    return Results.Created($"/api/orders/{created.Id}", created);
});
```

	Controllers	Minimal APIs
Style	Class-based, attributes, conventions	Lambda/function-based, explicit
Best for	Large APIs with many endpoints, filters, model conventions	Small services, microservices, simple endpoints
Filters	Action filters, result filters	Endpoint filters
Organization	One controller per resource	Route groups + extension methods per feature

Model binding and validation

ASP.NET Core automatically pulls values from the request into your method parameters.

Source	Attribute	Example
Route	`[FromRoute]`	`/orders/5` -> `int id`
Query string	`[FromQuery]`	`?page=2&size=20`

Source	Attribute	Example
Body (JSON)	`[FromBody]`	Complex object on POST/PUT (default for complex types with `[ApiController]`)
Header	`[FromHeader]`	`X-Correlation-Id`
Form	`[FromForm]`	File uploads, HTML forms
DI	`[FromServices]`	Services as action parameters

```
public record CreateOrderRequest(
    [Required, EmailAddress] string CustomerEmail,
    [Range(1, 100)] int Quantity,
    [Required, MinLength(1)] List<OrderLineDto> Lines);

// With [ApiController], invalid models auto-return 400 with ProblemDetails. No manual check
    needed.

// FluentValidation (popular alternative for complex rules)
public class CreateOrderValidator : AbstractValidator<CreateOrderRequest>
{
    public CreateOrderValidator()
    {
        RuleFor(x => x.CustomerEmail).NotEmpty().EmailAddress();
        RuleFor(x => x.Lines).NotEmpty();
        RuleForEach(x => x.Lines).ChildRules(l => l.RuleFor(x => x.Qty).GreaterThan(0));
    }
}
```

DTOs: never expose your entities

A **DTO** (Data Transfer Object) is a class shaped for the API contract. Returning EF entities directly leaks internal fields (password hashes!), couples your API to your database, causes circular-reference JSON errors, and invites over-posting attacks (clients setting fields they should not).

```
public record UserDto(int Id, string Email, string DisplayName);      // what the API returns

var dto = await db.Users.Where(u => u.Id == id)
    .Select(u => new UserDto(u.Id, u.Email, u.DisplayName))           // map in the query
    .FirstOrDefaultAsync();
// AutoMapper/Mapster exist, but explicit mapping is often clearer and safer.
```

HTTP status codes cheat sheet

Code	Meaning	Use
200 OK	Success with body	GET, successful PUT
201 Created	New resource made	POST (include `Location` header)
204 No Content	Success, no body	DELETE, PUT
400 Bad Request	Client sent invalid data	Validation failure
401 Unauthorized	Not authenticated	Missing/invalid token
403 Forbidden	Authenticated but not allowed	Wrong role/permission
404 Not Found	Resource doesn't exist	

Code	Meaning	Use
409 Conflict	State conflict	Duplicate, concurrency violation
422 Unprocessable Entity	Well-formed but semantically invalid	Business-rule failure
429 Too Many Requests	Rate limited	Include `Retry-After`
500 Internal Server Error	Server bug	Unhandled exception (never leak details)
503 Service Unavailable	Overloaded / down for maintenance	

Problem Details and global exception handling

```
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
app.UseExceptionHandler();

public class GlobalExceptionHandler(ILogger<GlobalExceptionHandler> log) : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception ex, CancellationToken
         ct)
    {
        log.LogError(ex, "Unhandled exception");
        var (status, title) = ex switch
        {
            NotFoundException => (404, "Resource not found"),
            ValidationException => (400, "Validation failed"),
            _ => (500, "An unexpected error occurred")
        };
        ctx.Response.StatusCode = status;
        await ctx.Response.WriteAsJsonAsync(new ProblemDetails { Status = status, Title = title
            }, ct);
        return true;
    }
}
```

Configuration and the Options pattern

Configuration comes from layered **providers**; later ones override earlier ones: `appsettings.json` -> `appsettings.{Environment}.json` -> user secrets (Development) -> environment variables -> command-line args.

```
// appsettings.json
{
  "Smtp": { "Host": "smtp.example.com", "Port": 587 },
  "ConnectionStrings": { "Default": "Server=...;" }
}

// Strongly typed options
public class SmtpOptions
{
    public const string Section = "Smtp";
    [Required] public string Host { get; init; } = "";
    [Range(1, 65535)] public int Port { get; init; }
}

builder.Services.AddOptions<SmtpOptions>()
    .BindConfiguration(SmtpOptions.Section)
    .ValidateDataAnnotations()
    .ValidateOnStart();          // fail fast at startup if misconfigured
```

```
public class SmtpEmailSender(IOptions<SmtpOptions> opts) { string host = opts.Value.Host; }

// Environment variable override uses double underscore:  Smtp__Host=other.example.com
```

TIP
IOptions<T> = read once at startup. IOptionsSnapshot<T> = re-read per request (scoped). IOptionsMonitor<T> = live updates, usable in singletons.

Logging (Serilog + structured logs)

```
builder.Host.UseSerilog((ctx, lc) => lc
    .ReadFrom.Configuration(ctx.Configuration)
    .Enrich.FromLogContext()
    .WriteTo.Console());

// Structured logging: use message TEMPLATES, not string interpolation
// good: searchable properties
log.LogInformation("User {UserId} placed order {OrderId}", userId, orderId);
// bad: loses structure
log.LogInformation($"User {userId} placed order {orderId}");
```

Level	Use for
Trace / Debug	Developer diagnostics; off in production
Information	Normal milestones: request completed, job started
Warning	Unexpected but recoverable: retry happened, fallback used
Error	An operation failed
Critical	App cannot continue / data loss risk

WATCH OUT
Never log secrets, tokens, passwords, full card numbers, or unnecessary personal data. Logs are copied around and retained far longer than you expect.

Authentication vs authorization

- **Authentication (AuthN)**: *Who are you?* Verifying identity (password, token, Microsoft Entra ID sign-in).
- **Authorization (AuthZ)**: *What are you allowed to do?* Roles, claims, policies.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.Authority = "https://login.microsoftonline.com/<tenant-id>/v2.0";   // Entra ID
        o.Audience = "api://my-api";
    });

builder.Services.AddAuthorizationBuilder()
    .AddPolicy("CanApprove", p => p.RequireRole("Manager").RequireClaim("department",
        "claims"));

[Authorize]                                  // must be signed in
[Authorize(Roles = "Admin")]                 // role check
```

```
[Authorize(Policy = "CanApprove")]          // policy check
[AllowAnonymous]                            // opt out
```

CORS, rate limiting, health checks

```
builder.Services.AddCors(o => o.AddPolicy("Web", p => p
    .WithOrigins("https://myapp.com")     // never AllowAnyOrigin together with credentials
    .AllowAnyHeader().AllowAnyMethod()));

builder.Services.AddRateLimiter(o => o.AddFixedWindowLimiter("api", w =>
{
    w.PermitLimit = 100; w.Window = TimeSpan.FromMinutes(1);
}));
app.UseRateLimiter();
app.MapControllers().RequireRateLimiting("api");

builder.Services.AddHealthChecks().AddDbContextCheck<AppDb>();
app.MapHealthChecks("/health");        // used by load balancers, Kubernetes, Azure
```

API design best practices

- Use **nouns** for resources and HTTP verbs for actions: `GET /orders/5`, not `GET /getOrder?id=5`.
- **Version** your API (`/api/v1/...` or header) before clients depend on it.
- **Paginate** every list endpoint (`?page=1&pageSize=20` or cursor-based) and cap the page size.
- Make `PUT`/`DELETE` **idempotent**: repeating them yields the same result. For safe retries of `POST`, accept an `Idempotency-Key` header.
- Return consistent error shapes (RFC 7807 Problem Details).
- Document with OpenAPI/Swagger; keep DTOs stable.
- Use **ETags / optimistic concurrency** to prevent lost updates.

Verb	Safe?	Idempotent?	Typical use
GET	Yes	Yes	Read
POST	No	No	Create / non-idempotent action
PUT	No	Yes	Replace whole resource
PATCH	No	Not guaranteed	Partial update
DELETE	No	Yes	Remove

Practice

1. Walk through what happens from the moment a request hits Kestrel to the moment your controller returns.
2. Explain Transient vs Scoped vs Singleton. Which would you use for `DbContext`? A clock? A cache?
3. Why is order of `UseAuthentication` and `UseAuthorization` important?
4. Why should you not return EF entities from controllers?
5. What is the Options pattern and why use `ValidateOnStart`?
6. 401 vs 403: what is the difference?

9. EF Core and SQL Foundations

KEY IDEA
An ORM like EF Core hides SQL, but it does not remove the need to understand it. When something is slow or wrong, the answer is always in the SQL that was generated. This chapter covers both: the SQL you should know by heart, and how EF Core maps onto it.

Relational database basics

- A **table** holds rows (records) with typed columns.
- A **primary key (PK)** uniquely identifies each row.
- A **foreign key (FK)** points to a PK in another table and enforces that the referenced row exists.
- An **index** is a sorted structure (usually a B-tree) that makes lookups fast at the cost of slower writes and extra storage.
- A **transaction** groups statements so they succeed or fail together.

SQL you must know

```
-- Create
CREATE TABLE Customers (
    Id INT PRIMARY KEY IDENTITY,
    Name NVARCHAR(100) NOT NULL,
    Email NVARCHAR(200) NOT NULL UNIQUE,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);
CREATE TABLE Orders (
    Id INT PRIMARY KEY IDENTITY,
    CustomerId INT NOT NULL REFERENCES Customers(Id),
    Total DECIMAL(18,2) NOT NULL,
    Status VARCHAR(20) NOT NULL
);

-- Insert / Update / Delete
INSERT INTO Customers (Name, Email) VALUES ('Ana', 'ana@x.com');
UPDATE Orders SET Status = 'Paid' WHERE Id = 7;
DELETE FROM Orders WHERE Status = 'Cancelled' AND Total = 0;

-- Select with filtering, sorting, paging
SELECT Id, Name FROM Customers
WHERE Name LIKE 'A%' AND CreatedAt >= '2026-01-01'
ORDER BY Name
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;      -- (LIMIT 10 OFFSET 20 in PostgreSQL/MySQL)
```

Joins

```
-- INNER JOIN: only rows that match on both sides
SELECT c.Name, o.Total
FROM Customers c
INNER JOIN Orders o ON o.CustomerId = c.Id;

-- LEFT JOIN: all customers, even those with no orders (Order columns NULL)
SELECT c.Name, o.Total
```

```
FROM Customers c
LEFT JOIN Orders o ON o.CustomerId = c.Id;

-- Find customers with NO orders
SELECT c.* FROM Customers c
LEFT JOIN Orders o ON o.CustomerId = c.Id
WHERE o.Id IS NULL;
```

Join	Returns
`INNER JOIN`	Rows with a match in both tables
`LEFT JOIN`	All rows from the left table + matches from the right (or NULL)
`RIGHT JOIN`	Mirror of left join (rarely used)
`FULL OUTER JOIN`	All rows from both sides
`CROSS JOIN`	Every combination (cartesian product)

Aggregation

```
SELECT c.Name, COUNT(*) AS OrderCount, SUM(o.Total) AS Spent, AVG(o.Total) AS Average
FROM Customers c
JOIN Orders o ON o.CustomerId = c.Id
WHERE o.Status = 'Paid'              -- filters ROWS before grouping
GROUP BY c.Name
HAVING SUM(o.Total) > 1000           -- filters GROUPS after aggregation
ORDER BY Spent DESC;
```

KEY IDEA

WHERE filters rows before grouping. **HAVING** filters groups after aggregation. The logical order is FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT.

Subqueries, CTEs, window functions

```
-- CTE (Common Table Expression): a named temporary result
WITH BigSpenders AS (
    SELECT CustomerId, SUM(Total) AS Spent
    FROM Orders GROUP BY CustomerId HAVING SUM(Total) > 1000
)
SELECT c.Name, b.Spent FROM BigSpenders b JOIN Customers c ON c.Id = b.CustomerId;

-- Window function: rank without collapsing rows
SELECT Id, CustomerId, Total,
       ROW_NUMBER() OVER (PARTITION BY CustomerId ORDER BY Total DESC) AS RankInCustomer,
       SUM(Total)  OVER (PARTITION BY CustomerId) AS CustomerTotal
FROM Orders;

-- Latest order per customer (common interview question)
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerId ORDER BY CreatedAt DESC) rn FROM Orders
) t WHERE rn = 1;
```

Indexes: what to know

```
CREATE INDEX IX_Orders_CustomerId ON Orders (CustomerId);
CREATE INDEX IX_Orders_Status_Created ON Orders (Status, CreatedAt) INCLUDE (Total);   --
    composite + covering
CREATE UNIQUE INDEX UX_Customers_Email ON Customers (Email);
```

- Index columns used in `WHERE`, `JOIN ON`, and `ORDER BY`. Foreign keys usually deserve an index.
- **Composite index order matters**: an index on `(Status, CreatedAt)` helps queries filtering on `Status` or on `Status AND CreatedAt`, but not on `CreatedAt` alone (leftmost-prefix rule).
- **Functions on indexed columns kill index use**: `WHERE YEAR(CreatedAt) = 2026` scans; `WHERE CreatedAt >= '2026-01-01' AND CreatedAt < '2027-01-01'` seeks.
- **Covering indexes** (`INCLUDE`) satisfy a query entirely from the index without touching the table.
- Every index slows `INSERT/UPDATE/DELETE`. Do not index everything.
- Read the **execution plan** to see seeks vs scans.

Transactions and ACID

Letter	Property	Meaning
A	Atomicity	All statements in the transaction succeed, or none do
C	Consistency	Data moves from one valid state to another (constraints hold)
I	Isolation	Concurrent transactions don't see each other's half-finished work
D	Durability	Once committed, data survives crashes

```
BEGIN TRANSACTION;
UPDATE Accounts SET Balance = Balance - 100 WHERE Id = 1;
UPDATE Accounts SET Balance = Balance + 100 WHERE Id = 2;
COMMIT;      -- or ROLLBACK on error
```

Isolation level	Prevents	Notes
Read Uncommitted	Nothing	Dirty reads possible
Read Committed (default in SQL Server/Postgres)	Dirty reads	Non-repeatable reads still possible
Repeatable Read	+ non-repeatable reads	Phantoms possible in some engines
Serializable	+ phantom reads	Safest, most blocking
Snapshot	Readers don't block writers	Row versioning; uses tempdb/extra storage

Normalization in 60 seconds

- **1NF**: each cell holds one value; no repeating groups (no comma-separated lists in a column).
- **2NF**: every non-key column depends on the *whole* key.
- **3NF**: non-key columns depend *only* on the key, not on other non-key columns ('the key, the whole key, and nothing but the key').
- **Denormalize deliberately** for read performance (reporting tables, caches), knowing you take on the job of keeping copies consistent.

EF Core: the model

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Email { get; set; } = "";
    public List<Order> Orders { get; set; } = new();       // navigation: one-to-many
}

public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }                    // foreign key
    public Customer Customer { get; set; } = null!;        // navigation: many-to-one
    public decimal Total { get; set; }
    public OrderStatus Status { get; set; }
    public byte[] RowVersion { get; set; } = [];           // concurrency token
}

public class AppDb(DbContextOptions<AppDb> options) : DbContext(options)
{
    public DbSet<Customer> Customers => Set<Customer>();
    public DbSet<Order> Orders => Set<Order>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        b.Entity<Customer>(e =>
        {
            e.HasIndex(c => c.Email).IsUnique();
            e.Property(c => c.Name).HasMaxLength(100).IsRequired();
        });
        b.Entity<Order>(e =>
        {
            e.Property(o => o.Total).HasPrecision(18, 2);
            e.Property(o => o.Status).HasConversion<string>().HasMaxLength(20);
            e.Property(o => o.RowVersion).IsRowVersion();
            e.HasOne(o => o.Customer).WithMany(c => c.Orders).HasForeignKey(o => o.CustomerId)
             .OnDelete(DeleteBehavior.Restrict);
        });
        // or: b.ApplyConfigurationsFromAssembly(typeof(AppDb).Assembly);
    }
}

// Registration (scoped by default)
builder.Services.AddDbContext<AppDb>(o => o.UseSqlServer(cs));   // or UseNpgsql / UseMySql
```

EF Core: querying

```
// Read-only queries: AsNoTracking is faster (no change tracking)
var customers = await db.Customers
    .AsNoTracking()
    .Where(c => c.Name.StartsWith("A"))
    .OrderBy(c => c.Name)
    .Skip(0).Take(20)
    .ToListAsync(ct);

// Eager loading
var withOrders = await db.Customers.Include(c => c.Orders).FirstAsync(c => c.Id == 1);

// Projection: fetch only what you need (best for APIs)
var summaries = await db.Customers
    .Select(c => new CustomerSummary(c.Id, c.Name, c.Orders.Count, c.Orders.Sum(o => o.Total)))
    .ToListAsync();

// Single lookups
var byKey = await db.Customers.FindAsync(5);                  // checks the tracker cache first
var one = await db.Customers.SingleOrDefaultAsync(c => c.Email == email);
bool exists = await db.Customers.AnyAsync(c => c.Email == email);

// Split large includes to avoid cartesian explosion
.Include(c => c.Orders).ThenInclude(o => o.Lines).AsSplitQuery()

// Raw SQL when needed (parameterized!)
var rows = await db.Customers.FromSqlInterpolated($"SELECT * FROM Customers WHERE Name =
    {name}").ToListAsync();
```

EF Core: writing and change tracking

EF Core **tracks** entities it loads. When you call `SaveChangesAsync`, it compares current values with the originals and generates the `INSERT`/`UPDATE`/`DELETE` statements, all in one transaction.

```
// Insert
var c = new Customer { Name = "Ana", Email = "ana@x.com" };
db.Customers.Add(c);
await db.SaveChangesAsync();         // c.Id is now populated

// Update (tracked entity: just change it)
var cust = await db.Customers.FindAsync(1);
cust!.Name = "Ana Maria";
await db.SaveChangesAsync();         // UPDATE Customers SET Name = ... WHERE Id = 1

// Delete
db.Customers.Remove(cust);
await db.SaveChangesAsync();

// Bulk operations without loading entities (EF Core 7+)
await db.Orders.Where(o => o.Status == OrderStatus.Cancelled).ExecuteDeleteAsync();
await db.Orders.Where(o => o.Total == 0).ExecuteUpdateAsync(s => s.SetProperty(o => o.Status,
    OrderStatus.Cancelled));

// Explicit transaction across several SaveChanges calls
await using var tx = await db.Database.BeginTransactionAsync();
try { /* work */ await db.SaveChangesAsync(); await tx.CommitAsync(); }
catch { await tx.RollbackAsync(); throw; }
```

Concurrency control

Two users edit the same record. Without protection, the last save silently wins. **Optimistic concurrency** uses a version column: the `UPDATE` includes `WHERE RowVersion = @original`. If someone else changed the row, zero rows are affected and EF throws `DbUpdateConcurrencyException`.

```
try { await db.SaveChangesAsync(); }
catch (DbUpdateConcurrencyException)
{
    return Conflict("Someone else changed this record. Reload and try again.");   // HTTP 409
}
```

Migrations

```
dotnet ef migrations add AddOrderStatus          # generates C# describing the schema change
dotnet ef migrations script --idempotent         # SQL script for DBAs / pipelines
dotnet ef database update                        # apply to the database
dotnet ef migrations remove                      # undo the last (unapplied) migration
```

- Commit migrations to source control; treat them as code.
- Review generated migrations. EF may drop and re-create a column when you meant to rename it.
- In production, prefer applying a **generated SQL script** or a dedicated migration step in CI/CD, not `Database.Migrate()` at app startup with multiple instances racing.
- Make destructive changes in steps (expand, migrate data, contract) to enable zero-downtime deploys.

EF Core gotchas checklist

Gotcha	Fix
N+1 queries from lazy loading or loops	`Include`, projection, or batch query
Returning entities from APIs	Project into DTOs
Tracking overhead on read-only queries	`AsNoTracking()`
`ToList()` before `Where`	Keep the query `IQueryable` until the end
Cartesian explosion with many `Include`s	`AsSplitQuery()` or separate queries
Using `DbContext` from multiple threads	One context per scope/operation
Long-lived `DbContext` growing its tracker	Short-lived scopes; call `ChangeTracker.Clear()` in batch jobs
String concatenation into raw SQL	Parameterized `FromSqlInterpolated` (SQL injection!)
Decimal precision warnings	`HasPrecision(18,2)`
Missing indexes	Add via `HasIndex`; check query plans

The repository pattern: do you need it with EF Core?

`DbContext` already *is* a unit of work and `DbSet<T>` already *is* a repository. A thin generic repository on top often just adds indirection and leaks `IQueryable` anyway. Many teams skip it and put queries in handlers/services. Use a repository when you want to hide persistence details behind a domain-focused interface (for example `IOrderRepository.GetOpenOrdersFor(customerId)`), or to swap storage in tests.

SQL vs NoSQL: a quick decision guide

Choose	When
Relational (SQL Server, PostgreSQL, MySQL)	Structured data, relationships, transactions, ad-hoc queries, strong consistency. Default choice.
Document (MongoDB, Cosmos DB)	Flexible/nested schemas, whole-aggregate reads/writes, horizontal scale
Key-value (Redis, DynamoDB)	Caching, sessions, counters, very fast lookups by key
Column-family / wide-column (Cassandra)	Massive write throughput, time-series
Search (Elasticsearch, Azure AI Search)	Full-text search, relevance ranking, faceting
Graph (Neo4j)	Highly connected data, recommendations, traversals

10. SOLID, Clean Code, and Architecture

KEY IDEA
SOLID is a set of five guidelines for writing code that is easy to change. They are not laws; they are warning lights. When a change in one place forces edits in ten others, one of these principles is probably being violated.

S: Single Responsibility Principle

A class should have **one reason to change**. If a class changes when the tax rules change *and* when the email template changes *and* when the database changes, it is doing too much.

```
// BAD: one class, four reasons to change
public class OrderService
{
    public void Place(Order o)
    {
        if (o.Lines.Count == 0) throw new Exception("empty");        // validation
        o.Total = o.Lines.Sum(l => l.Price * l.Qty) * 1.07m;         // pricing/tax
        using var conn = new SqlConnection("..."); /* INSERT ... */  // persistence
        new SmtpClient().Send(/* confirmation email */);              // notification
    }
}

// GOOD: each piece has one job; OrderService coordinates
public class OrderService(IOrderValidator validator, IPriceCalculator pricing,
                          IOrderRepository repo, INotifier notifier)
{
    public async Task PlaceAsync(Order o)
    {
        validator.EnsureValid(o);
        o.Total = pricing.Calculate(o);
        await repo.AddAsync(o);
        await notifier.OrderPlacedAsync(o);
    }
}
```

O: Open/Closed Principle

Code should be **open for extension, closed for modification**. Add new behavior by adding new code, not by editing tested code. The usual tell: a growing `if/else` or `switch` on a type.

```
// BAD: every new discount means editing this method
public decimal Discount(Customer c) => c.Type switch
{
    "Regular" => 0m,
    "Silver"  => 0.05m,
    "Gold"    => 0.10m,
    _ => 0m
};

// GOOD: add a new class, register it, done
public interface IDiscountRule { bool AppliesTo(Customer c); decimal Rate { get; } }
public class GoldRule : IDiscountRule { public bool AppliesTo(Customer c) => c.Type == "Gold";
    public decimal Rate => 0.10m; }

// DI injects ALL registrations
public class DiscountService(IEnumerable<IDiscountRule> rules)
```

```
{
    public decimal For(Customer c) => rules.Where(r => r.AppliesTo(c)).Select(r =>
        r.Rate).DefaultIfEmpty(0).Max();
}
```

L: Liskov Substitution Principle

Anywhere you use a base type, you must be able to substitute a subtype **without surprising behavior**. If a subclass throws `NotSupportedException` for an inherited method, or strengthens preconditions, the hierarchy is wrong.

```
// Classic violation
public class Rectangle { public virtual int W { get; set; } public virtual int H { get; set; } }
public class Square : Rectangle
{
    // surprise: setting W also changes H
    public override int W { set { base.W = base.H = value; } }
}
// Code that does r.W = 4; r.H = 5; and expects area 20 breaks when r is a Square.

// Another sign
public class ReadOnlyRepo : IRepository { public void Add(Entity e) => throw new
    NotSupportedException(); }
// Fix: split into IReadRepository and IWriteRepository (see ISP).
```

I: Interface Segregation Principle

Prefer **small, focused interfaces** over one fat interface. No class should be forced to implement methods it doesn't use.

```
// BAD
// a Robot can't eat or sleep
public interface IWorker { void Work(); void Eat(); void Sleep(); }

// GOOD
public interface IWorkable { void Work(); }
public interface IFeedable { void Eat(); }
public class Robot : IWorkable { public void Work() { } }
public class Human : IWorkable, IFeedable { public void Work() { } public void Eat() { } }
```

D: Dependency Inversion Principle

High-level policy should not depend on low-level details; both depend on abstractions. Your business logic should depend on `IOrderRepository`, not on `SqlOrderRepository`. The low-level class then *implements* the interface the high-level code owns.

```
// BEFORE: Application -> Infrastructure (high-level knows the low-level class)
public class ReportService { private readonly SqlDatabase _db = new(); }

// AFTER: both point at an abstraction owned by the high-level layer
public interface IReportData { Task<IReadOnlyList<Row>> GetRowsAsync(); }
public class ReportService(IReportData data) { }
public class SqlReportData : IReportData { ... }      // lives in Infrastructure
```

KEY IDEA
Dependency Inversion (a design principle) is not the same as **Dependency Injection** (a technique) or **IoC containers** (a tool). DI is how you *achieve* DIP in practice.

Other principles worth knowing

Principle	Meaning
DRY	Don't Repeat Yourself: one source of truth for each piece of knowledge. (But don't merge things that only *look* similar. Wrong abstraction is worse than duplication.)
KISS	Keep It Simple. The simplest design that works wins.
YAGNI	You Aren't Gonna Need It: don't build features or abstractions for hypothetical futures.
Law of Demeter	Talk to your friends, not strangers: avoid `a.GetB().GetC().DoThing()` chains.
Separation of concerns	Keep UI, business rules, and data access in different places.
Tell, don't ask	Tell an object to do something instead of pulling its data out and deciding for it.
Fail fast	Validate early; crash loudly on invalid state rather than limping along.
Principle of least astonishment	Behave the way a reader would expect.

Clean code habits

- **Names reveal intent**: `daysSinceLastLogin` beats `d`. Methods are verbs (`CalculateTotal`), classes are nouns.
- **Small methods** that do one thing at one level of abstraction. If you need a comment to explain a block, extract a method whose name is the comment.
- **Few parameters** (aim for 3 or fewer). Group related ones into a record.
- **No magic numbers/strings**: use named constants or enums.
- **Prefer early returns** (guard clauses) over deep nesting.
- **Comments explain why, not what.** The code says what.
- **Avoid boolean flag parameters**: `Save(order, true)` is unreadable. Split methods or use an enum.
- **Immutability by default**: `readonly`, `init`, records. Fewer moving parts.

```
// Nested and hard to follow
public decimal Ship(Order o)
{
    if (o != null)
    {
        if (o.Items.Count > 0)
        {
            if (o.Total > 100) { return 0; } else { return 9.99m; }
        }
    }
    return 0;
}

// Guard clauses: flat and readable
```

```
public decimal Ship(Order o)
{
    ArgumentNullException.ThrowIfNull(o);
    if (o.Items.Count == 0) return 0;
    return o.Total > FreeShippingThreshold ? 0 : StandardShipping;
}
```

Code smells (when to refactor)

Smell	Symptom	Typical fix
Long method	Scrolls for pages	Extract method
God class	Class that knows/does everything	Split by responsibility
Feature envy	Method uses another class's data more than its own	Move the method
Primitive obsession	Strings and ints for emails, money, IDs	Value objects / records
Shotgun surgery	One change touches many files	Consolidate related logic
Duplicate code	Copy-pasted blocks	Extract and reuse (carefully)
Long parameter list	6+ parameters	Parameter object
Switch on type	Repeated `switch`/`if` on a kind	Polymorphism / strategy
Comments as deodorant	Comment explains tangled code	Rename/extract instead

Value objects: fixing primitive obsession

```
public readonly record struct Email
{
    public string Value { get; }
    public Email(string value)
    {
        if (string.IsNullOrWhiteSpace(value) || !value.Contains('@'))
            throw new ArgumentException("Invalid email", nameof(value));
        Value = value.Trim().ToLowerInvariant();
    }
    public override string ToString() => Value;
}

// Impossible to pass a random string where an email belongs; validation lives in one place.
void Invite(Email to) { }

// strongly typed ids prevent mixing up Guids
public readonly record struct OrderId(Guid Value);
```

Layered architecture

```
+---------------------------------------------+
|  Presentation (Controllers, Razor, React)    |  talks HTTP, shapes responses
+---------------------------------------------+
|  Application (services, use cases, DTOs)     |  orchestrates a use case
+---------------------------------------------+
|  Domain (entities, value objects, rules)     |  the business, framework-free
+---------------------------------------------+
|  Infrastructure (EF Core, email, storage)    |  details: DB, APIs, files
```

```
+---------------------------------------------+
Classic layering: each layer depends on the one below.
```

Clean / Onion / Hexagonal architecture

These related styles **invert** the dependency on infrastructure: the Domain sits at the center and depends on nothing. Application depends on Domain. Infrastructure and API depend on Application, implementing the interfaces Application defines. The database and the web framework become replaceable details.

```
            +------------- API (controllers, DI composition root) --------------+
            |      +------------- Infrastructure (EF Core, SMTP, S3) -------+   |
            |      |      +----------- Application (use cases) -----+      |   |
            |      |      |      +------- Domain (entities) ----+   |      |   |
            |      |      |      |  no outward dependencies     |   |      |   |
            |      |      |      +------------------------------+   |      |   |
            |      |      +-----------------------------------------+      |   |
            |      +-------------------------------------------------------+   |
            +-------------------------------------------------------------- --+
                      Arrows of dependency all point INWARD.
```

> **TIP**
> Do not over-architect small apps. A 5-endpoint CRUD API does not need 6 projects, MediatR, and a repository per table. Start simple and add structure when the pain appears. Your Eltse Hall layering is justified because the domain has real rules (scheduling, role-based access).

Vertical slices (feature folders)

Instead of grouping by technical layer (Controllers/, Services/, Repositories/), group by **feature**: everything for 'PlaceOrder' (endpoint, validator, handler, DTOs) lives together. Changing a feature touches one folder. Often paired with CQRS and MediatR-style handlers.

```
Features/
  Orders/
    PlaceOrder/
      PlaceOrderEndpoint.cs
      PlaceOrderCommand.cs
      PlaceOrderHandler.cs
      PlaceOrderValidator.cs
    GetOrder/
      GetOrderQuery.cs
      GetOrderHandler.cs
```

CQRS in plain English

Command Query Responsibility Segregation: separate the model for *changing* data (commands: `PlaceOrder`) from the model for *reading* data (queries: `GetOrderSummary`). Commands enforce rules and return little; queries are optimized for display and may skip the domain model entirely (straight to projections or even a separate read database).

```
public record PlaceOrderCommand(Guid CustomerId, List<LineDto> Lines);
public record GetOrderQuery(Guid OrderId);

public class PlaceOrderHandler(AppDb db)
{
    public async Task<Result<Guid>> HandleAsync(PlaceOrderCommand cmd, CancellationToken ct) {
```

```
        ... }
}
public class GetOrderHandler(AppDb db)
{
    public Task<OrderDto?> HandleAsync(GetOrderQuery q, CancellationToken ct) =>
        db.Orders.AsNoTracking().Where(o => o.Id == q.OrderId).Select(o => new
            OrderDto(...)).FirstOrDefaultAsync(ct);
}
```

Domain-Driven Design (DDD) vocabulary

Term	Meaning	Example
Ubiquitous language	Developers and domain experts use the same words	'Claim', 'Adjuster', 'Correspondence'
Entity	Has identity that persists over time	`Order` with an Id
Value object	Defined by its values, immutable, no identity	`Money`, `Address`, `DateRange`
Aggregate	A cluster of objects treated as one unit for changes, with a root entity	`Order` (root) + `OrderLine`s
Aggregate root	The only entry point to modify the aggregate; enforces invariants	`order.AddLine(...)`
Repository	Collection-like access to aggregates	`IOrderRepository`
Domain event	Something that happened in the domain	`OrderPlaced`
Bounded context	A boundary within which a model/language is consistent	'Billing' vs 'Shipping' each have their own `Customer`

```
public class Order            // aggregate root
{
    private readonly List<OrderLine> _lines = new();
    public IReadOnlyList<OrderLine> Lines => _lines;
    public OrderStatus Status { get; private set; } = OrderStatus.Draft;

    public void AddLine(Product p, int qty)
    {
        if (Status != OrderStatus.Draft) throw new DomainException("Cannot modify a submitted
            order");
        if (qty <= 0) throw new DomainException("Quantity must be positive");
        _lines.Add(new OrderLine(p.Id, p.Price, qty));
    }

    public void Submit()
    {
        if (_lines.Count == 0) throw new DomainException("Cannot submit an empty order");
        Status = OrderStatus.Submitted;
    }
}
```

Monolith, modular monolith, microservices

	Monolith	Modular monolith	Microservices
Deployment	One unit	One unit	Many independent units
Boundaries	Often blurry	Enforced modules inside one app	Network boundaries
Complexity	Low	Medium	High: networking, observability, data consistency
Scaling	Whole app	Whole app	Per service
Team fit	Small teams	Mid-size teams	Many autonomous teams
Good start?	Yes	Yes, best default	Rarely at the beginning

KEY IDEA

'Start with a modular monolith' is widely recommended: you get clean boundaries without paying the distributed-systems tax. Extract a service later only when a boundary truly needs independent scaling or deployment.

Practice

1. Explain each SOLID letter with a one-line example.
2. What is the difference between Dependency Inversion and Dependency Injection?
3. Refactor a method with a 6-case `switch` on customer type using the strategy pattern.
4. When is a repository on top of EF Core worth it? When is it ceremony?
5. What is an aggregate root and why route all changes through it?
6. Argue for and against microservices for a team of four.

11. Design Patterns in C#

KEY IDEA
Design patterns are named, reusable solutions to problems that keep showing up. Their real value is vocabulary: saying 'use a decorator here' communicates a whole design in four words. Learn what problem each solves, and recognize the ones .NET already gives you.

How to use this chapter

Each pattern has: **Problem** (when you need it), **Idea**, a compact **C# example**, and **Where you've seen it** in .NET. Patterns come in three families:

Family	Concern	Patterns covered
Creational	How objects get created	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural	How objects are composed	Adapter, Decorator, Facade, Proxy, Composite
Behavioral	How objects communicate / share responsibility	Strategy, Observer, Command, Mediator, Template Method, State, Chain of Responsibility, Iterator
Enterprise / application	Typical app-level structure	Repository, Unit of Work, Specification, Outbox, Result, CQRS

KEY IDEA
Do not force patterns. Reach for one when you feel the specific pain it solves (a growing switch, hard-to-test creation logic, duplicated cross-cutting code). Patterns that appear without a problem become over-engineering.

CREATIONAL PATTERNS

Singleton

Problem: exactly one instance must exist (config, cache, clock). **Idea:** hold the single instance and give global access. **Modern advice:** do *not* hand-write singletons; register the class as a `Singleton` in the DI container so it stays testable.

```
// Hand-rolled (thread-safe via Lazy<T>) - avoid unless you must
public sealed class AppClock
{
    private static readonly Lazy<AppClock> _instance = new(() => new AppClock());
    public static AppClock Instance => _instance.Value;
    private AppClock() { }
    public DateTimeOffset Now => DateTimeOffset.UtcNow;
}

// Preferred: let DI manage the lifetime
builder.Services.AddSingleton<IClock, SystemClock>();
```

Seen in .NET: `ILogger` factories, `HttpClientFactory`, `IMemoryCache` (registered as singletons).

Factory Method

Problem: the code that *uses* an object should not know which concrete class to create. **Idea:** delegate creation to a method or class that returns an abstraction.

```
public interface INotifier { Task SendAsync(string to, string msg); }
public class EmailNotifier : INotifier { public Task SendAsync(string to, string msg) =>
    Task.CompletedTask; }
public class SmsNotifier   : INotifier { public Task SendAsync(string to, string msg) =>
    Task.CompletedTask; }

public class NotifierFactory(IServiceProvider sp)
{
    public INotifier Create(string channel) => channel switch
    {
        "email" => sp.GetRequiredService<EmailNotifier>(),
        "sms"   => sp.GetRequiredService<SmsNotifier>(),
        _ => throw new ArgumentException($"Unknown channel {channel}")
    };
}
// Usage: var n = factory.Create(user.PreferredChannel); await n.SendAsync(...);
```

Seen in .NET: `IHttpClientFactory.CreateClient`, `DbContextFactory`, `Activator.CreateInstance`, keyed services.

Abstract Factory

Problem: you need *families* of related objects that must be used together (for example, a UI kit for Windows vs macOS, or a cloud provider's storage + queue + secrets). **Idea:** a factory interface that creates a whole family.

```
public interface ICloudFactory { IStorage CreateStorage(); IQueue CreateQueue(); }
public class AzureFactory : ICloudFactory
{
    public IStorage CreateStorage() => new BlobStorage();
    public IQueue CreateQueue() => new ServiceBusQueue();
}
public class AwsFactory : ICloudFactory
{
    public IStorage CreateStorage() => new S3Storage();
    public IQueue CreateQueue() => new SqsQueue();
}
```

Builder

Problem: constructing an object takes many steps/options and a constructor with 10 parameters is unreadable. **Idea:** build step by step with a fluent API, then call `Build()`.

```
public class EmailMessage
{
    public string To { get; init; } = ""; public string Subject { get; init; } = "";
    public string Body { get; init; } = ""; public List<string> Cc { get; init; } = new();
}

public class EmailBuilder
{
    private string _to = "", _subject = "", _body = ""; private readonly List<string> _cc =
        new();
    public EmailBuilder To(string to) { _to = to; return this; }
    public EmailBuilder Subject(string s) { _subject = s; return this; }
    public EmailBuilder Body(string b) { _body = b; return this; }
    public EmailBuilder Cc(string c) { _cc.Add(c); return this; }
```

```
    public EmailMessage Build()
    {
        if (string.IsNullOrWhiteSpace(_to)) throw new InvalidOperationException("To is
            required");
        return new EmailMessage { To = _to, Subject = _subject, Body = _body, Cc = _cc };
    }
}

var mail = new
    EmailBuilder().To("ana@x.com").Subject("Hi").Body("...").Cc("boss@x.com").Build();
```

Seen in .NET: `WebApplication.CreateBuilder`, `StringBuilder`, `UriBuilder`, `ConfigurationBuilder`, `HostBuilder`.

Prototype

Problem: creating an object is costly, but copying an existing one is cheap. **Idea:** clone an existing instance. In C#, `record` + `with` gives you this for free.

```
var template = new ReportSettings("PDF", true, "Letter");
var variant = template with { Format = "HTML" };     // copy with a change
```

STRUCTURAL PATTERNS

Adapter

Problem: you have a class with the wrong interface (a third-party SDK, a legacy API), but your code expects a different one. **Idea:** wrap it in a class that translates. This is how you keep vendor types out of your domain.

```
// Your app's interface
public interface IPaymentGateway { Task<bool> ChargeAsync(decimal amount, string token); }

// Third-party SDK with a different shape
public class StripeSdkClient { public Task<StripeCharge> CreateChargeAsync(long cents, string
    source, string currency) => ...; }

// Adapter
public class StripeGateway(StripeSdkClient sdk) : IPaymentGateway
{
    public async Task<bool> ChargeAsync(decimal amount, string token)
    {
        var charge = await sdk.CreateChargeAsync((long)(amount * 100), token, "usd");
        return charge.Status == "succeeded";
    }
}
```

Decorator

Problem: add behavior (logging, caching, retry, metrics) to an object **without modifying it** and without subclass explosions. **Idea:** wrap an object implementing the same interface and delegate to it, adding behavior before/after. Decorators stack.

```
public interface IProductService { Task<Product?> GetAsync(int id); }

public class ProductService(AppDb db) : IProductService
{
    public Task<Product?> GetAsync(int id) => db.Products.FindAsync(id).AsTask();
}
```

```
public class CachedProductService(IProductService inner, IMemoryCache cache) : IProductService
{
    public async Task<Product?> GetAsync(int id) =>
        await cache.GetOrCreateAsync($"p:{id}", e =>
        {
            e.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
            return inner.GetAsync(id);
        });
}

public class LoggingProductService(IProductService inner, ILogger<LoggingProductService> log) :
    IProductService
{
    public async Task<Product?> GetAsync(int id)
    {
        log.LogInformation("Fetching product {Id}", id);
        return await inner.GetAsync(id);
    }
}

// Compose: Logging -> Cache -> Real
builder.Services.AddScoped<ProductService>();
builder.Services.AddScoped<IProductService>(sp =>
    new LoggingProductService(
        new CachedProductService(sp.GetRequiredService<ProductService>(),
            sp.GetRequiredService<IMemoryCache>()),
        sp.GetRequiredService<ILogger<LoggingProductService>>()));
// The Scrutor library offers services.Decorate<IProductService, CachedProductService>();
```

Seen in .NET: ASP.NET Core middleware pipeline, `Stream` wrappers (`GZipStream` wraps another stream), `DelegatingHandler` for `HttpClient`, MediatR pipeline behaviors.

Facade

Problem: a subsystem has many classes and complex steps; callers just want one simple call. **Idea:** provide a single, simple entry point.

```
public class CheckoutFacade(IInventory inv, IPayments pay, IShipping ship, IReceipts receipts)
{
    public async Task<Result> CheckoutAsync(Cart cart, PaymentInfo p)
    {
        if (!await inv.ReserveAsync(cart)) return Result.Fail("Out of stock");
        if (!await pay.ChargeAsync(cart.Total, p)) { await inv.ReleaseAsync(cart); return
            Result.Fail("Payment failed"); }
        await ship.ScheduleAsync(cart);
        await receipts.EmailAsync(cart);
        return Result.Ok();
    }
}
```

Proxy

Problem: control access to an object: lazy loading, access control, remote calls, caching. **Idea:** a stand-in with the same interface. **Seen in .NET:** EF Core lazy-loading proxies, WCF/gRPC client stubs, `DispatchProxy`, Castle DynamicProxy in mocking libraries.

Composite

Problem: treat individual items and groups of items uniformly (file system folders and files, menus, org charts). **Idea:** a common interface; containers hold children of that interface.

```
public interface IFileNode { string Name { get; } long Size { get; } }
public class FileNode(string name, long size) : IFileNode { public string Name => name; public
    long Size => size; }
public class FolderNode(string name) : IFileNode
{
    private readonly List<IFileNode> _children = new();
    public string Name => name;
    public long Size => _children.Sum(c => c.Size);     // recursion through the same interface
    public void Add(IFileNode n) => _children.Add(n);
}
```

BEHAVIORAL PATTERNS

Strategy

Problem: several interchangeable algorithms (pricing rules, sorting, shipping calculators) and a growing `switch`. **Idea:** put each algorithm in its own class behind one interface and inject the one you want. This is the pattern you will use most.

```
public interface IShippingStrategy { decimal Cost(Order o); }
public class StandardShipping : IShippingStrategy { public decimal Cost(Order o) => 5.99m; }
public class ExpressShipping  : IShippingStrategy { public decimal Cost(Order o) => 14.99m +
    o.Weight * 0.5m; }
public class FreeShipping     : IShippingStrategy { public decimal Cost(Order o) => 0m; }

public class Checkout(IShippingStrategy shipping)
{
    public decimal Total(Order o) => o.Subtotal + shipping.Cost(o);
}

// Or, lightweight: strategy as a delegate
public decimal Total(Order o, Func<Order, decimal> shippingCost) => o.Subtotal +
    shippingCost(o);
Total(order, o => o.Weight > 10 ? 20m : 5m);
```

Seen in .NET: `IComparer<T>` passed to `Sort`, `IEqualityComparer<T>`, `Func<>` arguments in LINQ, authentication handlers/schemes.

Observer

Problem: when one object changes, others need to know without tight coupling. **Idea:** subscribers register with a subject, which notifies them. **Seen in .NET:** `event`s, `IObservable<T>`/Rx, `INotifyPropertyChanged`, SignalR hubs, domain events.

```
public class StockTicker
{
    public event Action<string, decimal>? PriceChanged;
    public void Update(string symbol, decimal price) => PriceChanged?.Invoke(symbol, price);
}

var ticker = new StockTicker();
ticker.PriceChanged += (s, p) => Console.WriteLine($"{s} = {p}");
ticker.PriceChanged += (s, p) => { if (p > 100) Alert(s); };
ticker.Update("MSFT", 420m);
// Remember: unsubscribe (-=) when a short-lived subscriber outlives its use, or you leak
```

```
    memory.
```

Command

Problem: you want to treat a request as an *object*: queue it, log it, undo it, retry it. **Idea:** encapsulate an action and its data. **Seen in .NET:** `ICommand` in WPF/MAUI, MediatR requests, CQRS commands, background job payloads.

```
public interface ICommand { void Execute(); void Undo(); }

public class AddTextCommand(StringBuilder doc, string text) : ICommand
{
    public void Execute() => doc.Append(text);
    public void Undo() => doc.Length -= text.Length;
}

var history = new Stack<ICommand>();
void Run(ICommand c) { c.Execute(); history.Push(c); }
void Undo() { if (history.TryPop(out var c)) c.Undo(); }
```

Mediator

Problem: many objects talk to each other directly, creating a web of dependencies. **Idea:** a central mediator receives requests and routes them to handlers, so components only know the mediator. **Seen in .NET:** the MediatR library (check licensing for current versions), or a hand-rolled dispatcher.

```
public record GetOrderQuery(int Id) : IRequest<OrderDto?>;

public class GetOrderHandler(AppDb db) : IRequestHandler<GetOrderQuery, OrderDto?>
{
    public Task<OrderDto?> Handle(GetOrderQuery q, CancellationToken ct) =>
        db.Orders.Where(o => o.Id == q.Id).Select(o => new OrderDto(o.Id,
            o.Total)).FirstOrDefaultAsync(ct);
}

// Controller only knows IMediator
public async Task<IActionResult> Get(int id) => Ok(await mediator.Send(new GetOrderQuery(id)));
```

WATCH OUT

Mediator can hide control flow (you cannot just 'go to definition' to see who handles it). It is great for cross-cutting pipeline behaviors (validation, logging, transactions), but unnecessary for tiny apps.

Template Method

Problem: several workflows share the same skeleton but differ in a few steps. **Idea:** the base class defines the algorithm's skeleton; subclasses override specific steps.

```
public abstract class ReportGenerator
{
    public string Generate()                    // the template: fixed order
    {
        var data = LoadData();
        var body = Format(data);
        return Wrap(body);
    }
    protected abstract IEnumerable<string> LoadData();
    protected abstract string Format(IEnumerable<string> data);
    protected virtual string Wrap(string body) => $"<report>{body}</report>";   // optional hook
}
```

Seen in .NET: `BackgroundService.ExecuteAsync`, `ControllerBase` lifecycle, `Stream` subclasses.

State

Problem: an object's behavior depends on its state and the code is full of `if (status == ...)`. **Idea:** model each state as an object (or enforce transitions in one place) so illegal transitions are impossible.

```
public enum OrderStatus { Draft, Submitted, Paid, Shipped, Cancelled }

public class Order
{
    public OrderStatus Status { get; private set; }

    private static readonly Dictionary<OrderStatus, OrderStatus[]> Allowed = new()
    {
        [OrderStatus.Draft]     = [OrderStatus.Submitted, OrderStatus.Cancelled],
        [OrderStatus.Submitted] = [OrderStatus.Paid, OrderStatus.Cancelled],
        [OrderStatus.Paid]      = [OrderStatus.Shipped],
    };

    public void MoveTo(OrderStatus next)
    {
        if (!Allowed.TryGetValue(Status, out var ok) || !ok.Contains(next))
            throw new InvalidOperationException($"Cannot go from {Status} to {next}");
        Status = next;
    }
}
```

Chain of Responsibility

Problem: a request should pass through several handlers, any of which may handle it or pass it on. **Seen in .NET:** the middleware pipeline *is* this pattern; also validation chains and approval workflows.

```
public abstract class ApprovalHandler
{
    private ApprovalHandler? _next;
    public ApprovalHandler SetNext(ApprovalHandler n) { _next = n; return n; }
    public virtual string Handle(decimal amount) => _next?.Handle(amount) ?? "Rejected";
}
public class Manager : ApprovalHandler { public override string Handle(decimal a) => a <= 1_000
    ? "Manager approved" : base.Handle(a); }
public class Director : ApprovalHandler { public override string Handle(decimal a) => a <=
    10_000 ? "Director approved" : base.Handle(a); }

var chain = new Manager(); chain.SetNext(new Director());
Console.WriteLine(chain.Handle(5_000));   // Director approved
```

Iterator

Problem: traverse a collection without exposing how it is stored. **In C#:** this is built in: `IEnumerable<T>`, `foreach`, and `yield return` (see Chapter 4).

APPLICATION-LEVEL PATTERNS

Repository + Unit of Work

Repository hides data access behind a collection-like interface. **Unit of Work** tracks changes across repositories and commits them together. EF Core's `DbContext` already implements both, so wrap it only when you want domain-shaped query methods.

```
public interface IOrderRepository
{
    Task<Order?> GetAsync(Guid id, CancellationToken ct);
    Task<IReadOnlyList<Order>> OpenForCustomerAsync(Guid customerId, CancellationToken ct);
    void Add(Order order);
}
// AppDb implements it
public interface IUnitOfWork { Task<int> SaveChangesAsync(CancellationToken ct); }
```

Specification

Problem: reusable, combinable query criteria. **Idea:** package a filter as an object/expression.

```
public class OverdueOrders
{
    public Expression<Func<Order, bool>> Criteria =>
        o => o.Status == OrderStatus.Submitted && o.DueDate < DateTime.UtcNow;
}
var overdue = await db.Orders.Where(new OverdueOrders().Criteria).ToListAsync();
```

Outbox pattern

Problem: you save to the database *and* publish a message (event). If the app crashes between the two, you get inconsistency (data saved, message lost, or vice versa). **Idea:** in the *same transaction* that saves the data, also insert the message into an `Outbox` table. A background worker reads the table and publishes reliably, marking messages done. Delivery is **at-least-once**, so consumers must be idempotent.

```
await using var tx = await db.Database.BeginTransactionAsync();
db.Orders.Add(order);
db.OutboxMessages.Add(new OutboxMessage { Type = "OrderPlaced", Payload =
    JsonSerializer.Serialize(evt) });
await db.SaveChangesAsync();
await tx.CommitAsync();
// A BackgroundService polls OutboxMessages where ProcessedAt IS NULL, publishes to the bus,
    then marks processed.
```

Retry, Circuit Breaker, Timeout (resilience)

Networks fail. **Retry** transient failures with exponential backoff and jitter. **Circuit breaker** stops calling a failing dependency for a while so you do not pile on. **Timeout** prevents hanging forever. In .NET use `Microsoft.Extensions.Http.Resilience` (Polly under the hood).

```
builder.Services.AddHttpClient<PaymentsClient>()
    // retry + circuit breaker + timeouts with sensible defaults
    .AddStandardResilienceHandler();

// Manual retry idea:
// delay = baseDelay * 2^attempt + random jitter   (1s, 2s, 4s ... avoids thundering herds)
```

Saga (distributed transactions)

When a business process spans services, a single database transaction is impossible. A **saga** is a sequence of local transactions with **compensating actions** to undo earlier steps on failure (reserve stock -> charge card -> ship; if charging fails, release stock). Two styles: **choreography** (services react to each other's events) and **orchestration** (a coordinator tells each service what to do).

Pattern cheat sheet

Symptom you notice	Pattern to consider
Growing `switch` on a type/kind choosing behavior	Strategy (or polymorphism)
Need to add logging/caching/retry without touching the class	Decorator
Third-party API shape doesn't match yours	Adapter
Complex subsystem, callers want one call	Facade
Constructor has many optional parameters	Builder (or object initializer + `required`)
Creation logic depends on runtime input	Factory
Many parts notify each other directly	Mediator or Observer/events
Need undo, queue, or audit of operations	Command
Object behavior changes with status	State (explicit transition table)
Same algorithm skeleton, different steps	Template Method (or Strategy)
Save data and publish event atomically	Outbox
Calls to an unreliable service	Retry + Circuit Breaker + Timeout
Process spanning several services	Saga
Primitive strings/ints with rules	Value Object

Patterns .NET already gives you

Pattern	Where in .NET
Builder	`WebApplication.CreateBuilder`, `StringBuilder`
Factory	`IHttpClientFactory`, `IDbContextFactory<T>`
Singleton / lifetime mgmt	DI container lifetimes
Decorator	Streams, middleware, `DelegatingHandler`
Chain of Responsibility	Middleware pipeline, filters
Observer	`event`, `IObservable<T>`, SignalR
Strategy	`IComparer<T>`, authentication schemes, `Func<>`
Iterator	`IEnumerable<T>`, `yield return`
Repository + Unit of Work	`DbSet<T>` + `DbContext`

Pattern	Where in .NET
Template Method	`BackgroundService`, `ControllerBase`
Prototype	`record` + `with`
Options	`IOptions<T>` configuration binding

Practice

1. Replace a `switch (paymentType)` that calculates fees with the Strategy pattern, registered via DI.
2. Add caching to an existing service using the Decorator pattern without editing the service.
3. Explain how ASP.NET Core middleware is an example of Chain of Responsibility.
4. Why is the Outbox pattern needed? What delivery guarantee does it give?
5. Which pattern would you use to wrap a vendor SDK and keep it out of your domain layer?

12. Testing and Quality

> **KEY IDEA**
> Tests are what let you change code without fear. You already use xUnit and Postman; this chapter organizes the concepts so you can explain *why* and *what to test*, not just how.

The test pyramid

```
        /\
       /UI\           few, slow, brittle: end-to-end (Playwright, Cypress)
      /----\
     / Integ \        some: API + real DB, message bus (WebApplicationFactory,
        Testcontainers)
    /--------\
   /   Unit   \       many, fast, isolated: domain logic, pure functions
  /------------\
```

Level	Tests	Speed	Confidence in...
Unit	One class/method, dependencies faked	Milliseconds	Logic correctness
Integration	Several components together, often real DB	Seconds	Wiring, queries, serialization
End-to-end	Whole system through the UI/API	Slow	Real user flows
Contract	API shape agreed between consumer and provider	Fast	Services won't break each other

xUnit basics

```
public class PriceCalculatorTests
{
    [Fact]                                          // a single test
    public void Total_AppliesTenPercentDiscount_WhenCustomerIsGold()
    {
        // Arrange
        var calc = new PriceCalculator();
        var order = new Order { Subtotal = 200m, CustomerType = "Gold" };

        // Act
        var total = calc.Total(order);

        // Assert
        Assert.Equal(180m, total);
    }

    [Theory]                                        // same test, many inputs
    [InlineData(0, 0)]
    [InlineData(100, 100)]
    [InlineData(250, 225)]
    public void Total_HandlesBoundaries(decimal subtotal, decimal expected)
    {
        var total = new PriceCalculator().Total(new Order { Subtotal = subtotal, CustomerType =
            subtotal > 200 ? "Gold" : "None" });
        Assert.Equal(expected, total);
    }
```

```
    [Fact]
    public void Withdraw_Throws_WhenInsufficientFunds()
    {
        var acct = new BankAccount("Ana", 10m);
        var ex = Assert.Throws<InvalidOperationException>(() => acct.Withdraw(50m));
        Assert.Contains("Insufficient", ex.Message);
    }

    [Fact]
    public async Task SendAsync_Retries_OnTransientFailure()
    {
        await Assert.ThrowsAsync<TimeoutException>(() => sut.RunAsync());
    }
}
```

Test naming and structure

- Name tests `Method_Scenario_ExpectedResult` so a failure message tells you what broke.
- **Arrange-Act-Assert (AAA)**: set up, do one thing, verify. One logical assertion per test.
- Tests must be **independent** (no shared mutable state, any order), **deterministic** (no real clock/random/network), and **fast**.
- xUnit creates a **new test class instance per test**; use the constructor for setup and `IDisposable` for teardown. Use `IClassFixture<T>` for expensive shared setup.

Test doubles: fakes, stubs, mocks

Double	What it does	Example
Dummy	Fills a parameter, never used	`null` or empty object
Stub	Returns canned answers	`repo.GetAsync` returns a fixed order
Fake	Working lightweight implementation	In-memory repository, EF Core InMemory/SQLite
Mock	Verifies interactions happened	Assert `email.SendAsync` was called once
Spy	Records calls for later inspection	`FakeEmailSender.Sent` list

```
// Moq
var repo = new Mock<IOrderRepository>();
repo.Setup(r => r.GetAsync(It.IsAny<Guid>(), It.IsAny<CancellationToken>()))
    .ReturnsAsync(new Order());

var email = new Mock<IEmailSender>();
var sut = new OrderService(repo.Object, email.Object);

await sut.ShipAsync(orderId);

email.Verify(e => e.SendAsync("ana@x.com", It.IsAny<string>(), It.IsAny<string>()), Times.Once);

// NSubstitute alternative
var email2 = Substitute.For<IEmailSender>();
await email2.Received(1).SendAsync("ana@x.com", Arg.Any<string>(), Arg.Any<string>());
```

> **TIP**
> Prefer **state-based** assertions (check the resulting data) and fakes over heavy interaction mocking. Over-mocked tests break whenever you refactor, even when behavior is unchanged.

Testing EF Core code

Option	Pros	Cons
EF Core InMemory provider	Zero setup, fast	Not a relational DB: no constraints, transactions, or real SQL translation. Can hide bugs
SQLite in-memory	Real relational behavior, fast	Differs from SQL Server/PostgreSQL in some features
Testcontainers (real DB in Docker)	Highest fidelity	Slower; needs Docker in CI

```
// SQLite in-memory: keep the connection open for the life of the test
var conn = new SqliteConnection("DataSource=:memory:");
conn.Open();
var options = new DbContextOptionsBuilder<AppDb>().UseSqlite(conn).Options;
using var db = new AppDb(options);
db.Database.EnsureCreated();
```

Integration tests for APIs

```
public class OrdersApiTests(WebApplicationFactory<Program> factory) :
    IClassFixture<WebApplicationFactory<Program>>
{
    [Fact]
    public async Task GetOrder_ReturnsNotFound_ForUnknownId()
    {
        var client = factory.WithWebHostBuilder(b => b.ConfigureServices(s =>
        {
            // replace real services with fakes
            s.RemoveAll<IEmailSender>();
            s.AddSingleton<IEmailSender, FakeEmailSender>();
        })).CreateClient();

        var response = await client.GetAsync("/api/orders/999999");

        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
    }
}
// Make Program visible to tests:  public partial class Program { }   at the bottom of
    Program.cs
```

What to test (and not to)

Test	Skip
Business rules and edge cases (zero, negative, boundaries, null, empty)	Trivial getters/setters
Branching logic and state transitions	Framework code (EF Core, ASP.NET already tested)

Test	Skip
Authorization rules (who can/can't)	Private methods directly (test through public behavior)
Error handling paths	Implementation details that change constantly
Bug regressions (write the failing test first)	Third-party library internals

TDD in three steps

1. **Red**: write a small failing test describing the behavior you want.
2. **Green**: write the simplest code that makes it pass.
3. **Refactor**: clean up while the tests protect you.

Code coverage: a tool, not a goal

Coverage tells you which lines *ran*, not whether assertions are meaningful. 100% coverage with no assertions is worthless. Use it to find *untested areas*, and favor behavior coverage of risky code over a blanket percentage.

```
dotnet test --collect:"XPlat Code Coverage"
```

Postman and API testing

- Organize requests into **collections** with **environments** (dev/staging/prod variables like `{{baseUrl}}`).
- Add **test scripts** (JavaScript) to assert status codes, response shape, and values; chain requests by saving tokens/IDs into variables.
- Run collections in CI with **Newman** (`newman run collection.json -e env.json`).
- Use Postman for exploratory and contract checks; keep core business logic covered by xUnit.

```
// Postman test script
pm.test("Status is 201", () => pm.response.to.have.status(201));
const body = pm.response.json();
pm.test("Has id", () => pm.expect(body.id).to.be.a("number"));
pm.environment.set("orderId", body.id);
```

Other quality tools

Tool	Purpose
Roslyn analyzers / `.editorconfig`	Enforce style and catch bugs at compile time; treat warnings as errors in CI
SonarQube / SonarCloud	Static analysis: code smells, bugs, vulnerabilities, coverage trends
Dependabot / `dotnet list package --vulnerable`	Find outdated or vulnerable packages
StyleCop / `dotnet format`	Consistent formatting
Mutation testing (Stryker.NET)	Checks whether your tests actually catch injected bugs

Tool	Purpose
Code review	Highest-leverage quality practice: catches design issues tools can't

Code review checklist

- Does it do what the ticket says? Are edge cases handled?
- Are names clear? Is the code simple enough to understand in one read?
- Tests added or updated for behavior changes?
- Security: input validated, authorization checked, no secrets in code, no SQL string concatenation?
- Performance: any N+1, unbounded queries, blocking calls in async code?
- Error handling and logging appropriate (no swallowed exceptions, no sensitive data logged)?
- Backward compatibility: public API changes, migrations safe to deploy?

Practice

1. What is the difference between a mock, a stub, and a fake?
2. Why can the EF Core InMemory provider give false confidence?
3. Write a `[Theory]` for a function that validates passwords.
4. How do you test code that depends on `DateTime.UtcNow`?
5. What would you test in a method that transfers money between two accounts?

13. System Design

KEY IDEA
System design is about making trade-offs out loud. There is no single right answer; there are answers that fit the requirements and answers that do not. This chapter gives you the building blocks, then a repeatable process, then worked examples.

The vocabulary

Term	Plain meaning
Latency	How long one request takes (ms)
Throughput	How many requests per second the system handles
Availability	Fraction of time the system is up (99.9% = ~8.8 hours down per year)
Scalability	Ability to handle more load by adding resources
Reliability	Does the right thing, even when parts fail
Consistency	Do all readers see the same, latest data?
Durability	Committed data is not lost
SLA / SLO / SLI	Contract / internal target / the measured metric (e.g. p99 latency < 300 ms)
p50 / p95 / p99	Percentile latencies. p99 = slowest 1% of requests. Averages hide pain; watch tails

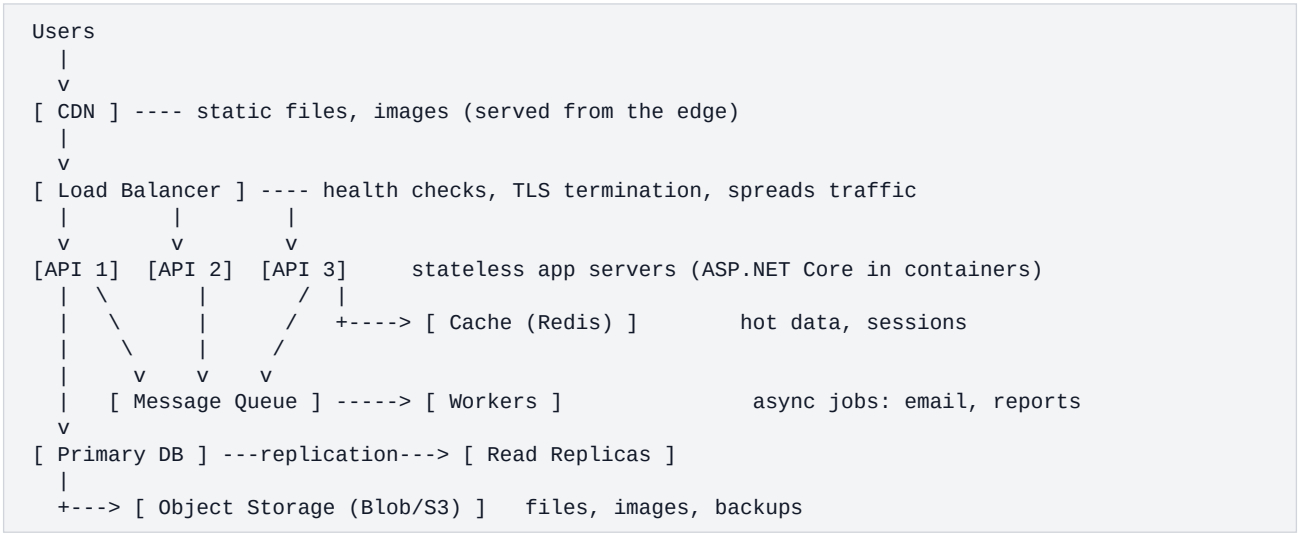
Availability	Downtime per year	Per month
99%	3.65 days	7.3 hours
99.9% (three nines)	8.76 hours	43.8 minutes
99.99% (four nines)	52.6 minutes	4.4 minutes
99.999% (five nines)	5.3 minutes	26 seconds

Scaling: vertical vs horizontal

- **Vertical (scale up)**: bigger machine (more CPU/RAM). Simple, but has a ceiling and is a single point of failure.
- **Horizontal (scale out)**: more machines behind a load balancer. Nearly unlimited, but your app must be **stateless** (no in-memory sessions tied to one server) and data must be shared or partitioned.

KEY IDEA
Stateless services are the key to horizontal scaling. Keep session/user state in a shared store (database, Redis, signed token) so any instance can serve any request.

A typical web architecture



Load balancing

Algorithm	How it picks	Good for
Round robin	Next server in turn	Similar servers, similar requests
Least connections	Server with fewest active connections	Uneven request duration
IP hash / sticky sessions	Same client -> same server	Legacy stateful apps (avoid if possible)
Weighted	Bigger servers get more traffic	Mixed hardware
Consistent hashing	Hash ring; minimal reshuffle when nodes change	Caches, sharded data

Layer 4 load balancers route by IP/port (fast, dumb). **Layer 7** balancers understand HTTP (route by path/header, terminate TLS, do auth/WAF). Examples: Azure Application Gateway / Front Door, AWS ALB, NGINX, Envoy.

Caching

Caching trades freshness for speed. Put it where it removes the most work: browser -> CDN -> API gateway -> app memory -> distributed cache (Redis) -> database.

Strategy	How it works	Trade-off
Cache-aside (lazy)	App checks cache; on miss loads from DB and fills cache	Most common; first request slow; stale until expiry/invalidation
Write-through	Write goes to cache and DB together	Always fresh cache; slower writes
Write-behind (back)	Write to cache, flush to DB later	Fast writes; risk of data loss on crash
Read-through	Cache itself loads from DB on miss	Simpler app code; cache layer needs loader
TTL expiry	Entries expire after a time	Simple freshness bound
Explicit invalidation	Delete/refresh key on change	Accurate; easy to get wrong

Eviction policies: LRU (least recently used), LFU (least frequently used), FIFO, TTL. **Cache stampede:** many requests miss the same key simultaneously and all hit the database; mitigate with locking/single-flight, jittered TTLs, or serving stale while refreshing.

Databases at scale

Replication

Copy data to other servers. **Primary-replica**: writes go to the primary, reads can go to replicas. Boosts read throughput and availability. Replicas lag slightly (**replication lag**), so a user may not immediately see their own write ('read-your-writes' problem). Mitigate by reading from the primary right after a write.

Sharding (partitioning)

Split data across databases so each holds a subset. Choose a **shard key** that spreads load evenly (user id hashed) and keeps related data together. Pitfalls: hot shards, cross-shard joins/transactions, painful re-sharding. Delay sharding as long as possible; first try indexes, caching, read replicas, and bigger hardware.

```
// simple, but changing N reshuffles almost everything
shard = hash(userId) % numberOfShards
consistent hashing / lookup table             // minimizes data movement when adding shards
```

CAP theorem and consistency models

In a distributed system, when a **network partition** happens, you must choose between **Consistency** (every read sees the latest write; some requests may be refused) and **Availability** (every request gets an answer; it might be stale). Partitions are a fact of life, so the real choice is C vs A *during failures*.

Model	Meaning	Example
Strong consistency	Reads always see the latest committed write	Single-primary SQL, banking balances
Eventual consistency	Replicas converge if updates stop	DNS, shopping-cart replicas, social feeds
Read-your-writes	You always see your own updates	Profile edit then reload
Causal	Related events are seen in order	Comment appears after the post it replies to

KEY IDEA
ACID (transactions, SQL) is about correctness within a database. BASE (Basically Available, Soft state, Eventually consistent) describes many NoSQL systems that favor availability. Choose per feature: payments need strong consistency; 'likes' counters rarely do.

Messaging and queues

A **queue** decouples producers from consumers. The producer drops a message and moves on; workers process at their own pace. Benefits: smooth traffic spikes (buffering), retries, independent scaling, resilience when a downstream service is down.

Concept	Meaning
Queue (point-to-point)	Each message handled by ONE consumer (work distribution). Azure Storage Queues, SQS
Topic / pub-sub	Each message delivered to MANY subscribers. Azure Service Bus topics, SNS, Kafka topics
Event streaming (Kafka, Event Hubs)	Durable ordered log; consumers read at their own offset; replayable
At-most-once	May lose messages, never duplicates
At-least-once	Never loses, may duplicate (most common) -> consumers must be **idempotent**
Exactly-once	Hard; usually approximated with at-least-once + idempotency/dedup
Dead-letter queue (DLQ)	Where poison messages go after repeated failure, for inspection
Back-pressure	Slowing producers when consumers can't keep up (bounded queues)

```
// Idempotent consumer: processing the same message twice is harmless
public async Task HandleAsync(PaymentCaptured msg)
{
    // already done
    if (await db.ProcessedMessages.AnyAsync(m => m.Id == msg.MessageId)) return;

    // ... apply the business change ...
    db.ProcessedMessages.Add(new ProcessedMessage { Id = msg.MessageId });
    await db.SaveChangesAsync();      // both in one transaction
}
```

Rate limiting

Algorithm	Idea	Note
Fixed window	N requests per calendar window	Simple; bursts at window edges
Sliding window	Count over the last T seconds	Smoother; more memory
Token bucket	Bucket refills at a steady rate; each request spends a token	Allows bursts up to bucket size; very common
Leaky bucket	Requests drain at a constant rate	Smooths traffic output
Concurrency limit	Max in-flight requests	Protects expensive endpoints

Rate limit **per user/API key/IP**, return `429` with `Retry-After`, and store counters in a shared store (Redis) when you have multiple instances.

API gateway, BFF, service discovery

- **API gateway**: single entry point for many services; handles auth, rate limiting, routing, TLS. (Azure API Management, YARP, Kong)
- **BFF (Backend for Frontend)**: a thin backend shaped for one client (mobile vs web) that aggregates calls.
- **Service discovery**: how services find each other as instances come and go (DNS, Kubernetes services, Consul).
- **Sidecar / service mesh**: infrastructure proxy handling retries, mTLS, metrics outside your code.

Observability: logs, metrics, traces

Pillar	Answers	Tools
Logs	What happened? (events with context)	Serilog, Seq, ELK, Azure Monitor
Metrics	How is it behaving? (numbers over time)	Prometheus, Grafana, Application Insights
Traces	Where did this request spend its time across services?	OpenTelemetry, Jaeger, Application Insights

Use a **correlation id** that flows through every service and appears in every log line so you can reconstruct one request. Alert on **symptoms users feel** (error rate, latency) rather than only causes (CPU).

Reliability patterns

- **Redundancy**: no single point of failure (multiple instances, zones, regions).
- **Health checks + auto-restart**: remove unhealthy instances from rotation.
- **Timeouts everywhere**: never wait forever on a dependency.
- **Retries with exponential backoff + jitter**, only for idempotent/transient failures.
- **Circuit breaker**: stop calling a failing dependency; fail fast.
- **Bulkheads**: isolate resources so one failing feature doesn't take everything down.
- **Graceful degradation**: serve cached/partial data when a dependency is down.
- **Backups and tested restores**; define RPO (how much data you can lose) and RTO (how long you can be down).
- **Idempotency keys** for safely retrying payments and other non-idempotent writes.

A repeatable system design process

1. **Clarify requirements.** Functional (what it does) and non-functional (scale, latency, availability, consistency, security). Ask who the users are and what matters most.
2. **Estimate scale.** Users, requests per second, data per item, read/write ratio, storage per year.
3. **Define the API and data model.** Core entities, endpoints, key queries.
4. **Sketch the high-level design.** Client, load balancer, services, database, cache, queue.
5. **Deep-dive the hard parts.** The bottleneck: schema, indexing, caching, partitioning, consistency.

6 **Discuss trade-offs and failures.** What if a node dies? A region? Traffic 10x?

7 **Evolve it.** What would you do first, and what would you add as it grows?

Back-of-the-envelope numbers

Quantity	Rule of thumb
Seconds per day	~86,400 (round to 100,000)
1M requests/day	~12 requests/second average (peak maybe 3-10x)
100M requests/day	~1,200 requests/second average
RAM access	~100 ns
SSD random read	~100 microseconds
Same-datacenter network round trip	~0.5 ms
Cross-continent round trip	~100-150 ms
1 KB x 1M items	~1 GB
Text row ~1 KB; image ~200 KB; video minute ~ tens of MB	Use to estimate storage

Worked example 1: URL shortener

Requirements: create a short link for a long URL; redirect quickly; links may expire; analytics optional. Assume 100M new URLs/month and a 100:1 read-to-write ratio.

Estimates: 100M/month is ~40 writes/second, ~4,000 reads/second. Each record ~500 bytes -> ~50 GB/month, ~3 TB over five years. Fits a sharded/partitioned store easily; reads dominate, so cache heavily.

```
POST /api/links        { "url": "https://long.example.com/...", "customAlias": "opt" } -> {
    "short": "https://sho.rt/aZ3k9Q" }
GET  /{code}           -> 301/302 redirect to the long URL     (302 if you want click analytics
    every time)

Table Links(Code PK, LongUrl, CreatedAt, ExpiresAt, OwnerId)
```

Generating the code: use base62 (`0-9a-zA-Z`). 7 characters = 62^7 = ~3.5 trillion combinations. Options: (1) **auto-increment id -> base62 encode** (simple, predictable, needs a coordinated id generator like a range allocator per server); (2) **random 7 chars + uniqueness check** (unguessable; retry on collision); (3) **hash of the URL** (same URL -> same code, collisions to handle).

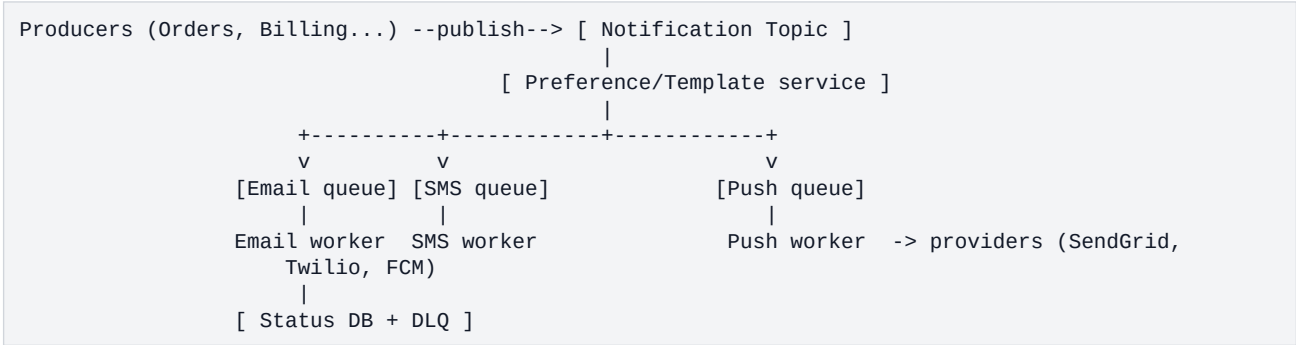
```
const string Alphabet = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
static string ToBase62(long n)
{
    if (n == 0) return "0";
    var sb = new StringBuilder();
    while (n > 0) { sb.Insert(0, Alphabet[(int)(n % 62)]); n /= 62; }
    return sb.ToString();
}
```

```
Client -> CDN/LB -> API -> Redis cache (code -> url)  --miss--> DB (partitioned by code)
                        \-> Queue -> Analytics worker -> click counts (async, off the hot path)
```

Key decisions: cache hot codes in Redis (LRU); redirects are cheap and read-heavy; write path validates URL (block malicious targets) and rate limits creators; expiry via TTL column + cleanup job; analytics handled asynchronously so redirects stay fast.

Worked example 2: notification service

Requirements: send email, SMS, and push notifications triggered by other services; respect user preferences; do not send duplicates; handle provider outages.



- **Decouple with a queue** so a slow email provider cannot slow the order service.
- **Idempotency**: each notification has a unique id; workers skip ones already sent.
- **Retries with backoff**; after N attempts move to a **dead-letter queue** and alert.
- **Provider failover**: if provider A is failing (circuit breaker open), route to provider B.
- **Rate limits per provider and per user** (do not spam).
- **Templates + localization** stored separately from code.
- **Audit log** of status (queued, sent, delivered, failed) for support.

Worked example 3: citation / ticket management app (your own project)

Design the municipal citation system you built. **Functional:** officers scan a driver's license barcode, record a violation with GPS location, issue a citation; admins review and report. **Non-functional:** works on flaky mobile connections, auditable, secure personal data.

- **Data model:** `Officer`, `Citation`, `Violation` (lookup), `Person` (from license), `Location`, `AuditLog`. Index by `IssuedAt`, `OfficerId`, `PersonId`, plus a spatial index for map queries.
- **Offline-tolerant client:** queue citations locally (IndexedDB/service worker), sync when connected; use a client-generated **idempotency key** so retries do not create duplicates.
- **Security:** role-based access (officer vs admin), encrypt data at rest and in transit, mask license numbers in logs, immutable audit trail, short-lived tokens, rate limits.
- **Scaling:** one PostgreSQL primary is plenty; add a read replica for reports; cache lookup tables; generate PDFs asynchronously via a queue.
- **Observability:** correlation ids, error alerts, dashboard for sync failures.

Worked example 4: course scheduling (constraint problem)

Your university scheduling platform assigns courses, professors, and classrooms. The design ideas are reusable:

- **Hard constraints** (must hold): a professor cannot teach two classes at once; a room cannot be double-booked; room capacity >= enrollment.
- **Soft constraints** (optimize): minimize gaps in professor schedules, avoid late classes, balance room usage. Model as a weighted score.
- **Algorithms:** greedy placement + backtracking, or constraint solvers (OR-Tools CP-SAT); start with a greedy heuristic that is easy to explain, then improve.
- **Validation as a separate component**: a `ConflictChecker` that takes any proposed schedule and returns violations; reuse it for manual edits and the generator.
- **Forecasting demand:** use historical enrollment per course/term (moving average or simple regression) to decide how many sections to open.
- **Make solving asynchronous**: submit a job, poll status, store result versions so planners can compare.

Other classic prompts to practice

Prompt	Core ideas to hit
Rate limiter service	Token bucket, Redis atomic ops (Lua/INCR), per-key limits, distributed sync, 429 + headers
Chat application	WebSockets/SignalR, message ordering, presence, offline delivery, fan-out, storage by conversation
News feed / timeline	Fan-out on write vs read, caching, ranking, celebrity problem
File storage (Dropbox-like)	Chunking, dedupe by hash, metadata DB vs blob store, sync, resumable uploads
Video streaming	Transcoding pipeline, CDN, adaptive bitrate, object storage
Search autocomplete	Trie, prefix indexes, caching top queries, ranking by popularity
Web crawler	Frontier queue, politeness, dedupe (URL hash/Bloom filter), distributed workers
Ride-hailing / maps	Geospatial index (geohash/quadtree), matching, location updates streaming
Payments system	Idempotency keys, ledger (double-entry), exactly-once effects, reconciliation, sagas
Distributed ID generator	Snowflake-style: timestamp + machine id + sequence; or ULIDs/GUIDv7

Monolith to cloud: Azure and AWS service map

Need	Azure	AWS
Run containers/web apps	App Service, Container Apps, AKS	ECS/Fargate, App Runner, EKS, Elastic Beanstalk
Functions / serverless	Azure Functions	Lambda

Need	Azure	AWS
Relational DB	Azure SQL, PostgreSQL Flexible Server	RDS, Aurora
NoSQL	Cosmos DB, Table Storage	DynamoDB
Cache	Azure Cache for Redis	ElastiCache
Object storage	Blob Storage	S3
Queues / messaging	Service Bus, Storage Queues, Event Hubs	SQS, SNS, Kinesis, MSK
Identity	Microsoft Entra ID	IAM, Cognito
Secrets	Key Vault	Secrets Manager, KMS
CI/CD	Azure DevOps, GitHub Actions	CodePipeline, CodeBuild, GitHub Actions
Container registry	Azure Container Registry	ECR
Monitoring	Application Insights, Azure Monitor	CloudWatch, X-Ray
CDN / edge	Front Door, Azure CDN	CloudFront
API management	API Management	API Gateway

Practice

1. Walk through the process above for a TODO app with 10 million users. Where would the first bottleneck appear?
2. Explain CAP in two sentences with an example.
3. Why must consumers of an at-least-once queue be idempotent? Show how.
4. When would you choose sharding, and what would you try first?
5. Design a rate limiter shared by 5 API instances. Where does the counter live?
6. What happens, step by step, when a user types a URL and presses enter? (DNS, TCP, TLS, HTTP, LB, app, DB, response)

14. Security Fundamentals

KEY IDEA
Most breaches come from a short list of well-known mistakes. You already assessed an app against OWASP guidance; this chapter turns that into a mental checklist you can apply to any API you build.

Core ideas

- **CIA triad**: Confidentiality (only the right people read it), Integrity (it hasn't been tampered with), Availability (it works when needed).
- **Least privilege**: give every user, service, and token the minimum access needed.
- **Defense in depth**: multiple layers, so one failure is not fatal.
- **Never trust input**: validate on the server, always, even if the client validates.
- **Secure by default; fail closed**: when in doubt, deny.
- **Assume breach**: log, monitor, and limit the blast radius.

OWASP Top 10: the list to know

Category	What it means	Defense in .NET
Broken access control	Users act outside their permissions (IDOR: change `/orders/5` to `/orders/6`)	Authorize every request; check ownership server-side; deny by default
Cryptographic failures	Weak/no encryption, bad hashing, exposed secrets	TLS everywhere; strong password hashing; Key Vault; data protection API
Injection	Untrusted input interpreted as code (SQL, command, LDAP)	Parameterized queries/EF Core; never concatenate input into commands
Insecure design	Missing security thinking at design time	Threat modeling; abuse-case tests
Security misconfiguration	Default creds, verbose errors, open storage, debug on	Hardened configs; remove unused features; secure headers
Vulnerable/outdated components	Known-CVE libraries	Dependabot, `dotnet list package --vulnerable`, regular updates
Identification & authentication failures	Weak passwords, session fixation, no MFA	ASP.NET Identity/Entra ID, MFA, lockout, secure cookies
Software & data integrity failures	Unsigned updates, insecure CI/CD, unsafe deserialization	Pin and verify dependencies, protect pipelines, avoid unsafe deserializers
Logging & monitoring failures	Attacks go unnoticed	Log auth events and failures, alert on anomalies
SSRF	Server fetches an attacker-chosen URL (reaching internal services)	Allow-list outbound hosts; block internal IP ranges

KEY IDEA
Check owasp.org for the current edition of the Top 10; the list is revised every few years, but these themes persist.

SQL injection

```
// VULNERABLE: attacker sends name = "'; DROP TABLE Users; --"
var sql = "SELECT * FROM Users WHERE Name = '" + name + "'";

// SAFE: parameters are data, never code
// LINQ -> parameterized
var users = await db.Users.Where(u => u.Name == name).ToListAsync();
var raw = await db.Users.FromSqlInterpolated($"SELECT * FROM Users WHERE Name =
    {name}").ToListAsync();  // parameterized

// FromSqlRaw with string interpolation using $"" is NOT safe. Use FromSqlInterpolated or
    explicit parameters.
using var cmd = new SqlCommand("SELECT * FROM Users WHERE Name = @name", conn);
cmd.Parameters.AddWithValue("@name", name);
```

Cross-site scripting (XSS)

Attacker-supplied script runs in another user's browser, stealing sessions or acting as them. **Defense:** output-encode by context (Razor and React encode by default), avoid `dangerouslySetInnerHTML` / `Html.Raw` with untrusted data, set a **Content-Security-Policy** header, and mark cookies `HttpOnly`.

```
// React: safe by default
<p>{userComment}</p>                              // escaped
// DANGEROUS unless html is sanitized (DOMPurify)
<div dangerouslySetInnerHTML={{__html: html}} />
```

CSRF (cross-site request forgery)

A malicious site makes the victim's browser send a request to your site, and the browser automatically attaches the victim's **cookies**. **Defense:** antiforgery tokens for cookie-authenticated state-changing requests, `SameSite=Lax/Strict` cookies, and requiring custom headers. APIs that use `Authorization: Bearer` headers (not cookies) are not CSRF-prone in the same way, but then you must protect tokens from XSS.

```
builder.Services.AddAntiforgery();
app.UseAntiforgery();

builder.Services.ConfigureApplicationCookie(o =>
{
    o.Cookie.HttpOnly = true;                     // JS cannot read it
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    o.Cookie.SameSite = SameSiteMode.Lax;
});
```

Authentication in depth

Passwords

- **Never store plain text or fast hashes** (MD5, SHA-1, plain SHA-256). Use a slow, salted, adaptive algorithm: **Argon2id, bcrypt, scrypt, or PBKDF2**. ASP.NET Identity's `PasswordHasher` does this for you.
- A **salt** is a unique random value per password so identical passwords hash differently and rainbow tables fail.
- Add **lockout** after repeated failures, **MFA**, breach-password checks, and rate limiting on login.
- Return the same message for unknown user and wrong password to avoid account enumeration.

Sessions vs tokens

	Cookie session	JWT bearer token
State	Server stores session; cookie holds id (or encrypted ticket)	Stateless: token carries claims, signed by issuer
Revocation	Easy: delete server session	Hard until expiry; use short lifetimes + refresh tokens
Best for	Server-rendered apps, same-site SPAs	APIs, mobile apps, cross-service calls
Main risk	CSRF	Token theft via XSS if stored in JS-accessible storage

JWT anatomy

```
header.payload.signature    (each part Base64Url-encoded)

header:   { "alg": "RS256", "typ": "JWT" }
payload:  { "sub": "user-123", "iss": "https://login...", "aud": "api://my-api",
            "exp": 1760650000, "roles": ["Admin"] }
signature: sign(header + "." + payload, privateKey)

The payload is ENCODED, NOT ENCRYPTED. Anyone can read it. Never put secrets in a JWT.
The API validates: signature, issuer (iss), audience (aud), expiry (exp), not-before (nbf).
```

OAuth 2.0 and OpenID Connect (OIDC)

- **OAuth 2.0** is about **authorization / delegated access**: letting an app act on your behalf without sharing your password. It issues **access tokens**.
- **OpenID Connect** is a thin layer on top that adds **authentication**: it issues an **ID token** telling the app *who the user is*.
- **Roles:** Resource Owner (user), Client (app), Authorization Server (Entra ID/Auth0), Resource Server (your API).

Flow	Use for
Authorization Code + PKCE	Browser SPAs, mobile, and web apps with users. The modern default
Client Credentials	Service-to-service with no user (daemon, background job)

Flow	Use for
Device Code	Devices with limited input (CLI tools, TVs)
On-Behalf-Of	API calls a downstream API as the signed-in user
Implicit / Password (ROPC)	Deprecated. Avoid

```
User -> App: "Sign in"
App -> Entra ID: redirect with client_id, scope, code_challenge (PKCE)
User signs in at Entra ID (+ MFA)
Entra ID -> App: redirect back with a one-time ?code=...
App -> Entra ID (back channel): exchange code + code_verifier for tokens
Entra ID -> App: id_token + access_token (+ refresh_token)
App -> Your API: Authorization: Bearer <access_token>
Your API validates the token's signature, issuer, audience, expiry, and scopes/roles
```

TIP

Do not build your own login system if you can use an identity provider (Entra ID, Auth0, Cognito, ASP.NET Identity). Authentication is easy to get subtly wrong.

Authorization patterns

Model	Idea	Example
RBAC (role-based)	Permissions attached to roles	`Admin`, `Officer`, `Viewer`
Claims-based	Decide based on claims in the token	`department == claims`
Policy-based	Named rules combining claims/requirements	`CanApprove`: manager AND under limit
Resource-based	Decision depends on the specific resource	User can edit **their own** documents only
ABAC	Attributes of user, resource, environment	Allow if user.region == record.region and time in business hours

```
// Resource-based check: ownership is a server-side rule, not something the client claims
var order = await db.Orders.FindAsync(id);
if (order is null) return NotFound();
var result = await authz.AuthorizeAsync(User, order, "CanViewOrder");
if (!result.Succeeded) return Forbid();

public class OrderOwnerHandler : AuthorizationHandler<OperationAuthorizationRequirement, Order>
{
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext ctx,
        OperationAuthorizationRequirement req, Order order)
    {
        if (order.OwnerId == ctx.User.FindFirstValue(ClaimTypes.NameIdentifier) ||
            ctx.User.IsInRole("Admin"))
            ctx.Succeed(req);
        return Task.CompletedTask;
    }
}
```

WATCH OUT

IDOR / BOLA (Broken Object Level Authorization) is the number-one API vulnerability. Checking *that a user is logged in* is not enough: always check that this user may access *this specific record*.

Secrets management

- Never commit secrets (connection strings, API keys, client secrets) to git. Scanners find leaked keys within minutes.
- Local dev: `dotnet user-secrets` or environment variables. Production: **Azure Key Vault / AWS Secrets Manager**, accessed with a **managed identity** so no credential is needed at all.
- Rotate secrets; use separate secrets per environment; grant read-only access to the specific secrets needed.
- If a secret leaks: **revoke and rotate immediately**, then clean history. Deleting the commit is not enough.

```
// Managed identity + Key Vault: no password in config
builder.Configuration.AddAzureKeyVault(new Uri("https://myvault.vault.azure.net/"), new
    DefaultAzureCredential());
```

Transport and headers

- **HTTPS/TLS everywhere**, plus HSTS (`app.UseHsts()`) so browsers refuse plain HTTP.
- **Security headers**: `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `X-Frame-Options`/`frame-ancestors` (clickjacking), `Referrer-Policy`, `Permissions-Policy`.
- **CORS** allows specific origins only. It is a browser rule, not a server firewall.
- **Rate limiting** and request size limits to blunt brute force and DoS.

Encryption vocabulary

Term	Meaning	Example
Hashing	One-way fingerprint; cannot be reversed	Passwords (with slow hash), file integrity (SHA-256)
Symmetric encryption	Same key encrypts and decrypts	AES-GCM for data at rest
Asymmetric encryption	Public key encrypts / verifies; private key decrypts / signs	RSA, ECDSA; TLS handshake; JWT RS256
Digital signature	Proves who produced data and that it was not altered	JWTs, signed packages
HMAC	Keyed hash for message authentication	Webhook signature verification
Encryption at rest / in transit	Protect stored data / data moving over networks	Disk/DB encryption; TLS

TIP

Never invent your own cryptography. Use vetted APIs: `System.Security.Cryptography`, ASP.NET Core Data Protection, libsodium. Use `CryptographicOperations.FixedTimeEquals` to compare secrets (timing attacks).

Securing your CI/CD and containers

- Use **OIDC federation** from GitHub Actions to Azure/AWS instead of long-lived cloud credentials stored as secrets.
- Pin action versions (by SHA for sensitive repos); restrict workflow permissions (`permissions: contents: read`).
- Scan images and dependencies (Trivy, Dependabot, CodeQL).
- Run containers as **non-root**, use minimal base images, don't bake secrets into images.
- Protect main branch: required reviews and status checks.

Threat modeling in 5 minutes (STRIDE)

Letter	Threat	Question to ask	Mitigation
S	Spoofing	Can someone pretend to be someone else?	Strong authentication
T	Tampering	Can data be modified?	Integrity checks, signatures, authorization
R	Repudiation	Can someone deny doing it?	Audit logs
I	Information disclosure	Can data leak?	Encryption, access control, error hygiene
D	Denial of service	Can it be overwhelmed?	Rate limits, quotas, autoscaling
E	Elevation of privilege	Can a user become admin?	Least privilege, authorization checks

Security checklist for a new API

1. Authentication on all endpoints by default; explicit `[AllowAnonymous]` for the few public ones.
2. Authorization checks on every resource access, including ownership.
3. Input validation and size limits; DTOs instead of entities (no over-posting).
4. Parameterized queries only.
5. HTTPS + HSTS; secure cookies; strict CORS.
6. Rate limiting and lockout on login-like endpoints.
7. Secrets in a vault; none in source control or logs.
8. Generic error responses to clients; detailed errors only in logs.
9. Audit logging for sensitive actions.
10. Dependencies scanned and updated; CI pipeline hardened.

15. DevOps: Git, Docker, CI/CD, Cloud

KEY IDEA
DevOps is the practice of getting code from your laptop to production safely and repeatedly. You already use GitHub Actions, Azure, Docker and AWS; here is the connective tissue so you can explain every step.

Git essentials

Git stores snapshots of your project as **commits** in a graph. A **branch** is just a movable label pointing at a commit. **HEAD** points to where you are now.

```
git status                          # what changed?
git add -p                          # stage changes interactively (review hunks)
git commit -m "Add order validation"
git switch -c feature/orders        # create + switch to a branch
git push -u origin feature/orders   # publish branch
git pull --rebase                   # update your branch on top of remote changes
git log --oneline --graph --decorate
git diff / git diff --staged
git stash / git stash pop           # shelve work in progress
git restore file.cs                 # discard working-tree changes to a file
git revert <sha>                    # new commit that undoes a commit (safe for shared history)
git reset --soft HEAD~1             # undo last commit, keep changes staged
git cherry-pick <sha>               # copy one commit onto current branch
git bisect start/good/bad           # binary-search for the commit that introduced a bug
git reflog                          # recover "lost" commits
```

Merge vs rebase

	Merge	Rebase
What it does	Joins histories with a merge commit	Replays your commits on top of another branch
History	Preserves true history (can be noisy)	Linear, clean history
Safe on shared branches?	Yes	**No**: never rebase commits others have pulled
Typical use	Merging PRs into main	Updating your feature branch with latest main

Branching strategies

- **Trunk-based development**: short-lived branches (hours to a day) merged into `main` often, with feature flags for unfinished work. Best fit for CI/CD.
- **GitHub Flow**: branch from main, open a PR, review, merge, deploy. Simple and common.
- **Git Flow**: long-lived `develop`, `release`, `hotfix` branches. Heavier; suits scheduled releases.

Good commits and PRs

- Small, focused commits with messages that explain *why*. Conventional Commits (`feat:`, `fix:`, `chore:`) help automate changelogs.
- Small PRs (under ~400 lines) get better reviews. Describe context, testing done, and risks.
- `.gitignore` build outputs (`bin/`, `obj/`), secrets, `.env` files, IDE folders.

Docker fundamentals

- An **image** is a read-only template (layers). A **container** is a running instance of an image.
- A **Dockerfile** describes how to build the image. Each instruction creates a cached layer, so order from least to most frequently changing.
- A **registry** stores images (Docker Hub, Azure Container Registry, AWS ECR).
- Containers share the host kernel (lighter than VMs) and are isolated via namespaces and cgroups.

A production-style multi-stage Dockerfile for ASP.NET Core

```
# ---- build stage: has the full SDK ----
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY ["src/MyApp.Api/MyApp.Api.csproj", "src/MyApp.Api/"]
COPY ["src/MyApp.Application/MyApp.Application.csproj", "src/MyApp.Application/"]
RUN dotnet restore "src/MyApp.Api/MyApp.Api.csproj"        # cached unless csproj files change
COPY . .
RUN dotnet publish "src/MyApp.Api/MyApp.Api.csproj" -c Release -o /app/publish --no-restore

# ---- runtime stage: small image, no SDK ----
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
USER app                                   # run as non-root (image provides an 'app' user)
ENTRYPOINT ["dotnet", "MyApp.Api.dll"]
```

```
docker build -t myapp:1.0 .
docker run -d -p 8080:8080 --name myapp -e ConnectionStrings__Default="..." myapp:1.0
docker ps / docker logs -f myapp / docker exec -it myapp sh
docker stop myapp && docker rm myapp
docker images / docker rmi <id>
docker system prune                        # clean unused data
```

docker compose for local development

```
# compose.yaml
services:
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      ConnectionStrings__Default: "Host=db;Database=app;Username=app;Password=dev"
    depends_on: [db]
  db:
    image: postgres:16
    environment: { POSTGRES_USER: app, POSTGRES_PASSWORD: dev, POSTGRES_DB: app }
    volumes: [dbdata:/var/lib/postgresql/data]      # persist data outside the container
volumes:
  dbdata:
```

TIP

Add a `.dockerignore` (bin, obj, .git, node_modules) to keep the build context small and builds fast. Containers are **ephemeral**: anything written inside is lost when it is removed, so persist state in volumes or external services.

CI/CD concepts

Term	Meaning
Continuous Integration (CI)	Every push is automatically built and tested so integration problems surface immediately
Continuous Delivery	Every passing build is *deployable*; release is a button press
Continuous Deployment	Every passing build is deployed to production automatically
Artifact	Build output (a Docker image, zip) stored and promoted unchanged through environments
Environment	dev / staging / prod, each with its own config and secrets
Pipeline	Automated stages: restore -> build -> test -> scan -> package -> deploy
Build once, deploy many	The same artifact goes to every environment; only configuration differs

GitHub Actions: a complete .NET pipeline

```
# .github/workflows/ci-cd.yml
name: CI/CD
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read
  id-token: write          # needed for OIDC login to Azure

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '9.0.x' }
      - run: dotnet restore
      - run: dotnet build --no-restore -c Release
      - run: dotnet test --no-build -c Release --collect:"XPlat Code Coverage"
      - run: dotnet publish src/MyApp.Api -c Release -o out
      - uses: actions/upload-artifact@v4
        with: { name: app, path: out }

  deploy:
    needs: build-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production          # can require manual approval
    steps:
      - uses: actions/download-artifact@v4
        with: { name: app, path: out }
      - uses: azure/login@v2         # OIDC: no stored password
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      - uses: azure/webapps-deploy@v3
        with:
```

```
        app-name: my-web-app
        package: out
```

- **Workflow** (YAML file) -> **jobs** (run on a **runner**, in parallel unless `needs`) -> **steps** (shell commands or reusable **actions**).
- **Secrets** are encrypted per repo/environment and referenced as `${{ secrets.NAME }}`. They are masked in logs.
- Cache dependencies (`actions/cache` or `setup-dotnet` cache) to speed builds.
- Use a **matrix** to test multiple OSes/versions: `strategy: matrix: dotnet: ['8.0.x','9.0.x']`.
- Protect `main` so PRs must pass the workflow before merging.

Docker image pipeline to a registry (ECR/ACR)

```
    - uses: docker/setup-buildx-action@v3
    - uses: docker/login-action@v3
      with: { registry: myregistry.azurecr.io, username: ${{ secrets.ACR_USER }}, password:
          ${{ secrets.ACR_PASS }} }
    - uses: docker/build-push-action@v6
      with:
        push: true
        tags: myregistry.azurecr.io/myapp:${{ github.sha }}     # immutable tag = commit SHA
```

Deployment strategies

Strategy	How	Pros	Cons
Recreate	Stop old, start new	Simplest	Downtime
Rolling	Replace instances gradually	No downtime, cheap	Two versions live briefly; slow rollback
Blue/green	Run new stack beside old; switch traffic	Instant rollback	Double infrastructure
Canary	Send small % of traffic to new version, watch metrics, expand	Limits blast radius	Needs good monitoring
Feature flags	Deploy code dark; enable per user/percent	Decouples deploy from release	Flag cleanup debt

Database migrations and zero-downtime: new code must work with the old schema *and* the old code must work with the new schema during a rollout. Use **expand/contract**: (1) add new column nullable, (2) deploy code that writes both, (3) backfill, (4) switch reads, (5) remove old column later.

Infrastructure as Code (IaC)

Define infrastructure (servers, databases, networks) in versioned files rather than clicking in a portal. Benefits: repeatable, reviewable, and recoverable.

Tool	Notes
Bicep / ARM	Azure-native declarative language
Terraform / OpenTofu	Multi-cloud, huge ecosystem; keeps state file

Tool	Notes
Pulumi	Real languages (C#, TypeScript) for infrastructure
AWS CloudFormation / CDK	AWS-native
Ansible	Configuration management of machines

```
// Bicep: a storage account
resource sa 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'myappstore${uniqueString(resourceGroup().id)}'
  location: resourceGroup().location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}
```

Kubernetes in one page

Concept	Plain meaning
Cluster / Node	A set of machines (nodes) running your containers
Pod	Smallest unit: one or more containers that share network/storage
Deployment	Declares 'run N replicas of this pod'; handles rolling updates and self-healing
Service	Stable network address/load balancer in front of changing pods
Ingress	HTTP routing from outside the cluster to Services
ConfigMap / Secret	Configuration and sensitive values injected into pods
Namespace	Logical partition of a cluster
HPA	Horizontal Pod Autoscaler: scales pods by CPU/metrics
Probes	`liveness` (restart me?), `readiness` (send me traffic?), `startup`

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: myapp }
spec:
  replicas: 3
  selector: { matchLabels: { app: myapp } }
  template:
    metadata: { labels: { app: myapp } }
    spec:
      containers:
        - name: api
          image: myregistry.azurecr.io/myapp:abc123
          ports: [{ containerPort: 8080 }]
          readinessProbe: { httpGet: { path: /health, port: 8080 } }
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "500m", memory: "256Mi" }
```

KEY IDEA
You often do not need Kubernetes. Managed options (Azure App Service, Container Apps, AWS ECS/Fargate, App Runner) run containers with far less operational burden. Pick Kubernetes when you need its flexibility and have people to run it.

Cloud basics

Model	You manage	Examples
IaaS	OS, runtime, app	Virtual machines (Azure VM, EC2)
PaaS	App and data only	App Service, Elastic Beanstalk, managed databases
Containers as a service	Container image	Azure Container Apps, ECS/Fargate
FaaS / serverless	Just functions	Azure Functions, AWS Lambda (pay per execution, scale to zero)
SaaS	Nothing (just use it)	Microsoft 365, GitHub

- **Regions and availability zones**: deploy across zones for resilience; across regions for disaster recovery.
- **Managed identity / IAM roles**: give workloads identities instead of passwords.
- **Cost awareness**: set budgets and alerts; right-size; delete unused resources; use autoscaling and reserved/spot pricing where it fits.
- **Shared responsibility**: the cloud provider secures the infrastructure; you secure your configuration, identities, and data.

Monitoring and incident response

- **Golden signals**: latency, traffic, errors, saturation.
- Alert on user-facing symptoms, route alerts to an on-call owner, and attach **runbooks** (step-by-step fixes).
- During an incident: **mitigate first** (roll back, disable the flag, scale up), diagnose second.
- After: a **blameless postmortem** with timeline, root causes (plural), and action items.

Twelve-factor app (cloud-friendly design in brief)

Principle	Meaning
Config in environment	No environment-specific values baked into the image
Stateless processes	Keep state in backing services, not in instance memory/disk
Logs as streams	Write to stdout; platform collects them
Disposability	Fast startup, graceful shutdown (handle SIGTERM)
Dev/prod parity	Keep environments as similar as possible

Principle	Meaning
Backing services as resources	DB, cache, queue attached by URL/config; swappable
Build, release, run separated	Immutable artifacts promoted through stages

Practice

1. Explain the difference between an image and a container, and why multi-stage builds are used.
2. Describe what happens in your pipeline from `git push` to the app serving traffic.
3. Blue/green vs canary: when would you choose each?
4. Why use OIDC federation from GitHub Actions to Azure instead of a stored secret?
5. How do you deploy a breaking database change without downtime?
6. Rebase vs merge: when is each appropriate?

16. TypeScript and React Essentials

KEY IDEA
As a C# developer, TypeScript will feel familiar: it is a typed superset of JavaScript designed by the same lead architect as C#. React adds a way to build UIs from small functions. This chapter covers the parts you use daily.

JavaScript facts that surprise C# developers

Topic	JavaScript/TypeScript behavior	C# comparison
Equality	Use `===` (strict). `==` coerces types (`0 == '0'` is true)	`==` is type-safe
Falsy values	`false, 0, '', null, undefined, NaN` are falsy	Only `bool` is condition-able
null vs undefined	`undefined` = never assigned; `null` = intentionally empty	Only `null`
Numbers	One `number` type (64-bit float); `BigInt` for huge ints	int, long, decimal...
Variables	`const` (no reassign), `let` (block scoped); avoid `var`	`readonly` / normal locals
Objects	Open bags of properties; compared by reference	Classes; value semantics need records
Async	Promises + `async/await`; single-threaded event loop	Tasks + threads
`this`	Depends on how a function is called; arrow functions capture it lexically	Always the instance
Arrays	Dynamic; `map/filter/reduce` are LINQ-like	`List<T>` + LINQ

TypeScript essentials

```
// Types
let name: string = "Ana";
let age: number = 30;
let tags: string[] = ["a", "b"];
let pair: [string, number] = ["x", 1];          // tuple
let maybe: string | undefined;                   // union
// string literal union (like an enum, but lighter)
type Status = "pending" | "paid" | "shipped";

// Interfaces and type aliases describe shapes
interface User {
  id: number;
  email: string;
  nickname?: string;          // optional
  readonly createdAt: Date;
}
type OrderLine = { sku: string; qty: number };

// Functions
function total(lines: OrderLine[]): number {
  return lines.reduce((sum, l) => sum + l.qty, 0);
}
const double = (n: number): number => n * 2;

// Generics
function first<T>(xs: T[]): T | undefined { return xs[0]; }
```

```
interface ApiResult<T> { data: T; error?: string }

// Utility types
type UserPreview = Pick<User, "id" | "email">;
type UserUpdate = Partial<User>;
type Dict = Record<string, number>;
type NoId = Omit<User, "id">;

// Narrowing: the compiler tracks what you've checked
function show(x: string | number) {
  if (typeof x === "string") return x.toUpperCase();   // x is string here
  return x.toFixed(2);                                  // x is number here
}

// Discriminated unions: powerful state modeling
type Load =
  | { state: "loading" }
  | { state: "error"; message: string }
  | { state: "ok"; data: User[] };

function render(l: Load) {
  switch (l.state) {
    case "loading": return "...";
    case "error":   return l.message;      // compiler knows message exists here
    case "ok":      return l.data.length;
  }
}

// Optional chaining and nullish coalescing (same as C# ?. and ??)
const city = user?.address?.city ?? "Unknown";
```

TIP

Turn on `"strict": true` in tsconfig.json. It enables strict null checks (C#'s nullable reference types, but for TypeScript) and catches most real bugs. Avoid `any`; use `unknown` and narrow.

Modern JavaScript you will read daily

```
// Destructuring and spread
const { id, email } = user;
const [first, ...rest] = items;
const updated = { ...user, email: "new@x.com" };      // copy with change (immutable update)
const merged = [...a, ...b];

// Array methods (LINQ equivalents)
items.filter(x => x.active)          // Where
     .map(x => x.name)               // Select
     .find(x => x.id === 3)          // FirstOrDefault
     .some(x => x.qty > 0)           // Any
     .every(x => x.qty > 0)          // All
     .reduce((s, x) => s + x.qty, 0) // Aggregate/Sum
     .sort((a, b) => a.n - b.n)      // OrderBy (mutates! copy first: [...items].sort)
     .includes(5)                    // Contains

// Template strings
const msg = `Hello ${user.name}, you have ${count} items`;

// Promises and async/await
async function load() {
  try {
    const res = await fetch("/api/orders");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);   // fetch does NOT throw on 404/500
    return (await res.json()) as Order[];
  } catch (e) { console.error(e); throw e; }
```

```
}
await Promise.all([load(), loadUser()]);                    // like Task.WhenAll

// Modules
export function helper() {}
export default App;
import App, { helper } from "./App";
```

React: the mental model

React UIs are a **function of state**: `UI = f(state)`. You describe what the screen should look like for the current data; when the data changes, React re-runs your function and updates the DOM efficiently (via a virtual DOM diff). You never manually poke the DOM.

- **Component**: a function that returns JSX (HTML-like syntax).
- **Props**: inputs passed from parent to child (read-only), like constructor/method parameters.
- **State**: data that changes over time and triggers re-render when updated.
- **Effects**: code that synchronizes with something outside React (network, timers, subscriptions).
- **One-way data flow**: data goes down via props; events go up via callbacks.

```
type CounterProps = { initial?: number; label: string };

export function Counter({ initial = 0, label }: CounterProps) {
  const [count, setCount] = useState(initial);         // state

  return (
    <div>
      <p>{label}: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>+1</button>   {/* functional update */}
    </div>
  );
}

<Counter label="Clicks" initial={5} />
```

Hooks you must know

Hook	Purpose	Rule of thumb
`useState`	Local state	Update immutably; use functional form when new value depends on old
`useEffect`	Side effects after render (fetch, subscribe)	List all reactive dependencies; return a cleanup function
`useRef`	Mutable box that survives renders / DOM element reference	Changing it does NOT re-render
`useMemo`	Cache an expensive computed value	Only when profiling shows need
`useCallback`	Cache a function identity	Useful when passing to memoized children
`useContext`	Read shared data without prop drilling	Good for theme/auth; not for rapidly changing data
`useReducer`	State with complex transitions	Like a tiny state machine/command pattern
Custom hooks	Reuse stateful logic: `useOrders()`	Name starts with `use`

```
function OrderList({ customerId }: { customerId: number }) {
  const [orders, setOrders] = useState<Order[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // cancel if component unmounts / id changes
    const controller = new AbortController();
    setLoading(true);
    fetch(`/api/customers/${customerId}/orders`, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error(`HTTP ${r.status}`); return r.json(); })
      .then(setOrders)
      .catch(e => { if (e.name !== "AbortError") setError(e.message); })
      .finally(() => setLoading(false));
    return () => controller.abort();                          // cleanup
  }, [customerId]);                                           // re-run when customerId changes

  if (loading) return <Spinner />;
  if (error) return <p role="alert">{error}</p>;
  // key is required in lists
  return <ul>{orders.map(o => <li key={o.id}>{o.id}: {o.total}</li>)}</ul>;
}
```

The rules of hooks

- Call hooks only at the **top level** of a component or custom hook (not in loops, conditions, or nested functions).
- Call hooks only from React functions.
- State updates are **asynchronous/batched**: `setCount(count + 1)` twice still adds 1 with a stale `count`; use `setCount(c => c + 1)`.
- Never mutate state directly. `items.push(x); setItems(items)` will not re-render. Use `setItems([...items, x])`.

Rendering patterns

```
// Conditional rendering
{isAdmin && <AdminPanel />}
{user ? <Welcome user={user} /> : <Login />}

// Lists need stable, unique keys (not array index if the list can reorder)
{items.map(i => <Row key={i.id} item={i} />)}

// Forms: controlled inputs
const [email, setEmail] = useState("");
<input value={email} onChange={e => setEmail(e.target.value)} />

<form onSubmit={async e => { e.preventDefault(); await save({ email }); }}>
  <button type="submit" disabled={!email}>Save</button>
</form>

// Lifting state up: put shared state in the nearest common parent, pass down as props
// Composition: children prop
function Card({ title, children }: { title: string; children: React.ReactNode }) {
  return <section><h2>{title}</h2>{children}</section>;
}
```

Data fetching and state management

Kind of state	Tool	Notes
Local UI state	`useState`, `useReducer`	Default choice
Server state (data from API)	TanStack Query (React Query), SWR, RTK Query	Caching, refetch, loading/error states, invalidation: do not hand-roll
Global client state	Context, Zustand, Redux Toolkit	Only for truly shared client state (auth, theme, cart)
URL state	React Router params / search params	Filters, pagination, selected id: shareable
Form state	React Hook Form + Zod	Validation schemas shared with types

```
// TanStack Query: caching + dedupe + background refetch for free
const { data, isPending, error } = useQuery({
  queryKey: ["orders", customerId],
  queryFn: () => api.get<Order[]>(`/customers/${customerId}/orders`),
});

const qc = useQueryClient();
const create = useMutation({
  mutationFn: (o: NewOrder) => api.post("/orders", o),
  onSuccess: () => qc.invalidateQueries({ queryKey: ["orders"] }),   // refetch stale lists
});
```

Typing an API client against your ASP.NET Core backend

```
// Keep client types in sync with the API (generate from OpenAPI with openapi-typescript /
    NSwag)
export async function api<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`/api${path}`, {
    ...init,
    headers: { "Content-Type": "application/json", ...init?.headers },
    credentials: "include",                     // send cookies when using cookie auth
  });
  if (!res.ok) {
    // ProblemDetails from ASP.NET Core
    const problem = await res.json().catch(() => null);
    throw new Error(problem?.title ?? `HTTP ${res.status}`);
  }
  return res.status === 204 ? (undefined as T) : res.json();
}
```

Performance, accessibility, and testing

- **Avoid unnecessary re-renders**: keep state close to where it is used; split components; use `React.memo` only after measuring. React DevTools Profiler shows what rendered and why.
- **Code-split** big routes with `React.lazy` + `Suspense`; use virtualized lists for thousands of rows.
- **Accessibility**: use semantic HTML (`button`, `nav`, `label`), alt text, keyboard focus, sufficient contrast, ARIA only when native HTML can't do it.
- **Testing**: Vitest/Jest + React Testing Library (test what the user sees: `screen.getByRole('button', {name: /save/i})`), Playwright for end-to-end.

- **Security**: React escapes by default; avoid `dangerouslySetInnerHTML`; do not store long-lived tokens in `localStorage` if you can use `HttpOnly` cookies.

```
test("shows error when email is empty", async () => {
  render(<SignupForm />);
  await userEvent.click(screen.getByRole("button", { name: /sign up/i }));
  expect(await screen.findByRole("alert")).toHaveTextContent(/email is required/i);
});
```

Tooling quick map

Tool	Role
Node.js + npm/pnpm	JavaScript runtime and package manager (package.json = .csproj)
Vite	Fast dev server and bundler (replaced Create React App)
Next.js	React framework with server rendering, routing, API routes
ESLint + Prettier	Linting and formatting
Express	Minimal Node web framework (you used it with PostgreSQL)
Tailwind CSS	Utility-first CSS
Zod	Runtime schema validation that infers TypeScript types

Express + PostgreSQL in 20 lines (to refresh your citation app stack)

```
import express from "express";
import { Pool } from "pg";

const app = express();
app.use(express.json());
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

app.get("/api/citations/:id", async (req, res, next) => {
  try {
    const { rows } = await pool.query("SELECT * FROM citations WHERE id = $1", [req.params.id]);
         // parameterized!
    if (rows.length === 0) return res.status(404).json({ title: "Not found" });
    res.json(rows[0]);
  } catch (err) { next(err); }
});

app.use((err: Error, _req: express.Request, res: express.Response, _next: express.NextFunction)
    => {
  console.error(err);
  res.status(500).json({ title: "Unexpected error" });          // do not leak details
});
app.listen(3000);
```

Practice

1. Why does `setCount(count + 1)` twice in one handler only add one? How do you fix it?
2. What is the dependency array in `useEffect`, and what happens if you leave it out?
3. What is the difference between props and state?
4. Why is `key` needed in lists and why is the array index a poor key for reorderable lists?
5. Why use TanStack Query instead of `useEffect` + `fetch` everywhere?
6. TypeScript: what is a discriminated union and why is it good for UI states?

17. Data Structures and Algorithms Refresher

KEY IDEA
You do not need competitive-programming skill for most jobs, but interviews and performance tuning both lean on a core toolkit. This chapter gives the patterns, each with a plain explanation and a C# solution.

Choosing the right data structure

Need	Use	Why
Fast lookup by key	`Dictionary<K,V>`	O(1) average
Uniqueness / membership	`HashSet<T>`	O(1) average
Ordered by insertion, index access	`List<T>`	Cache-friendly array
First-in, first-out	`Queue<T>`	BFS, job queues
Last-in, first-out	`Stack<T>`	DFS, undo, parsing
Always get the smallest/largest	`PriorityQueue<T,P>`	O(log n) push/pop; top-K, scheduling
Sorted data with range queries	`SortedSet<T>`, `SortedDictionary`	O(log n), ordered iteration
Prefix search / autocomplete	Trie	Walk characters, shared prefixes
Connectivity / grouping	Union-Find (disjoint set)	Near O(1) merge/find
Probabilistic membership at scale	Bloom filter	Tiny memory, false positives allowed

Complexity cheat sheet

Structure	Access	Search	Insert	Delete
Array / List	O(1)	O(n)	O(n) middle, O(1) end	O(n)
Linked list	O(n)	O(n)	O(1) with node	O(1) with node
Hash table	n/a	O(1) avg	O(1) avg	O(1) avg
Balanced BST / SortedSet	O(log n)	O(log n)	O(log n)	O(log n)
Heap / PriorityQueue	peek O(1)	O(n)	O(log n)	O(log n)
Stack / Queue	n/a	O(n)	O(1)	O(1)

Algorithm	Time	Notes
Binary search	O(log n)	Sorted input required
Quicksort (average)	O(n log n)	Worst O(n squared); in-place; `List.Sort` uses introsort

Algorithm	Time	Notes
Merge sort	O(n log n)	Stable; extra memory
Counting / radix sort	O(n + k)	Special cases (small integer range)
BFS / DFS	O(V + E)	Graph traversal
Dijkstra	O((V + E) log V)	Shortest path, non-negative weights

Pattern 1: Hash map for lookups

Two-sum: find two numbers adding to a target. Brute force checks all pairs: O(n squared). Instead, remember each number you have seen and check whether its complement exists: O(n).

```
public static (int, int)? TwoSum(int[] nums, int target)
{
    var seen = new Dictionary<int, int>();           // value -> index
    for (int i = 0; i < nums.Length; i++)
    {
        int need = target - nums[i];
        if (seen.TryGetValue(need, out int j)) return (j, i);
        seen[nums[i]] = i;
    }
    return null;
}
```

Pattern 2: Two pointers

Use two indexes moving through a sorted array or string. **Example: is a string a palindrome (ignoring non-letters)?**

```
public static bool IsPalindrome(string s)
{
    int l = 0, r = s.Length - 1;
    while (l < r)
    {
        while (l < r && !char.IsLetterOrDigit(s[l])) l++;
        while (l < r && !char.IsLetterOrDigit(s[r])) r--;
        if (char.ToLowerInvariant(s[l]) != char.ToLowerInvariant(s[r])) return false;
        l++; r--;
    }
    return true;
}
```

Pattern 3: Sliding window

Maintain a window over a sequence and slide it, updating state incrementally instead of recomputing.
Longest substring without repeating characters:

```
public static int LongestUnique(string s)
{
    var lastSeen = new Dictionary<char, int>();
    int best = 0, start = 0;
    for (int i = 0; i < s.Length; i++)
    {
        if (lastSeen.TryGetValue(s[i], out int prev) && prev >= start)
            start = prev + 1;                       // shrink window past the duplicate
        lastSeen[s[i]] = i;
```

```
        best = Math.Max(best, i - start + 1);
    }
    return best;
}
```

Pattern 4: Binary search

```
public static int BinarySearch(int[] sorted, int target)
{
    int lo = 0, hi = sorted.Length - 1;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;               // avoids overflow of (lo + hi)
        if (sorted[mid] == target) return mid;
        if (sorted[mid] < target) lo = mid + 1; else hi = mid - 1;
    }
    return -1;
}
// Also useful: "binary search on the answer" (e.g., minimum capacity that works)
```

Pattern 5: Stack

Valid parentheses: a closing bracket must match the most recent unmatched opening one: exactly what a stack tracks.

```
public static bool ValidBrackets(string s)
{
    var pairs = new Dictionary<char, char> { [')'] = '(', [']'] = '[', ['}'] = '{' };
    var stack = new Stack<char>();
    foreach (char c in s)
    {
        if (pairs.ContainsValue(c)) stack.Push(c);
        else if (pairs.TryGetValue(c, out var open))
        {
            if (stack.Count == 0 || stack.Pop() != open) return false;
        }
    }
    return stack.Count == 0;
}
```

Pattern 6: BFS and DFS on graphs

A **graph** is nodes plus edges. **BFS** (queue) explores level by level and finds the **shortest path in an unweighted graph**. **DFS** (stack or recursion) goes deep first; good for exploring all paths, cycle detection, and connected components.

```
// Graph as adjacency list
var graph = new Dictionary<int, List<int>>
{
    [1] = [2, 3], [2] = [4], [3] = [4], [4] = []
};

public static int ShortestPath(Dictionary<int, List<int>> g, int start, int goal)
{
    var queue = new Queue<(int node, int dist)>();
    var visited = new HashSet<int> { start };
    queue.Enqueue((start, 0));
    while (queue.Count > 0)
    {
```

```
        var (node, dist) = queue.Dequeue();
        if (node == goal) return dist;
        foreach (var next in g[node])
            // Add returns false if already present
            if (visited.Add(next)) queue.Enqueue((next, dist + 1));
    }
    return -1;
}

// DFS (recursive)
static void Dfs(Dictionary<int, List<int>> g, int node, HashSet<int> visited)
{
    if (!visited.Add(node)) return;
    foreach (var next in g[node]) Dfs(g, next, visited);
}
```

Pattern 7: Recursion and memoization (dynamic programming)

Recursion: a function that calls itself on a smaller problem, with a base case. **Memoization**: cache results so overlapping subproblems are computed once. Naive Fibonacci is exponential; with a cache it is linear.

```
static long Fib(int n, Dictionary<int, long>? memo = null)
{
    memo ??= new();
    if (n < 2) return n;
    if (memo.TryGetValue(n, out var v)) return v;
    return memo[n] = Fib(n - 1, memo) + Fib(n - 2, memo);
}

// Bottom-up version (tabulation): O(n) time, O(1) space
static long FibIter(int n)
{
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) (a, b) = (b, a + b);
    return a;
}

// Coin change: fewest coins to make an amount (classic DP)
static int MinCoins(int[] coins, int amount)
{
    var dp = new int[amount + 1];
    Array.Fill(dp, int.MaxValue); dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        foreach (var c in coins)
            if (c <= a && dp[a - c] != int.MaxValue)
                dp[a] = Math.Min(dp[a], dp[a - c] + 1);
    return dp[amount] == int.MaxValue ? -1 : dp[amount];
}
```

KEY IDEA

Recognize DP when: (1) you can break the problem into smaller overlapping subproblems and (2) the best answer for the whole is built from best answers of parts. Define the state ('dp[i] = best answer for the first i items'), the transition, and the base case.

Pattern 8: Priority queue (top-K, scheduling)

```
// Top K frequent words: min-heap of size K keeps the K best seen so far
public static List<string> TopK(IEnumerable<string> words, int k)
{
    var counts = words.GroupBy(w => w).ToDictionary(g => g.Key, g => g.Count());
    var heap = new PriorityQueue<string, int>();           // element, priority (smallest first)
    foreach (var (word, c) in counts)
    {
        heap.Enqueue(word, c);
        if (heap.Count > k) heap.Dequeue();                // drop the smallest
    }
    var result = new List<string>();
    while (heap.Count > 0) result.Add(heap.Dequeue());
    result.Reverse();
    return result;                                         // O(n log k)
}
```

Pattern 9: Sorting + intervals

```
// Merge overlapping intervals (meeting rooms, schedule conflicts: relevant to your scheduler!)
public static List<(int s, int e)> Merge(List<(int s, int e)> xs)
{
    var sorted = xs.OrderBy(x => x.s).ToList();
    var result = new List<(int s, int e)>();
    foreach (var cur in sorted)
    {
        if (result.Count > 0 && cur.s <= result[^1].e)
            result[^1] = (result[^1].s, Math.Max(result[^1].e, cur.e));   // overlap: extend
        else result.Add(cur);
    }
    return result;
}

// Do two time slots conflict? (the core of schedule conflict detection)
static bool Overlaps(DateTime aStart, DateTime aEnd, DateTime bStart, DateTime bEnd)
    => aStart < bEnd && bStart < aEnd;
```

Pattern 10: Linked list and trees

```
public class Node<T>(T value) { public T Value = value; public Node<T>? Next; }

// Reverse a singly linked list in place
public static Node<int>? Reverse(Node<int>? head)
{
    Node<int>? prev = null;
    while (head != null)
    {
        var next = head.Next;
        head.Next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

public class TreeNode { public int Val; public TreeNode? Left, Right; }

// Tree depth (recursion mirrors tree structure)
static int Depth(TreeNode? n) => n == null ? 0 : 1 + Math.Max(Depth(n.Left), Depth(n.Right));

// In-order traversal of a BST yields sorted values
```

```
static void InOrder(TreeNode? n, List<int> acc)
{
    if (n == null) return;
    InOrder(n.Left, acc); acc.Add(n.Val); InOrder(n.Right, acc);
}
```

Pattern 11: Caching with LRU

A **Least Recently Used** cache evicts the oldest-used item when full. Combine a `Dictionary` (O(1) lookup) with a `LinkedList` (O(1) move to front).

```
public class LruCache<K, V> where K : notnull
{
    private readonly int _cap;
    private readonly Dictionary<K, LinkedListNode<(K key, V val)>> _map = new();
    private readonly LinkedList<(K key, V val)> _order = new();      // front = most recent

    public LruCache(int capacity) => _cap = capacity;

    public bool TryGet(K key, out V? value)
    {
        if (_map.TryGetValue(key, out var node))
        {
            _order.Remove(node); _order.AddFirst(node);              // mark as recently used
            value = node.Value.val; return true;
        }
        value = default; return false;
    }

    public void Put(K key, V val)
    {
        if (_map.TryGetValue(key, out var existing)) { _order.Remove(existing);
            _map.Remove(key); }
        else if (_map.Count >= _cap)
        {
            // evict LRU
            var last = _order.Last!; _order.RemoveLast(); _map.Remove(last.Value.key);
        }
        var node = new LinkedListNode<(K, V)>((key, val));
        _order.AddFirst(node); _map[key] = node;
    }
}
```

Problem-solving approach for interviews

1. **Restate** the problem and ask clarifying questions (input size, duplicates, negative numbers, empty input, sorted?).
2. **Work an example** by hand, including an edge case.
3. **State the brute force** and its complexity. This is a fine starting point.
4. **Find the bottleneck** and pick a pattern/data structure that removes it.
5. **Code it cleanly**, narrating decisions. Use meaningful names.
6. **Test with your examples** plus edge cases (empty, one element, duplicates, large).
7. **State final time and space complexity.**

If you see...	Think...
Find pair / count occurrences / duplicates	Hash map or set

If you see...	Think...
Sorted array	Binary search, two pointers
Substring / subarray with a constraint	Sliding window
Matching / nesting / undo	Stack
Shortest path (unweighted), levels	BFS
All combinations / permutations / paths	Backtracking / DFS
Optimal value with overlapping subproblems	Dynamic programming
Top K / kth largest / merge K lists	Heap (priority queue)
Intervals / scheduling	Sort by start (or end), sweep
Prefix lookups	Trie
Connected groups	Union-Find or DFS/BFS

Practice

1. Implement `IsAnagram(a, b)` in O(n). (Count characters in an array/dictionary.)
2. Find the first non-repeating character in a string.
3. Given a sorted array, remove duplicates in place and return the new length.
4. Detect a cycle in a linked list. (Fast and slow pointers.)
5. Given course prerequisites, determine if all courses can be finished. (Topological sort / cycle detection in a directed graph.)
6. Design a function that, given professors' availability intervals, finds common free slots.

18. Interview Q&A, Cheat Sheets, and Study Plan

KEY IDEA
Use this last chapter for quick review. The questions below are the ones .NET interviewers really ask; the answers are compact so you can say them out loud in under a minute.

C# and .NET: rapid-fire answers

Question	Short answer
Value type vs reference type?	Value types copy their data on assignment (int, struct, enum). Reference types copy the pointer, so variables share one heap object (class, string, array, record).
Why is `string` immutable and what's the cost?	Safety and thread-safety, enabling interning. Every modification allocates a new string; use `StringBuilder` for repeated appends.
`struct` vs `class`?	Struct = value type, copied, no inheritance, good for small immutable data. Class = reference type, identity, inheritance, allocated on heap.
`abstract class` vs `interface`?	Abstract class shares code and state in a hierarchy (single inheritance). Interface is a contract a type can implement many of. Prefer interfaces for dependencies.
`const` vs `readonly`?	`const` is a compile-time literal baked into callers. `readonly` is set at runtime in the constructor/initializer.
`ref` vs `out`?	`ref`: variable must be initialized by caller, callee may change it. `out`: callee must assign it; used to return extra values.
What is boxing?	Wrapping a value type in an `object` on the heap. Allocation + copy; avoid in hot paths (use generics).
What does `virtual`/`override`/`new` do?	`virtual` allows overriding; `override` replaces it (runtime dispatch); `new` hides the base member (no polymorphism). Hiding is usually a bug.
Explain `IEnumerable` vs `IQueryable`.	`IEnumerable` executes in memory with delegates. `IQueryable` builds an expression tree that a provider (EF Core) translates to SQL, so filters run in the database.
What is deferred execution?	LINQ queries run when enumerated, not when defined. Enumerating twice runs it twice; materialize with `ToList()` when needed.
What is a delegate? An event?	Delegate = type-safe function reference (`Action`, `Func`). Event = delegate with restricted access: outsiders can subscribe/unsubscribe; only the owner invokes.
`async`/`await`: what actually happens?	Compiler builds a state machine. At an incomplete `await` the method returns a Task and frees the thread; the continuation runs when the awaited operation finishes.
`Task.Run` in ASP.NET Core?	Usually unnecessary and harmful for I/O work; it just moves work to another pool thread. Use real async APIs.
Why not `.Result` / `.Wait()`?	Blocks a thread, risking deadlocks and thread-pool starvation. Go async all the way.

Question	Short answer
`lock` vs `SemaphoreSlim`?	`lock` is synchronous, can't contain `await`. `SemaphoreSlim.WaitAsync` is async-friendly and can limit N concurrent entrants.
How does GC work?	Generational, mark-and-compact. Gen0 collects often and cheaply; survivors promote to Gen1 then Gen2. Large objects go on the LOH.
`IDisposable` and `using`?	For deterministic release of unmanaged resources (handles, connections). `using` calls `Dispose` even on exceptions.
`==` vs `.Equals()` vs `ReferenceEquals`?	`==` can be overloaded (strings compare contents). `Equals` is virtual value-equality. `ReferenceEquals` checks identity.
`record` vs `class`?	Record gives value-based equality, `with` copies, and concise syntax. Ideal for DTOs, messages, value objects.
Nullable reference types?	Compiler annotations (`string?`) and flow analysis warning about possible null dereferences. Not runtime enforcement.
What is dependency injection?	Objects receive their dependencies from outside (usually via constructor) instead of creating them, making code loosely coupled and testable.
Service lifetimes?	Transient: new each time. Scoped: one per request. Singleton: one per app. Never inject scoped into singleton.
Middleware order matters because...?	Each middleware wraps the next. Exception handling goes first/outermost; authentication before authorization; routing before endpoints.
Filters vs middleware?	Middleware is for the whole HTTP pipeline. Filters run inside MVC/minimal endpoint execution and know about actions, model binding, results.
How do you handle exceptions globally?	`IExceptionHandler` / exception-handling middleware returning ProblemDetails; log with correlation id; never leak internals.
JWT vs cookie auth?	Cookie: server session, great for same-site web apps (CSRF risk). JWT: stateless bearer token for APIs/mobile; harder to revoke, protect from theft.
EF Core: tracking vs no-tracking?	Tracking detects changes for `SaveChanges`. `AsNoTracking` skips that overhead for read-only queries.
What is the N+1 problem?	One query for parents then one per child. Fix with `Include`, projection, or batched queries.
How do you prevent lost updates?	Optimistic concurrency with a row version/concurrency token; handle `DbUpdateConcurrencyException` (return 409).
Abstract factory vs factory method?	Factory method creates one product via an overridable method/class. Abstract factory creates families of related products.

Design and architecture: rapid-fire answers

Question	Short answer
SOLID in one sentence each	S: one reason to change. O: extend without modifying. L: subtypes substitutable. I: small focused interfaces. D: depend on abstractions.
Composition vs inheritance?	Prefer composition ('has-a'): more flexible and less coupled than deep inheritance trees.

Question	Short answer
When use Repository over `DbContext` directly?	When you want domain-shaped query methods hidden behind an interface or need to swap storage; otherwise DbContext is already UoW + repository.
What is CQRS?	Separate write model (commands, rules) from read model (queries, projections). Helps when reads and writes have different shapes and scale.
Monolith vs microservices?	Start with (modular) monolith; split into services only when independent scaling/deployment/team autonomy justify network and data-consistency costs.
How do you keep services consistent without distributed transactions?	Sagas with compensations, outbox pattern, idempotent consumers, eventual consistency.
Idempotency?	Doing an operation multiple times has the same effect as once. Essential for retries; use idempotency keys.
CAP theorem?	During a partition you choose consistency or availability. Pick per use case.
How do you scale a read-heavy API?	Cache (CDN/Redis), read replicas, indexes, pagination, projection, async processing for heavy work.
How would you design rate limiting?	Token bucket per key in a shared store (Redis) with atomic ops; return 429 and Retry-After.
How do you roll out risky changes?	Feature flags, canary or blue/green, expand/contract migrations, fast rollback, monitoring.

Behavioral questions: use STAR

Situation (context, one sentence) -> **Task** (your responsibility) -> **Action** (what *you* did, specific) -> **Result** (measurable outcome and what you learned). Prepare 5 stories that you can adapt. Candidates from your own resume:

Story theme	Source experience	Angle to emphasize
Owning a project end-to-end	Contract work for maintenance companies in Brazil	Requirements -> architecture -> cloud deployment -> client support; replaced paper workflows
Building under real users	Eltse Hall (Resident Assistants, Hall Director)	Iterating on feedback; role-based auth; CI/CD on every push
Security mindset	Citation app OWASP assessment	Found auth/session/data-integrity risks; how you prioritized and fixed
Complex problem solving	Course scheduling and demand forecasting	Constraints, trade-offs, validating results
Production quality	Homesteaders claims-correspondence platform	Refactoring shared modules, logging, tests, OAuth with Entra ID
Tool building / initiative	DotDet analysis platform	Calibrating to cut false positives; validation with 10+ apps
Learning quickly	Moving between VTEX, insurance, government, industrial	How you ramp up on unfamiliar domains
Handling conflict/feedback	Code reviews at VTEX / Homesteaders	Receiving feedback, changing approach, outcomes

> **TIP**
> Quantify results wherever honest: users served, time saved, defects reduced, deployments per week, queries sped up. Even small numbers beat adjectives.

Questions to ask the interviewer

- What does the path from commit to production look like? How often do you deploy?
- How do you handle on-call and incident response?
- What does good code review look like on this team?
- What's the biggest technical debt or scaling challenge right now?
- How are junior engineers mentored in the first 90 days?
- How do you measure the success of engineers on this team?

One-page C# cheat sheet

```
TYPES     int long double decimal bool char string object var dynamic    T? (nullable)
COLLECT.  List<T> Dictionary<K,V> HashSet<T> Queue<T> Stack<T> PriorityQueue<T,P>
CLASSES   class record struct enum interface abstract sealed static partial
MEMBERS   field property method ctor indexer event operator
    (public/internal/private/protected)
NULLS     ?.  ??  ??=  !  is null  is not null  ArgumentNullException.ThrowIfNull(x)
LINQ      Where Select SelectMany OrderBy GroupBy Join Any All First Single Count Sum
          ToList ToDictionary Distinct Skip Take Zip Aggregate MaxBy
ASYNC     async Task<T> await  Task.WhenAll  CancellationToken  ConfigureAwait
RESOURCES using var x = ...;   IDisposable   IAsyncDisposable  await using
PATTERNS  x is Type t   x is { Prop: > 5 }   x switch { ... }   [1, .., 3]
STRINGS   $"{x}"  """raw"""  StringBuilder  string.Join  StringComparison.Ordinal*
ERRORS    try/catch/finally  throw;  when(...)  custom exceptions  Result<T> for expected
    failures
DI        AddTransient / AddScoped / AddSingleton   ctor injection   IOptions<T>
```

One-page SQL cheat sheet

```
SELECT cols FROM t WHERE cond GROUP BY cols HAVING cond ORDER BY cols OFFSET n FETCH NEXT m ROWS
     ONLY;
JOIN: INNER (matches) | LEFT (all left) | RIGHT | FULL | CROSS
AGG:  COUNT(*) SUM AVG MIN MAX         WINDOW: ROW_NUMBER() OVER (PARTITION BY .. ORDER BY ..)
CTE:  WITH name AS (SELECT ...) SELECT ... FROM name
SET:  UNION (dedupe) | UNION ALL | INTERSECT | EXCEPT
NULL: use IS NULL / IS NOT NULL (never = NULL);  COALESCE(a, b)
TXN:  BEGIN TRAN; ...; COMMIT | ROLLBACK        INDEX: seek (fast) vs scan (slow)
EXPLAIN / execution plan: always check for scans on big tables
```

HTTP and REST cheat sheet

```
GET read | POST create | PUT replace | PATCH partial update | DELETE remove
2xx success (200, 201, 204)   3xx redirect (301, 302, 304)
4xx client error (400, 401, 403, 404, 409, 422, 429)   5xx server error (500, 502, 503, 504)
Headers: Authorization, Content-Type, Accept, Cache-Control, ETag / If-Match, Idempotency-Key,
    Retry-After
Idempotent: GET PUT DELETE   Not: POST (unless you add idempotency keys)
```

Git and Docker cheat sheet

```
git switch -c x | add -p | commit | pull --rebase | push -u origin x | revert | reset --soft |
    stash | reflog
docker build -t app . | run -p 8080:8080 app | ps | logs -f | exec -it <c> sh | compose up
    --build
Dockerfile: FROM sdk AS build -> restore -> publish -> FROM aspnet -> COPY --from=build ->
    ENTRYPOINT
```

A 12-week study plan

Consistency beats intensity. Aim for 45-60 minutes on weekdays; use weekends for building something small that uses what you studied.

Week	Focus	Chapters	Build / practice
1	C# core, collections, strings, nulls	1, 2	Write a console inventory app using Dictionary, List, LINQ
2	OOP, generics, delegates, records	3, 4	Model shapes/accounts; add events and a Result type
3	LINQ and data handling	5	Solve 15 LINQ exercises (grouping, joins, aggregates)
4	Async and concurrency	6, 7	Build a concurrent downloader with `WhenAll`, cancellation, `SemaphoreSlim`
5	ASP.NET Core	8	Small API: validation, DI, Options, global error handling, JWT auth
6	SQL and EF Core	9	Add a database; write joins by hand, then in EF; find an N+1 and fix it
7	SOLID, clean code, architecture	10	Refactor week-6 API into layers; replace a `switch` with Strategy
8	Design patterns	11	Implement Decorator (caching), Factory, Outbox in the API

Week	Focus	Chapters	Build / practice
9	Testing	12	Unit tests (xUnit + fakes), integration tests with WebApplicationFactory
10	System design + security	13, 14	Do 3 design exercises on paper; run an OWASP checklist on your API
11	DevOps + React/TS	15, 16	Dockerize + GitHub Actions deploy; build a typed React client with TanStack Query
12	DSA and mock interviews	17, 18	20 problems across patterns; 2 mock interviews (system design + coding)

KEY IDEA

Retention tip: after each section close the guide and **explain it aloud** or write it from memory (the 'Feynman technique'). Space reviews at 1 day, 1 week, and 1 month. Build small projects from scratch instead of only re-reading.

Recommended resources

- **Microsoft Learn** (learn.microsoft.com): official C#, .NET, ASP.NET Core, EF Core, Azure docs. The best free reference.
- **C# in Depth** (Jon Skeet) and **Pro C# / C# 12 in a Nutshell**: deep language references.
- **Designing Data-Intensive Applications** (Martin Kleppmann): the best book on distributed systems and databases.
- **System Design Interview** (Alex Xu): interview-oriented walk-throughs.
- **Clean Architecture** (Robert C. Martin) and **Domain-Driven Design Distilled** (Vaughn Vernon): architecture and modeling.
- **Refactoring** (Martin Fowler) and **Head First Design Patterns / Refactoring.Guru**: patterns and code smells with examples.
- **Nick Chapsas**, **Milan Jovanovic**, **Tim Corey**, **Raw Coding** (YouTube): practical modern .NET content.
- **OWASP Cheat Sheet Series** and the **OWASP Top 10**: security.
- **LeetCode / NeetCode / Exercism**: algorithm and language practice.

A final word

Forgetting syntax is normal and fixable with a reference like this one. What differentiates strong engineers is judgment: knowing the trade-offs, picking the simplest thing that works, and writing code others can change safely. You already ship real software for real users; use this guide to put names and mental models on what you do.